Lecture 1: Introduction to Data Structures

Summary of Lecture:

- Uses of Data, Information
- Introduction to Data Structures

Have you ever wondered?

What is the most important factor for any event/ task to take place?
Any task or job done involves the movement/ processing of data.

Let us see what data is and what are its uses

THE USE OF DATA

Before starting our discussion on data structures, let us first consider what data are and how they support information systems. Fig. below illustrates the roles that data play in the events of any organization. The large circle labeled "data" represents the overall data resource of the

organization. This resource is an active participant in the organization's operations and planning. The small circles represent individual elements or items of data. These elements can be considered to be raw facts. They are aggregated and summarized in various meaningful ways to form information. Decisions are made on the basis of this information. The results of decisions are actions, which in turn generate more data. These data then can be incorporated into another cycle of the decision-making process.

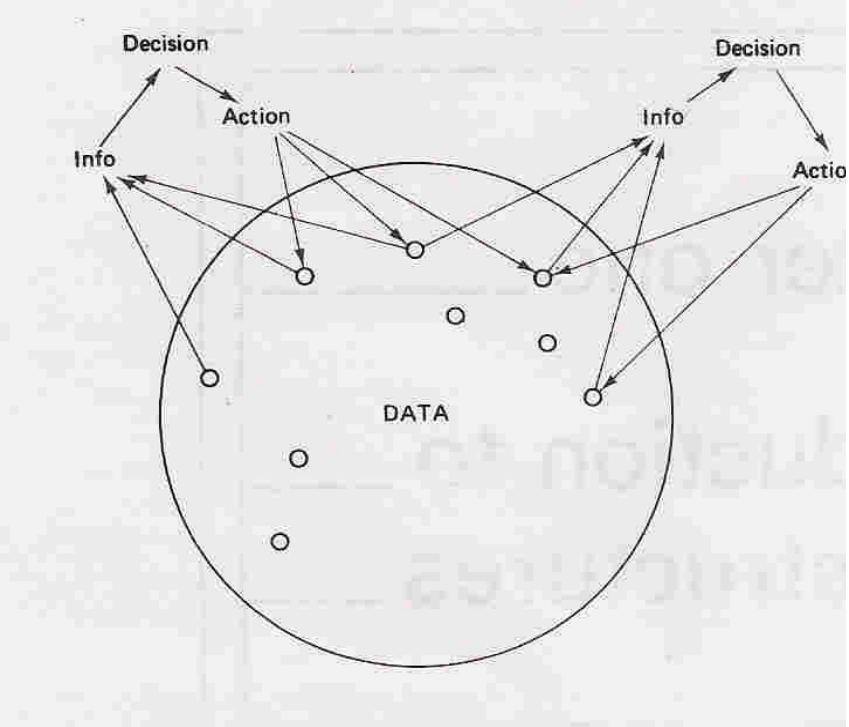


Fig: Data-An essential ingredient in taking action based on informed decision-making

Sources of Information

Any given data element may participate in the generation of multiple "pieces" of information. It

is important that there be flexibility in the ways that data items can be aggregated and summarized, so that meaningful information can be produced to support the decisions at hand. Note also that some data elements in an organization's data resource may have resulted from actions taken by forces or bodies external to the organization. The actions of competitors, the behavior of customers or clients, and the forces of legal requirements all enter into an organization's decision-making process.

Kinds of Decisions

The data resource must support several types of decision-making. It is convenient to recognize three levels of decisions:

- **Operational decisions**, which govern the daily activities of the organization.
- Control decisions, which determine the way the organization executes its designated mission. These decisions are sometimes called **tactical decisions**.
- Planning decisions, which develop and define the organization's mission. These decisions are sometimes termed **strategic decisions**.

DATA MANAGEMENT

Data are expensive. They must be managed in such a way that they are correct and are available to produce needed information. Aspects of data handling include the following:

- Measurement
- Collection
- Transcription
- Validation
- Organization
- Storage

- Aggregation
- Update
- Retrieval
- Protection

Data Management Objectives

The objective of a data management system is to make the data resource resilient, flexible, and adaptable to supporting an organization's decision-making processes. Four useful guidelines for a data management system are:-

1. Data must be represented and stored so that they can be accessed later.
2. Data must be organized so that they can be selectively and efficiently accessed.
3. Data must be processed and presented so that they support the user environment effectively.
4. Data must be protected and managed so that they retain their value.

Introduction to Data Structures

This unit comprising of this lesson and some following ones is an introductory unit and gives you an understanding of what a data structure is. Knowledge of data structures is required of people who design and develop computer programs of any kind : systems software or applications software. As you have learnt in earlier blocks, data are represented by data values held temporarily within program's data area or recorded permanently on a file. Often the different data values are related to each other. To enable programs to make use of these relationships, these data values must be in an organized form. The organized collection of data is called a data structure. The programs have to follow certain rules to access and process the structured data. We may, therefore, say data are represented that:

Data Structure = Organized Data + Allowed Operations.

If you recall, this is an extension of the concept of data type. We defined a data type as:

Data Type = Permitted Data Values + Operations

Further, we had seen that simple data type can be used to built new scalar data types, for example enumerated type in C. Similarly there are standard data structures which are often used in their own right and can form the basis for complex data structures, like Arrays, are basic building block for more complex data structures.

Designing and using data structures is an important programming skill. In this and in subsequent units, we are going to discuss various data structures.

We may classify these data structures as linear and non-linear data structures. However, this is not the only way to classify data structures. In linear data structure the data items are arranged in a linear sequence like in an array. In a non-linear, the data items are not in sequence. An example of a non-linear data structure is a tree.

Data structures may also be classified as homogenous and non- homogenous data structures. An Array is a homogenous structure in which all elements are of same type. In non-homogenous structures the elements may or may not be of the same type. Records are common example of non-homogenous data structures.

Another way of classifying data structures is as static or dynamic data structures. Static structures are ones whose sizes and structures associated memory location are fixed at compile time. Dynamic structures are ones, which expand or shrink as required during the program execution and their associated memory locations change. Records are a common example of non-homogenous data structures.

A CLASSIFICATION OF DATA STRUCTURES

Logical Data Structures

Data structures are very important in computer systems. In a program, every variable is of some explicitly or implicitly defined data structure, which determines the set of operations that are legal upon that variable. The data structures that we discuss here are termed *logical* data structures. There may be several different *physical* representations on storage possible for each logical data structure. For each data structure that we consider, several possible mappings to storage will be introduced.

Primitive and Simple Structures

Table 1 below categorizes some of the data structures that are discussed in this book. Some are *primitive*: that is, they are not composed of other data structures, examples are: integers, booleans, and characters. Other data structures can be constructed from one or more primitives. The simple data structures built from primitives are strings, arrays, and records. Many programming languages support these data structures.

Linear and Nonlinear Structures

Simple data structures can be combined in various ways to form more complex structures. The two fundamental kinds of more complex data structures are linear and nonlinear, depending on the complexity of the logical relationships they represent. The linear data structures that we will discuss include stacks, queues, and linear linked lists. The nonlinear data structures include trees and graphs. We will find that there are many types of tree structures that are useful in information systems.

File Organizations

The data structuring techniques applied to collections of data that are managed as "black boxes" by operating systems are commonly called *file organizations*. A file carries a name, contents, a location where it is kept, and some administrative information, for example, who owns it and how big it is. The four basic kinds of file organization are sequential, relative, indexed sequential, and multikey file organizations. These organizations determine how the contents of these are structured. They are built on the data structuring techniques.

Primitive data structures	Simple data structures	Compound data structures			File organizations
			Nonlinear		
		Linear	Binary	N-ary	
Integer (1) Boolean (1) Character (1)	String (1) Array (2) Record (3)	Stack (4) Queue (5) Linked list (6)	Binary Tree (8) Binary Search Tree (8)	Graph (7) General Tree (8) M-way Search Tree (14) B-Tree (14) B*, B+-Tree (14) Trie (14)	Sequential (11) Relative (13) Indexed Sequential (15) Multikey (16)

Table 1

Lecture 2: Abstract Data Types

Summary of lecture:

- Concept of ADT's
- Examples

Abstract Data Types (ADTs):

A useful tool for specifying in the logical properties of a data type is the abstract data type, or ADT. Fundamentally, a data type is a collection of values and a set of operations on those values. That collection and those operations form a mathematical construct that may be implemented using a particular hardware or software data structure. The term 'abstract data type' refers to the basic mathematical concept that defines the data type.

In defining an abstract data type as a mathematical concept, we are not concerned with space or time efficiency. Those are implementation issues. In fact, the definition of an ADT is not concerned with implementation details at all. It may not even be possible to implement a particular ADT on a particular piece of hardware or using a particular software system. For example, we have already seen that the ADT integer. Not universally implementable. Nevertheless, by specifying the mathematical and logical properties of a data type or structure, the ADT is a useful guideline to implementors and a useful tool to programmers who wish to use the data type correctly.

There are a number of methods for specifying an ADT. The method that we use is semiformal and borrows heavily from C notation but extends that notation where necessary. To illustrate the concept of an ADT and our specification method, consider the ADT RATIONAL, which corresponds to the mathematical concept of a rational number. A rational number is a number that can be expressed as the quotient of two integers. The operations of on rational numbers that

we define are the creation of a rational number from two Integers, addition, and multiplication: and-testing for equality. The following is an initial specification of this ADT:

```
        /*value definition*/
abstract typedef <integer, integer> RATIONAL;
condition RATIONAL[l] != 0;

        /*operator definition*/
abstract RATIONAL" makerational(a,b)
int a,b;
precondition b!= 0;
postcondition makerationa7[0] == a;
              makerational[1] == b;

abstract RATIONAL add(a,b)                    /* written a + b */
RATIONAL a,b;
postcondition add[l] == a[l] * b[l];
              add[O] == a[O] * b[l] + b[O] * a[l];

abstract RATIONAL mult(a,b)                   /* written a * b */
RATIONAL a, b;
postcondition mu7t[0] == aIO] * b[O];
              mu7t[1] == a[l] * b[l];



abstract equal(a,b)                           /* written a == b */
RATIONAL a,b;
postcondition equal == (a[O]*b[l] == b[O]*a[l]);
```

An ADT consists of two parts: a value definition and an operator definition. The Value definition defines the collection of values for the ADT and consists of two parts: a definition clause and a condition clause. For example, the value definition for the ADT RATIONAL states that a RATIONAL value consists of two integers, the second of which does not equal 0. Of course, the two integers that make up a rational number are the numerator and the denominator. We use array notation (square brackets) to indicate the parts of an abstract type.

The keywords **abstract typedef** introduce a value definition and the keyword **condition** is usedl to specify any conditions on the newly defined type. In this definition, the condition specifies that the denominator may not be 0. The definition clause is required, but the condition clause may not be necessary for every ADT.

Immediately following the value definition comes the operator definition. Each operator is defined as an abstract function with three parts: a header, the optional conditions, and the post conditions. For example, the operator definition of the ADT Rational includes the operations of creation (*makerational*), addition (*add*) and multiplication (*mult*), as well as a test for equality (*equal*). Let us consider the specification for multiplication first, since it is the simplest. It contains a header and post conditions, but no preconditions:

```
abstract RATIONAL mult (a,b)              /* written a*b */
RATIONAL a,b;
post condition mult[O] == a[O]*b[O];
                  mu7t[1] == a[l]*b[l];
```

The header of this definition is the first two lines, which are just like a C function header. The keyword **abstract** indicates that this is not a C function but an ADT operator definition. The comment beginning with the new keyword written indicates an alternative way of writing the function.

The post condition specifies what the operation does. In a post condition, the name of the function (in this case mult) is used to denote the result of the operation. Thus, mult[O] represents the numerator of the result, and mult[1] the denominator of the result. That is, it specifies what conditions become true after the operation is executed. In this example, the postcondition specifies that the numerator of the result of a rational multiplication equals the integer product of the numerators of the two inputs, and that the denominator equals the integer products of the two denominators.

The specification for addition (add) is straightforward and simply states that

a0/ a1 + b0/ b1 = (a0 * b1 + a 1 * b0)/ a1 *b1

The creation operation (makerational) creates a rational number from two integers and contains the first example of a precondition. In general, preconditions specify any restrictions that must be satisfied before the operation can be applied. In this example, the precondition states that makerational cannot be applied if its second parameter is 0.

The specification for equality (equal) is more significant and more complex in concept. In general, any two values in an ADT are "equal" if and only if the values of their components are equal. Indeed, it is usually assumed that an equality (and an inequality) operation exists and is defined that way, so that no explicit equal operator definition is required. The assignment operation (setting the value of one object to the value of another) is another example of an operation that is often assumed for an ADT and is not specified explicitly.

A more accurate ADT specification for RATIONAL is the following:

```
/*value definition*/
abstract typedef <int, int> RATIONAL;
condition RATIONAL[1] != 0;

/*operator definition*/
abstract equal(a,b)                                    /* written a == b*/
```

```
RATIONAL a, b;
postcondition equal == (a[O]*b[1] == b[O]*a[1]);

abstract RATIONAL makerationa1(a,b)                    /* written [a,b]*/
int a, b;
precondition b!= 0;
postcondition makerational [0] *b == a*makerational [1]

abstract RATIONAL add(a,b)                             /* written a + b */
RATIONAL a, b;
postcondition add == [a[0] * b[l] + b[0] * a[l], a[l]*b[l]]


abstract RATIONAL mu1t(a,b)                            /* written a * b */
RATIONAL a, b;
postcondition mult == [a[0] * b[0], a[l] * b[l] ]
```

Here, the equal operator is defined first, and the operator = = is extended to rational equality using the written clause. That operator is then used to specify the results of subsequent rational operations (add and mult).

The result of the make rational operation on the integers a and b produces a rational that equals a/b, but the definition does not specify the actual values of the resulting numerator and denominator. The specification for makerational also introduces the notation [a,b] for the rational formed from integers a and b, and this notation is then used in defining add and mult.

The definitions of add and mult specify that their results equal the unreduced results of the corresponding operation, but the individual components are not necessarily equal.

Notice, again, that in defining these operators we are hot specifying how they are to be computed, only what their result must be. How they are computed is determined by their implementation, not by their specification.

Review Exercises:

1. What is data? Why is it important?

2. What is a data structure? Why is such a concept required?

3. What is an abstract data type? What does it depict? How does it differ from a data structure?

Lesson 3: More on ADTs using simple examples, eg Stacks, Queues

Summary of Lecture:

- Examples of abstract data-types
- Why are they required?

The Stack as an Abstract Data Type

The representation of a stack as an abstract data type is straightforward. We use eltype to denote the type of the stack element and parameterize the stack type with eltype.

abstract typedef <<eltype>> STACK (eltype);

abstract empty(s)
STACK (eltype) s;
postcondition empty = = (len(s) = = 0);

abstract eltype pop(s)
STACK (e1type) s;
precondition empty(s) = = FALSE;
postcondition pop = = first(s');
s = = sub(s', 1, 1en(s') - 1);

abstract push(s, e1t)

STACK (eltype) s;
eltype elt;
postcondition s = = <elt> + s';

The Queue as an Abstract Data Type

The representation of a queue as an abstract data type is straightforward. We use **eltype** to denote the type of the queue element and parameterize the queue type queue **eltype**.

abstract typedef << eltype>>QUEUE(eltype);

abstract empty(q)
QUEUE (eltype) q;
postcondition empty = = (len(q) = = 0);

abstract eltype remove(q)
QUEUE (e1type) q;
precondition empty(q) = = FALSE;
postcondition remove = = first(q');
q = = sub(q' , 1, len(q') - 1);

abstract insert(q, e1t)
QUEUE (eltype) q;
eltype elt;
postcondition q = = q'+<elt>;

C Implementation of Queues

How shall a queue be represented in C? One idea is to use an array to hold elements of the queue and to two variables, front and rear, to hold the positions within the array of the first and last elements of the queue. We might declare a queue. We might declare a queue q of integers by

```
#define MAXQUEUE 100
struct queue {
        int items(MAXQUEUE];
        int front, rear;
}q;
```

Of course, using an array to hold a queue introduces the possibility of overflow if the queue should grow larger than the size of the array. Ignoring the possibility underflow and overflow for the moment, the operation insert (q, x) could be implemented by the statements

```
q.items [++q.rear] = x;
```

and the operation x = remove(q) could be implemented by

```
x = q.items [q.front++];
```

Lecture 4: Review of previous topics

Lesson 5: Data Structures- Operations

Summary of Lecture:

- Concepts of Data Structures
- Why are they required?
- Study of algorithms

DATA STRUCTURES

Data may be organized in many different ways; the logical or mathematical model of a particular organization of data is called a data structure.

Why are data structures required?

The choice of a particular data model depends on two considerations. First, it must be rich enough in structure to mirror the actual relationships of the data in the real world.
On the other hand, the structure should be simple enough that one can effectively process the data when necessary.

DATA STRUCTURE OPERATIONS

The data appearing in our data structures are processed by means of certain operations. In fact, the particular data structure that one chooses for a given situation depends largely on the frequency with which specific operations are performed.

The following are the major operations performed on data structures:

(1) **Traversing**: Accessing each record exactly once so that certain items in the record may be processed. (This accessing and processing is sometimes called "visiting" the record.)
(2) **Searching**: Finding the location of the record with a given key value, or finding the locations of all records that satisfy one or more conditions.
(3) **Inserting**: Adding a new record to the structure.
(4) **Deleting**: Removing a record from the structure.

Sometimes two or more of the operations may be used in a given situation; e.g., we may want to delete the record with a given key, which may mean we first need to search for the location of the record. The following two operations, which are used in special situations, are also be considered:

(1) **Sorting**: Arranging the records in some logical order (e.g., alphabetically according to some NAME key, or in numerical order according to some NUMBER key, such as social security number or account number)
(2) **Merging**: Combining the records in two different sorted files into a single sorted file

Other operations, e.g., **copying** and **concatenation**, are also used.

EXAMPLE :
An organization contains a membership file in which each record contains the following data for a given member:

Name, Address, Telephone Number, Age, Sex

(a) Suppose the organization wants to announce a meeting through a mailing. Then one would traverse the file to obtain Name and Address for each member.
(b) Suppose one wants to find the names of all members living in a certain area. Again one would traverse the file to obtain the data.
(c) Suppose one wants to obtain Address for a given Name. Then one would search the file for the record containing Name.

(d) Suppose a new person joins the organization. Then one would insert his or her record into the file.

(e) Suppose a member dies. Then one would delete his or her record from the file.

(f) Suppose a member has moved and has a new address and telephone number. Given the name of the member, one would first need to search for the record in the file. Then one would perform the "update"- i.e., change items in the record with the new data.

(g) Suppose one wants to find the number of members 65 or older. Again one would traverse the file, counting such members.

Lecture 6: More on to Data Structures

Summary of Lecture:

- Complexity of Algorithms
- Time-Space Tradeoff

ALGORITHMS: COMPLEXITY, TIME-SPACE TRADEOFF

An algorithm is a well-defined list of steps for solving a particular problem. One major challenge of programming is to develop efficient algorithms for the processing of our data. The time and space it uses are two major measures of the efficiency of an algorithm. The complexity of an algorithm is the function, which gives the running time and/or space in terms of the input size.
Each of our algorithms involves a particular data structure. Accordingly, we may not always be able to use the most efficient algorithm, since the choice of data structure depends on many things, including the type of data and the frequency with which various data operations are applied. Sometimes the choice of data structure involves a time-space tradeoff: by increasing the amount of space for storing the data, one may be able to reduce the time needed for processing the data, or vice versa. We illustrate these ideas with two examples. .

Searching Algorithms

Consider a membership file, in which each record contains, among other data, the name and telephone number of its member. Suppose we are given the name of a member and we want to find his or her telephone number. One way to do this is to linearly search through the file, i.e., to apply the following algorithm:

Linear Search: Search each record of the file, one at a time, until finding the given Name and hence the corresponding telephone number.

First of all, it is clear that the time required to execute the algorithm is proportional to the number of comparisons. Also, assuming that each name in the file is equally likely to be picked, it is intuitively clear that the average number of comparisons for a file with n records is equal to n/2; that is, the complexity of the linear search algorithm is given by C(n) = n/2.

The above algorithm would be impossible in practice if we were searching through a list consisting of thousands of names, as in a telephone book. However, if the names are sorted alphabetically, as in telephone books, then we can use an efficient algorithm called binary search.

Binary Search: Compare the given Name with the name in the middle of the list; this tells which half of the list contains Name. Then compare Name with the name in the middle of the correct half to determine which quarter of the list contains Name. Continue the process until finding Name in the list.

One can show that the complexity of the binary search algorithm is given by $C(n) = \log_2 n$

Thus, for example, one will not require more than 15 comparisons to find a given Name in a list. containing 25000 names.

Although the binary search algorithm is a very efficient algorithm, it has some major drawbacks. Specifically, the algorithm assumes that one has direct access to the middle name in the list or a sublist. This means that the list must be stored in some type of array. Unfortunately, inserting an element in an array requires elements to be moved down the list, and deleting an element from an array requires elements to be moved up the list.

The telephone company solves the above problem by printing a new directory every year while keeping a separate temporary file for new telephone customers. That is, the telephone company updates its files every year. On the other hand, a bank may want to insert a, new customer in its file almost instantaneously. Accordingly, a linearly sorted list may not be the best data structure for a bank.

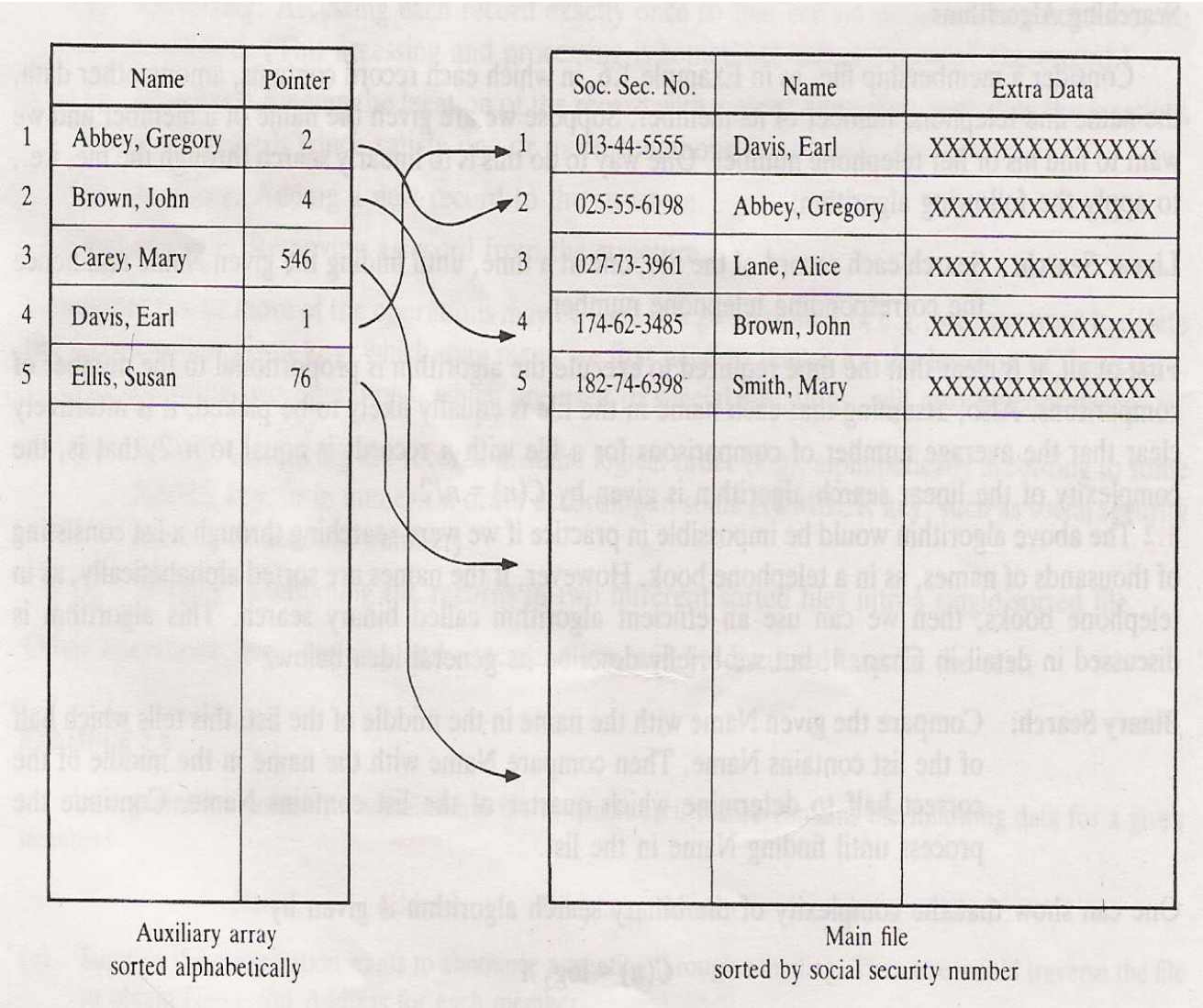
An Example of Time-Space Tradeoff

Suppose a file of records contains names, social security numbers and much additional

information among its fields. Sorting the file alphabetically and using a binary search is a very efficient way to find the record for a given name. On the other hand, suppose we are given only the social security number of the person. Then we would have to do a linear search for the record, which is extremely time-consuming for a very large number of records.

How can we solve such a problem?

One way is to have another file that is sorted numerically according to social security number. This, however, would double the space required for storing the data. Another way, pictured in Fig. below, is to have the main file sorted numerically by social security number and to have an auxiliary array with only two columns, the first column containing an alphabetized list of the names and the second column containing pointers which give the locations of the corresponding records in the main file. This is one way of solving the problem that is used frequently, since the additional space, containing only two columns, is minimal for the amount of extra information it provides.


Name
Pointer
1 Abbey, Gregory 2
2 Brown, John 4
3 Carey, Mary 546
4 Davis, Earl 1
5 Ellis, Susan 76
Auxiliary array
sorted alphabetically
Soc. Sec. No.
Name
Extra Data
1 013-44-5555 Davis, Earl XXXXXXXXXXXXXX
2 025-55-6198 Abbey, Gregory XXXXXXXXXXXXXX
3 027-73-3961 Lane, Alice XXXXXXXXXXXXXX
4 174-62-3485 Brown, John XXXXXXXXXXXXXX
5 182-74-6398 Smith, Mary XXXXXXXXXXXXXX
Main file
sorted by social security number

COMPLEXITY OF ALGORITHMS

The analysis of algorithms is a major task in computer science. In order to compare algorithms, we must have some criteria to measure the efficiency of our algorithms. This section discusses this important topic.

Suppose M is an algorithm, and suppose n is the size of the input data. The time and space used by the algorithm M are the two main measures for the efficiency of M. The time is measured by counting the number of key operations-in sorting and searching algorithms, for example, the number of comparisons. That is because key operations are so defined that the time for the other operations is much less than or at most proportional to the time for the key operations. The space is measured by counting the maximum of memory needed by the algorithm.

The **complexity** of an algorithm M is the function f(n) which gives the running time and/or storage space requirement of the algorithm in term, of the size n of the input data. Frequently, the storage space required by an algorithm is simply a multiple of the data size n. Accordingly, unless otherwise stated or implied, the term "complexity" shall refer to the running time of the algorithm.

The following example illustrates that the function f(n), which gives the running time of an algorithm, depends not only on the size n of the input data but also on the particular data.

EXAMPLE

Suppose we are given an English short story TEXT, and suppose we want to search through TEXT for the first occurrence of a given 3-1etter word W. If W is the 3-letter word "the," then it is likely that W occurs near the beginning of TEXT, so f(n) will be small. On the other hand, if W is the 3-letter word "zoo," then W may not appear in TEXT at all, so f(n) will be large.

The above discussion leads us to the question of finding the complexity function f(n) for certain cases. The two cases one usually investigates in complexity theory are as follows:

(1) Worst case: the maximum value of f(n) for any possible input

(2) Average case: the expected value of f(n)

Sometimes we also consider the minimum possible value of f(n), called the best case.

The analysis of the average case assumes a certain probabilistic distribution for the input data; one such assumption might be that all possible permutations of an input data set are equally likely. The average case also uses the following concept in probability theory. Suppose the numbers n_1 n_2, . . ., n_k occur with respective probabilities P_I' P_2' . . . , P_k' Then the expectation or average

$$E = n_1P_1, + n_2P_2 + ... + n_kP_k$$

These ideas are illustrated in the following example.

EXAMPLE : Linear Search

Suppose a linear array DATA contains n elements, and suppose a specific ITEM of information is given. We want either to find the location LOC of ITEM in the array DATA, or to send some message, such as LOC = 0, to indicate that ITEM does not appear in DATA. The linear search algorithm solves this problem by comparing ITEM, one by one, with each element in DATA. That is, we compare ITEM with DATA[l], then DATA[2], and so on, until we find LOC such that ITEM = DATA[LOC]. A formal presentation of this algorithm follows.

Algorithm: (Linear Search)

A linear array DATA with N elements and a specific ITEM &f information are given. This algorithm finds the location LOC of ITEM in the array DATA or sets LOC = O.

1. [Initialize] Set K:= land LOC:= O.
2. Repeat Steps 3 and 4 while LOC = 0 and K≤ N.
3. If ITEM = DA T A[K], then: Set LOC: = K.
4. Set K: = K + 1. [Increments counter]

[End of Step 2 loop.]

5. [Successful?]

If LOC = 0, then:

Write: ITEM is not in the array-DATA.

Else:

Write: LOC is the location of ITEM.

[End of If structure.]

6. Exit.

The complexity of the search algorithm is given by the number C of comparisons between ITEM and DATA [K]. We seek C (n) for the worst case and the average case.

Worst Case

Clearly the worst case occurs when ITEM is the last element in the array DATA or is not there at all. In either situation, we have

$$C(n)=n$$

Accordingly, C (n) = n is the worst-case complexity of the linear search algorithm.

Average Case

Here we assume that ITEM does appear in DATA, and that it is equally likely to occur at any position in the array. Accordingly, the number of comparisons can be any of the numbers 1,2,3, . . . , n, and each number occurs with probability p = 1/n. Then

$$C(n) = 1.$$

This agrees with our intuitive feeling that the average number of comparisons needed to find the location of ITEM is approximately equal to half the number of elements in the DATA list.

Review Exercises:

1. Why are stacks and queues represented as abstract data type?

2. What are the different operations performed on data structures? Why are they required?

3. What is meant by the complexity factor of an algorithm? How is it optimized?

Lecture 7: Arrays, and how different data structures implement same ADT.

Summary of Lecture:

- Concept of Arrays
- Different operations performed using Arrays

This lesson discusses a very common linear structure called all array. Since arrays are usually easy to traverse, search and sort, they are frequently used to store relatively permanent collections of data.

LINEAR ARRAYS

A ***linear array*** is a list of a finite number n of homogeneous data elements (i.e., data elements of the same type) such that:

(a) The elements of the array are referenced respectively by an *index set* consisting of n Consecutive numbers.

(b) The elements of the array are stored respectively in successive memory locations.

The number n of elements is called the *length* or *size* of the array. If not explicitly stated, we will assume the index set consists of the integers 1, 2, . . . , n. In general, the length or the number of data elements of the array can be obtained from the index set by the formula

$$\text{Length} = \text{VB} - \text{LB} + 1$$

Where UB is the largest index, called the *upper bound*, and LB is the smallest index, called the *lower bound*, of the array. Note that length = VB when LB = 1.

The elements of an array A may be denoted by the subscript notation

$$A_1\ A_2, A_3, \ldots, A_n$$

or by the parentheses notation (used in FORTRAN, PLl1 and BASIC)

$$A(l), A(2), \ldots, A(N)$$

Or by the bracket notation (used in Pascal)

$$A[l], A[2], A[3], \ldots, A[N]$$

We will usually use the subscript notation or the bracket notation. Regardless of the notation, the number K in A[K] is called a *subscript* or an *index* and A[K] is called a; ***subscripted variable***. Note that subscripts allow any element of A to be referenced by its relative position in A.

EXAMPLE

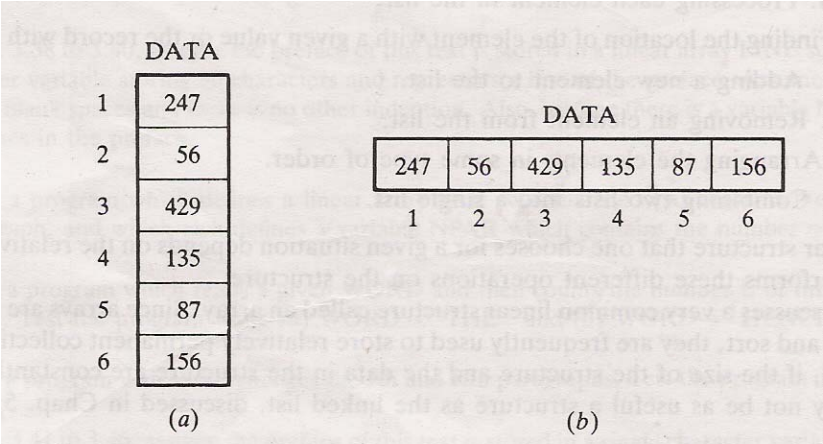
(a) Let DATA be a 6-element linear array of integers such that

DATA[1] =247 DATA[2] = 56 DA T A[3] = 429 DATA[4] = 135 DATA[5] = 87 DATA[6] = 156

Sometimes we will denote such an array by simply writing

DATA: 247, 56, 429, 135, 87, 156

The array DATA is frequently pictured as in Fig. (a) or Fig. (b).



(b) An automobile company uses an array AUTO to record the number of automobiles sold each year from 1932 through 1984. Rather than beginning the index set with 1, it is more useful to begin the index set with 1932 so that.

AUTO[K] = number of automobiles sold in the year K

Then LB = 1932 is the lower bound and UB = 1984 is the upper bound of AUTO.

Length = UB - LB + 1 = 1984 -1930 + 1 = 55

That is, AUTO contains 55 elements and its index set consists of all integers from 1932 through 1984.

Each programming language has its own rules for declaring arrays. Each such declaration must give, implicitly or explicitly, three items of information: (1) the name of the array, (2) the data type of the array and (3) the index set of the array.

REPRESENTATION OF LINEAR ARRAYS IN MEMORY

Let LA be a linear array in the memory of the computer. Recall that the memory of the computer is simply a sequence of addressed locations as pictured in Fig. below. Let us use the notation

LOC (LA [K]) = address of the element LA [K] of the array LA

As previously noted, the elements of LA are stored in successive memory cells. Accordingly, the computer does not need to keep track of the address of every element of LA, but needs to keep track only of the address of the first element of LA, denoted by

Base (LA)

and called the **base address** of LA. Using this address Base (LA), the computer calculates the address of any element of LA by the following formula:

LOC (LA [K]) = Base(LA) + w(K - lower bound) (4.2)

where w is the number of. words per memory cell for the array _A. Observe that the time to calculate LOC(LA[K]) is essentially the same for any value of K. Furthermore, given any subscript K, one can locate and access the content of LA[K] without scanning any other element of LA.

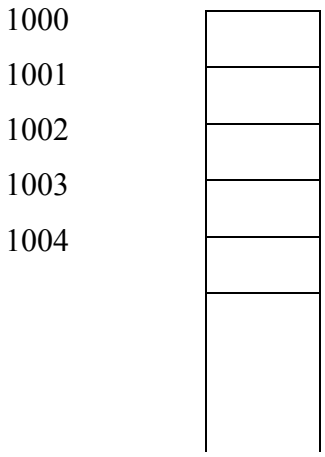


Fig: Computer memory

EXAMPLE

Consider the array AUTO in previous example, which records the number of automobiles, sold

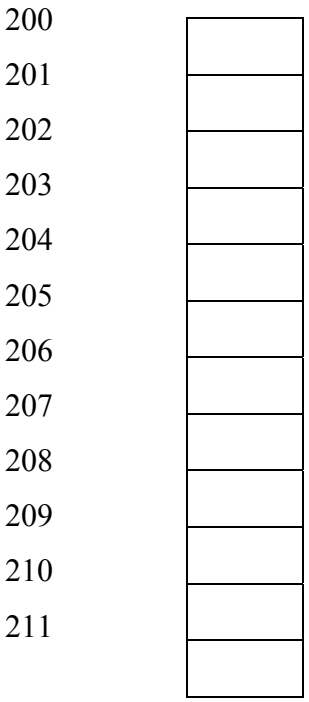
each year from 1932 through 1984. Suppose AUTO appears in memory as pictured in Fig. below. That is, Base (AUTO) = 200, and w = 4 words per memory cell for AUTO. Then

LOC(AUTO[1932]) = 200, LOC(AUTO[1933]) = 204, LOC(AUTO[1934]) = 208, . . .

The address of the array element for the year K = 1965 can be obtained as:

LOC (AUTO [1965]) = Base (AUTO) + w (1965 -lower bound) = 200 + 4(1965 - 1932) = 332

Thus, the contents of this element can be obtained without scanning any other element in array AUTO.



.

Remark: A collection A of data elements is said to be indexed if any element of A, which we shall call A_K, can be located and processed in a time that is independent of K., The above discussion indicates that linear arrays can be indexed. This is a very important property of linear arrays. In fact, linked lists, which are covered in the next section, do not have this property.

TRAVERSING LINEAR ARRAYS

Let A be a collection of data elements stored in the memory of the computer. Suppose we want to print the contents of each element of A or suppose we want to count the number of elements of A with a given property. This can be accomplished by traversing A, that is, by accessing and processing (frequently called visiting) each element of A exactly once.

The following algorithm traverses a linear array LA. The simplicity of the algorithm comes from the fact that LA is a linear structure. Other linear structures, such as linked lists, can also be easily traversed. On the other hand, the traversal of nonlinear structures, such as trees and graphs, is considerably more complicated.

Algorithm: (Traversing a Linear Array) Here LA is a linear array with lower bound LB and upper bound UB. This algorithm traverses LA applying an operation PROCESS to each element of LA.

1. [Initialize counter.] Set K:= LB.
2. Repeat Steps 3 and 4 while K≤UB.
3. [Visit element.] Apply PROCESS to LA[K].
4. [Increase counter.] Set K: = K + 1.

   [End of Step 2 loop.]
5. Exit.

Lesson 8: Insertion and Deletion in Arrays

Summary of Lecture:

- Insertion in arrays
- Deletions in arrays

INSERTING AND DELETING

Let A be a collection of data elements in the memory of the computer. "Inserting" refers to the operation 'of adding another element to the collection A, and "deleting" refers to the operation of removing one of the elements from A. This section discusses inserting and deleting when A is a linear array.

Inserting an element at the "end" of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element. On the other hand, suppose we need to insert an element in the middle of the array. Then, on the average, half of the elements must be moved downward to new locations to accommodate the new element and keep the order of the other elements.

Similarly, deleting an element at the "end" of an array presents no difficulties, but deleting an element somewhere in the middle of the array would require that each subsequent element be moved one location upward in order to "fill up" the array.

EXAMPLE

Suppose TEST has been declared to be a 5-element array but data have been recorded only

for TEST[l], TEST[2] and TEST[3]. If X is the value of the next test, then one simply assigns

TEST[4]:= X

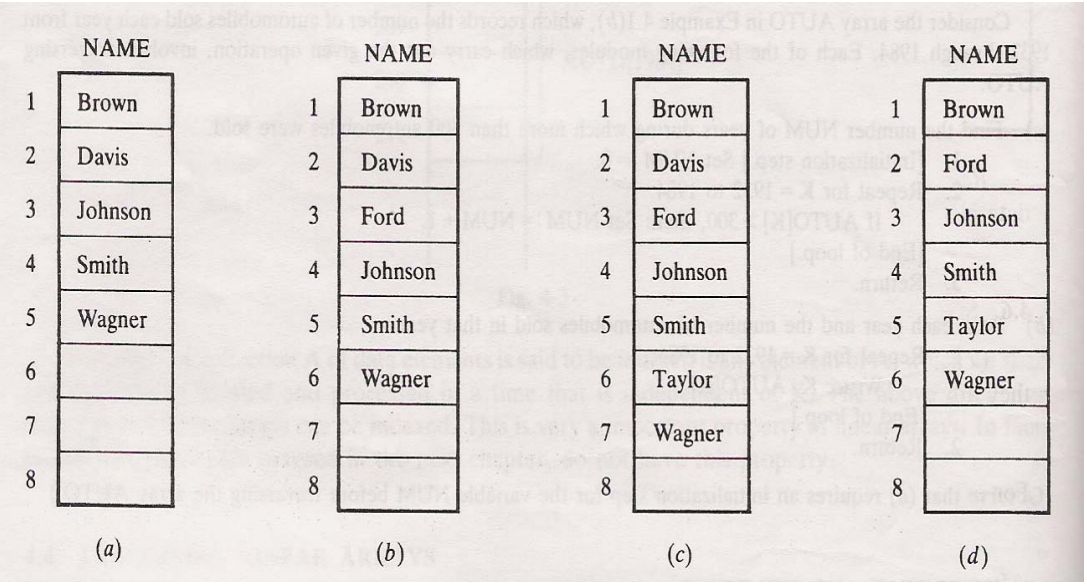
to add X to the list. Similarly, if Y is the value of the subsequent test, then we simply assign

TEST[5]:= Y

to add Y to the list. Now, however, we cannot add any new test scores to the list.

EXAMPLE

Suppose NAME is an 8-element linear array, and suppose five names are in the array, as in Fig. (a). Observe that the names are listed alphabetically, and suppose we want to keep the array names alphabetical at all times. Suppose Ford is added to the array. Then Johnson, Smith and Wagner must each be moved downward one location, as in Fig. (b). Next suppose Taylor is added to the array; then Wagner must be moved, as in Fig. (c). Last, suppose Davis is removed from the array. Then the five names Ford, Johnson, Smith, Taylor and Wagner must each be moved upward one location, as in Fig. (d). Clearly such movement of data would be very expensive if thousands of names were in the array.



The following algorithm inserts a data element ITEM into the Kth position in a linear array LA with N elements. The first four steps create space in LA by moving downward one location each element from the Kth position on. We emphasize that these elements are moved in reverse order-i.e. first LA[N], then LA[N - 1], . . . , and last LA[K];-otherwise data might be erased.

In more detail, we first set J:= N and then, using J as a counter, decrease J each time the loop is executed until J reaches K. The next step, Step 5, inserts ITEM into the array in the space just created. Before the exit from the algorithm, the number N of elements in LA is increased by 1 to account for the new element.

Algorithm: (Inserting into a Linear Array) **INSERT (LA, N, K, ITEM)**

Here LA is a linear array with N elements and K is a positive integer such that K≤ N. This algorithm inserts an element ITEM into the Kth position in LA.

1. [Initialize counter.] Set J : = N.
2. Repeat Steps 3 and 4 while J >= K.
3. [Move J^{th} element downward.] Set LA [J + 1]:= LA[J].
4. [Decrease counter.] Set J : = J - 1.
   [End of Step 2 loop.]
5. [Insert element.] Set LA [K] : = ITEM.
6 [Reset N.] Set N:= N +1.
7. Exit.

The following algorithm deletes the Kth element from a linear array LA and assigns it to a variable ITEM.

Algorithm: (Deleting from a Linear Array) **DELETE(LA, N, K_ ITEM)**

Here LA is a linear array with N elements and K is a positive integer such that K ≤ N. This

algorithm deletes the Kth element from LA.

1 Set ITEM:= LA[K].
2. Repeat for J = K to N - 1:
   [Move J + 1st element upward.] Set LA[J]:= LA[J + 1].
   [End of loop.]
3. [Reset the number N of elements in LA.] Set N:= N - 1.
4. Exit.

Lecture 9: Discussion of previous topics and their implementation in C++/ C

Lecture 10: MULTIDIMENSIONAL ARRAYS

Summary of Lecture:

- Kinds of Arrays
- Kinds of matrices with uses

The linear arrays discussed so far are also called one-dimensional arrays, since each element in the array is referenced by a single subscript. Most programming languages allow two-dimensional and three-dimensional arrays, i.e., arrays where elements are referenced, respectively, by two and three subscripts. In fact, some programming languages allow the number of dimensions for an array to be as high as 7. This section discusses these multidimensional arrays.

Two-Dimensional Arrays

A two-dimensional m x n array A is a collection of m X n data elements such that each element is specified by a pair of integers (such as J, K), called *subscripts*, with the property that

$$1 \leq J \leq m \quad \text{and} \quad 1 \leq K \leq n$$

The element of A with first subscript j and second subscript k will be denoted by

. $$A_{J,K} \quad \text{or} \quad A[J, K]$$

Two-dimensional arrays are called **matrices** in mathematics and **tables** in business applications; hence two-dimensional arrays are sometimes called **matrix arrays**. .

There is a standard way of drawing a two-dimensional m x n array A where the elements of A form a rectangular array with m rows and n columns and where the element A[J, K] appears in row J and column K. (A **row** is a horizontal list of elements, and a **column** is a vertical list of elements.) Following figure shows the case where A has 3 rows and 4 columns. Each row contains those elements with the same first subscript, and each column contains those elements with the same second subscript.

		Columns			
		1	2	3	4
Rows	1	A[1, 1]	A[1, 2]	A[1, 3]	A[1, 4]
	2	A[2, 1]	A[2, 2]	A[2, 3]	A[2, 4]
	3	A[3, 1]	A[3, 2]	A[3, 3]	A[3, 4]

Figure: 2-dimensional 3 X 4 array A

Representation of Two-Dimensional Arrays in Memory

Let A be a two-dimensional *m x n* array. Although A is pictured as a rectangular array of elements with m rows and n columns, the array will be represented in memory by a block of m.n sequential memory locations.

Specifically, the programming language will store the array A either

(1) column by column is what is called **column-major order**, or

(2) row by row, in **row-major order**. Figure below shows these two ways when A is a two-dimensional 3 x 4 array. We emphasize that the particular representation used depends upon the programming language, not the user.

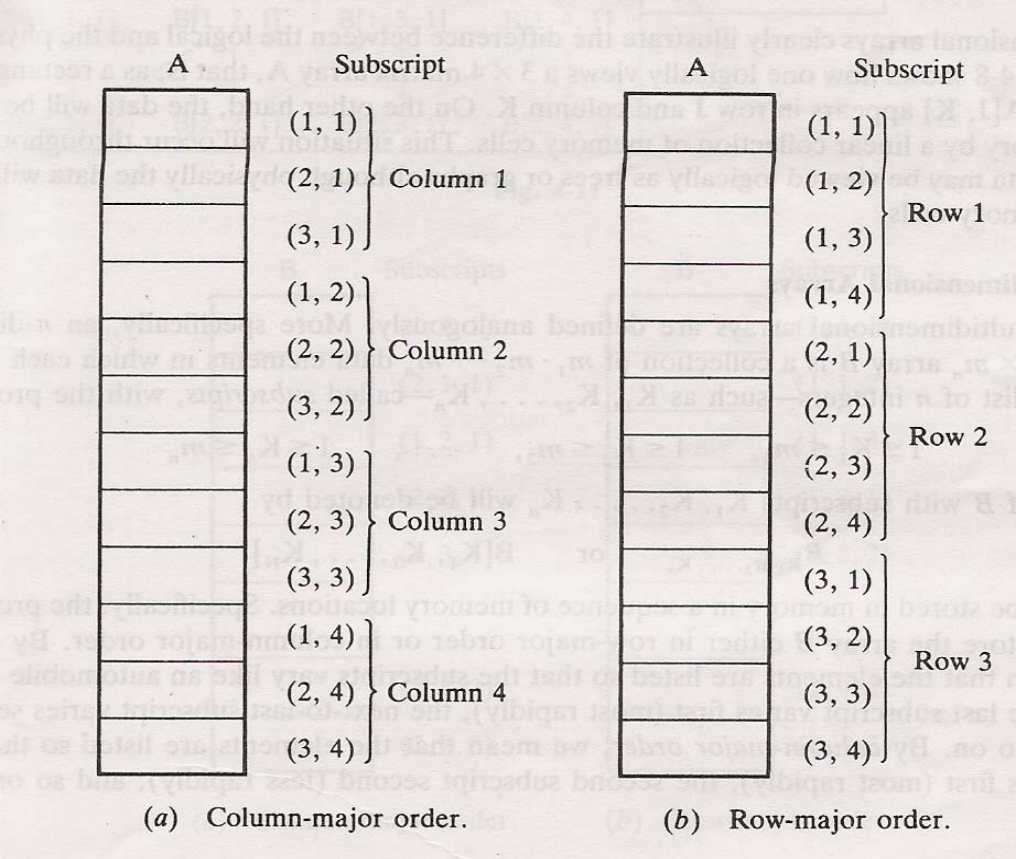


Fig: Representing a 2-D array in memory

Recall that, for a linear array LA, the computer does not keep track of the address LOC(LA[K]) of every element LA[K] of LA, but does keep track of Base(LA), the address of the first element of LA. The computer uses the formula

$$LOC(LA[K]) = Base(LA) + w(K - 1)$$

to find the address of LA[K] in time independent of K. (Here w is the number of words per memory cell for the array LA, and 1 is the lower bound of the index set of LA.)

A similar situation also holds for any two-dimensional m x n array A. That is, the computer keeps track of Base(A)-the address of the first element A[l, 1] of A-and computes the address

LOC(A[J, K]) of A[J, K] using the formula

(Column-major order) LOC(A[J, K]) = Base(A) + w[M(K -1) + (J -1)]

or the formula

(Row-major order) LOC(A[J, K]) = Base(A) + w[N(J - 1) + (K - 1)]

Again, w denotes the number of words per memory location for the array A. Note that the formulas are linear in J and K, and that one can find the address LOC(A[J, K]) in time independent of J and K.

EXAMPLE

Consider the 25 x 4 matrix array SCORE. Suppose Base (SCORE) = 200 and there are w = 4 words per memory cell. Furthermore, suppose the programming language stores two-dimensional arrays using row-major order. Then the address of SCORE [12,3], the third test of the twelfth student, follows:

LOC (SCORE [12, 3]) = 200 + 4[4(12 -1) + (3 -1)] = 200 + 4[46] = 384

General Multidimensional Arrays

General multidimensional arrays are defined analogously. More specifically, an n-dimensional m_1 x m_2 x . . . x m_n array B is a collection of m_1 . m_2 . . . m_n data elements in which each element is specified by a list of n integers-such as K_1 K_2, . . . , K_n-called subscripts, with the property that

$$1 \le K_1 \le m_1. \quad 1 \le K_2 \le m_2, \ldots\ldots\ldots\ldots\ldots 1 \le k_n \le m_n$$

The element of B with subscripts K_1, K_2, . . . , K_n will be denoted by

$B_{K1 \cdot K2 \,..\, Kn}$.or $B[K_1, K_2, \ldots, K_N]$

The array will be stored in memory in a sequence of memory locations. Specifically, the programming language will store the array B either in row-major order or in column-major order. By row-major order, we mean that the elements are listed so that the subscripts vary like an automobile odometer i.e., so that the last subscript varies first (most rapidly), the next-to-last subscript varies second (1ess rapidly), and so on. By column-major order, we mean that the elements are listed so that the first subscript varies first (most rapidly), the second subscript second (less rapidly), and so on.

MATRICES

"Vectors" and "matrices" are mathematical terms, which refer to collections of numbers, which are analogous, respectively, to linear and two-dimensional arrays: That is,

(a) An n-element vector V is a list of n numbers usually given in the form

$$V= (V_1\ V_2, \ldots, V_n)$$

(b) An m X n matrix A is an array of m.n numbers arranged in m rows and n columns as follows:

$$A = \begin{pmatrix} A_{11} & A_{12} \ldots\ldots\ldots\ldots A_{1n} \\ A_{21} & A_{22} \ldots\ldots\ldots\ldots A_{2n} \\ \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\ A_{m1} & A_{m2} \ldots\ldots\ldots\ldots\ldots A_{mn} \end{pmatrix}$$

In the context of vectors and matrices, the term **scalar** is used for individual numbers.
A matrix with one row (column) may be viewed as a vector and, similarly, a vector may be viewed as a matrix with only one row (column).

A matrix with the same number n of rows and columns is called a **square matrix** or an **n-square** matrix. The diagonal or **main diagonal** of an n-square matrix A consists of the elements $A_1, A_{22} \ldots\ldots A_{nn}$.

Algebra of Matrices

Suppose A and Bare m x n matrices. The sum of A and B, written A + B, is the m x n matrix obtained by adding corresponding elements from A and B; and the product of a scalar k and the matrix A, written k. A, is the m x n matrix obtained by multiplying each element of A by k. (Analogous operations are defined for n-element vectors.)

Matrix multiplication is best described by first defining the scalar product of two vectors. Suppose U and V are n-element vectors. Then the scalar product of U and V, written U.V, is the scalar obtained by multiplying the elements of U by the corresponding elements of V, and then adding::

$$U.\ V= U_1\ V_1 + U_2\ V_2 +\ldots\ldots + U_n\ V_n = \sum_{k=1}^{n} U_k V_k$$

It is emphasized that U. V is a scalar, not a vector.

Now suppose A is an m x p and suppose B is a p x n matrix. The product of A and B, written AB, is the m x n matrix C whose ij^{th} element Cij is given by

$$C_{ij}=A_{i1}B_{1j}+A_{i2}B_{2j}+\ldots..+A_{ip}B_{pj} = \sum_{k=1}^{p} A_{ik} B_{kj}$$

That is, Cij is equal to the scalar product of row i of A and column j of B.

SPARSE MATRICES

Matrices with a relatively high proportion of zero entries are called **sparse matrices**. Two general types of n-square sparse matrices, which occur in various applications, are pictured in Fig. below. (It is sometimes customary to omit blocks of zeros in a matrix as in fig) The first matrix, where all entries above the main diagonal are zero or, equivalently, where nonzero entries can only occur on or below the main diagonal, is called a (lower) **triangular matrix**. The second matrix, where nonzero entries can only occur on the diagonal or on elements immediately

above or below the diagonal, is called a **tridiagonal matrix**.

$$\begin{pmatrix} 4 & & & & \\ 3 & -5 & & & \\ 1 & 0 & 6 & & \\ -7 & 8 & -1 & 3 & \\ 5 & -2 & 0 & 2 & -8 \end{pmatrix} \qquad \begin{pmatrix} 5 & -3 & & & & \\ 1 & 4 & 3 & & & \\ & 9 & -3 & 6 & & \\ & & 2 & 4 & -7 & \\ & & & 3 & -1 & 0 \\ & & & & 6 & -5 & 8 \\ & & & & & 3 & -1 \end{pmatrix}$$

(a) Triangular matrix. (b) Tridiagonal matrix.

The natural method of representing matrices in memory as two-dimensional arrays may not be suitable for sparse matrices. That is, one may save space by storing only those entries, which may be nonzero. This is illustrated for triangular matrices in the following example. Other cases will be discussed in the solved problems.

TRIANGULAR ARRAY

Suppose we want to place in memory the triangular array A in fig. below. Clearly it would be wasteful to store those entries above the main diagonal of A, since we know they are all zero; hence we store only the other entries of A in a linear array B as indicated by the arrows. That is, we let

$B[I] = a_{ll}$, $B[2] = a_{21}$ $B[3] = a_{22}$, $B[3] = a_{31}$,.....

Observe first that B will contain only

$$1 + 2 + 3 + 4 + n = 1/2\ n(n + 1)$$

elements, which is about half as many elements as a two-dimensional n x n array. Since we will require the value of a_{JK} in our programs, we will want the formula that gives us the integer L in terms of J and K where

$$B[L] = a_{JK}$$

Observe that L represents the number of elements in the list up to and including $a_{JK.,}$ Now there are

$$1 + 2 + 3 + \ldots + (J - 1) = \frac{J(J - 1)}{2}$$

elements in the rows above a_{JK} and there are K elements in row J up to and including a_{JK} Accordingly,

$$L = \frac{J(J-l)}{2} + K$$

yields the index that accesses the value a_{JK} from the linear array B.

$$A = \begin{pmatrix} a_{11} \\ a_{21} \rightarrow a_{22} \\ a_{31} \rightarrow a_{32} \rightarrow a_{33} \\ \ldots\ldots\ldots\ldots \\ \ldots\ldots\ldots\ldots \\ \ldots\ldots\ldots\ldots \\ a_{n1} \rightarrow a_{n2} \rightarrow a_{n3} \rightarrow .. \rightarrow a_{nn} \end{pmatrix}$$

TRIDIAGONAL ARRAY

Consider an n-square tridiagonal array A as shown in fig below. Note that A has n elements on the diagonal and n - 1 elements above and n - 1 elements below the diagonal. Hence A contains at most 3n - 2 nonzero elements. Suppose we want to store A in a linear array B as indicated by the arrows in Fig i.e:,

$B[l] = a_{ll}$ $B[2] = a_{12}$ $B[3] = a_{21,}$ $B[4] = a_{22}$.....

Find the formula that will give us L in terms of J and K such that

$$B[L] = A[J, K]$$

(so that one can access the value of A[J, K] from the array B).

Note that there are 3(J - 2) + 2 elements above A[J, K] and K - J + 1 elements to the left of A[J, K].

Hence

$$L = [3(J - 2) + 2] + [K - J + 1] + 1 = 2J + K - 2$$

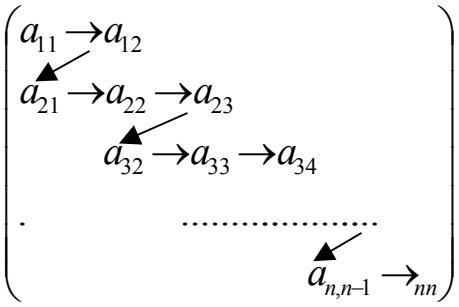


Fig. Tridiagonal array.

Lecture 11: Simple Data Structure - STACK

Summary of Lecture:

- What is a Stack?
- What operations can be performed on it?
- Applications

The linear lists and linear arrays allowed one to insert and delete elements at any place in the list-at the beginning, at the end, or in the middle. There are certain frequent situations in computer science when one wants to restrict insertions and deletions so that they can take place only at the beginning or the end of the list, not in the middle. Two of the data structures that are useful in such situations are stacks and queues.

A stack is a linear structure in which items may be added or removed only at one end. Figure below pictures three everyday examples of such a structure: a stack of dishes, a stack of pennies and a stack of folded towels. Observe that an item may be added or removed only from the top of any of the stacks. This means, in particular, that the last item to be added to a stack is the first item to be removed. Accordingly, stacks are also called last-in first-out (LIFO) lists. Other names used for stacks are "piles" and "push-down lists." Although the stack may seem to be a very restricted type of data structure, it has many important applications in computer science.

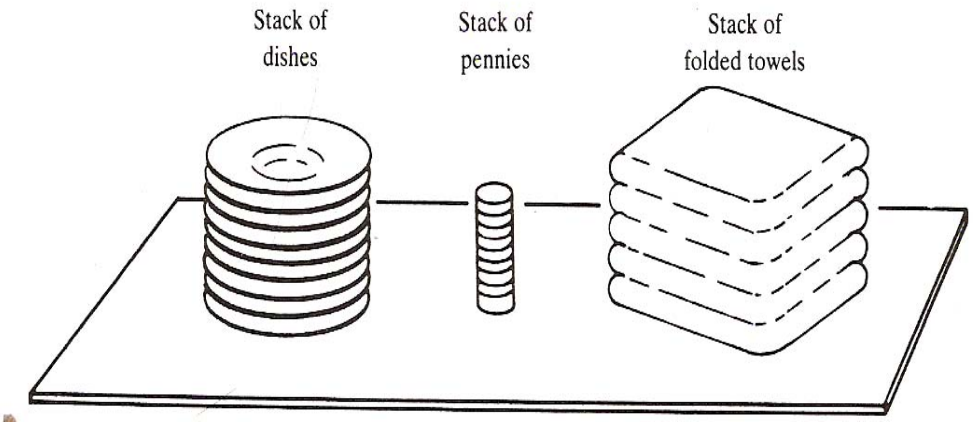


Figure: Stack

The notion of recursion is fundamental in computer science. This topic is introduced in this chapter because one way of simulating recursion is by means of a stack structure.

Definitions:

A stack is a list of elements in which an element may be inserted or deleted only at one end, called me top of the stack. This means, in particular, that elements are removed from a stack in the reverse order of that in which they were inserted into the stack.
Special terminology is used for two basic operations associated with stacks:

(a) "Push" is the term used to insert an element into a stack.

(b) "Pop" is the term used to delete an element from a stack.

EXAMPLE:

Suppose the following 6 elements are pushed, in order, onto an empty stack:

AAA, BBB, CCC, DDD, EEE, FFF

Figure below shows three ways of picturing such a stack. For notational convenience, we will frequently designate the stack by writing:

STACK: AAA, BBB, CCC, DDD, EEE, FFF

The implication is that the right-most element is the top element. We emphasize that, regardless of the way a stack is described, its underlying property is that insertions and deletions can occur only at the top of the stack. This means EEE cannot be deleted before FFF is deleted, DDD cannot be deleted before EEE and FFF are deleted, and so on. Consequently, the elements may be popped from the stack only in the reverse order of that in which they were pushed onto the stack.

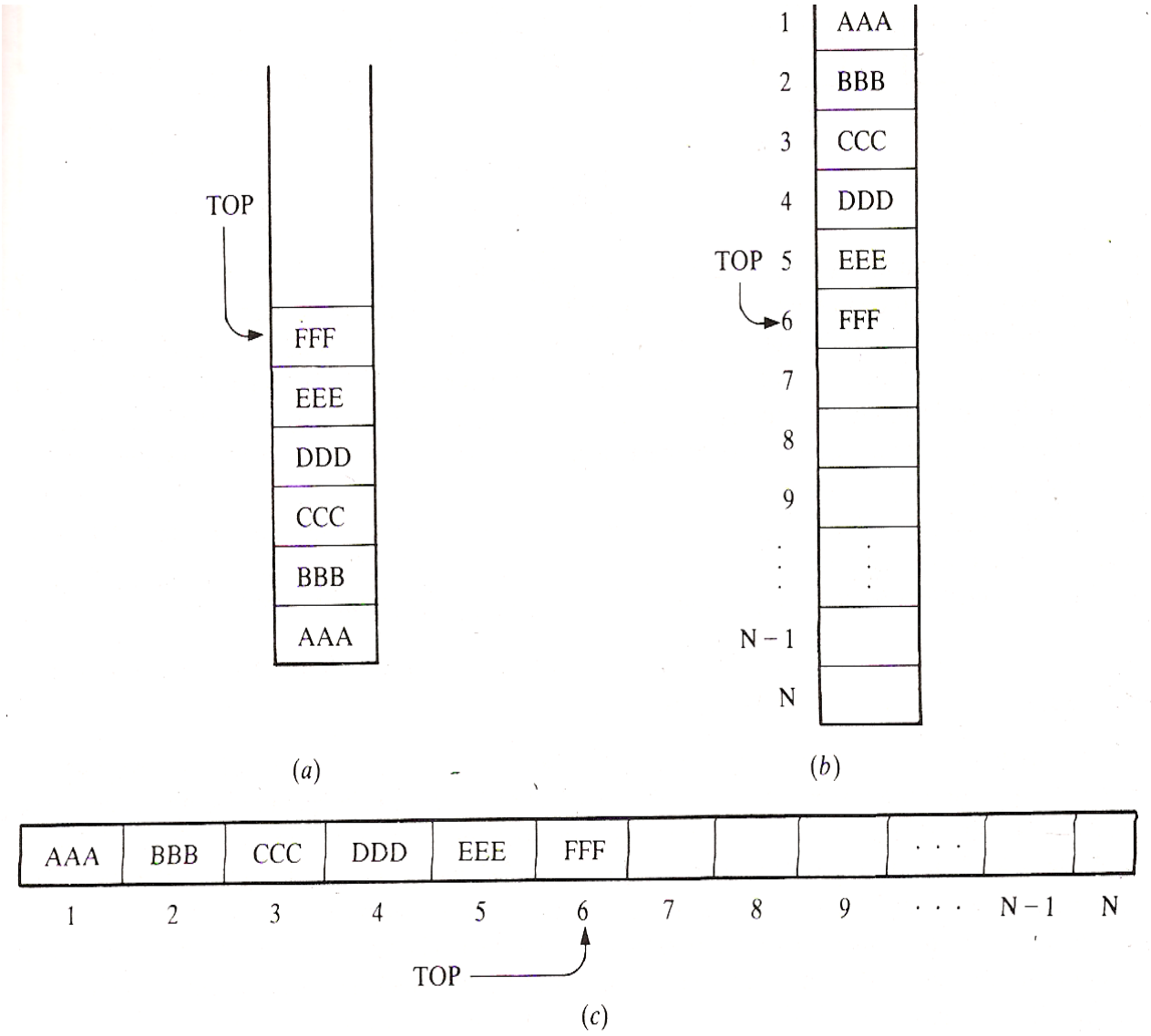


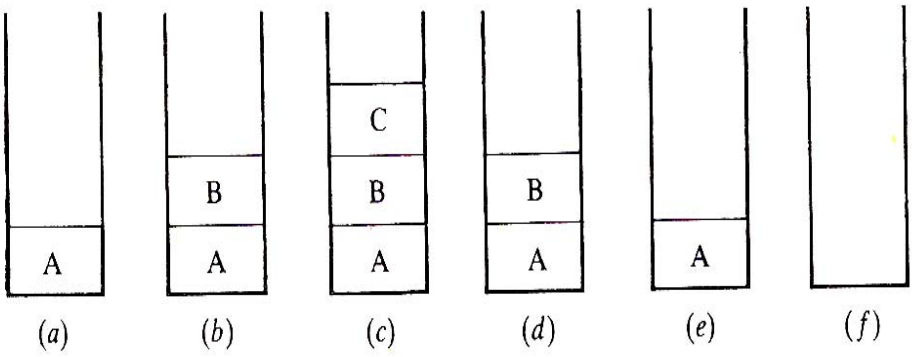
Fig: Diagrams of Stacks

Consider again the AVAIL list of available nodes. Recall that free nodes were removed only from the beginning of the AVAIL list, and that new available nodes were inserted only at the beginning of the AVAIL list. In other words, the AVAIL list was implemented as a stack. This implementation of the AVAIL list as a stack is only a matter of convenience rather than an inherent part of the structure.

Postponed Decisions

Stacks are frequently used to indicate the order of the processing of data when certain steps of the processing must be postponed until other conditions are fulfilled. This is illustrated as follows.

Suppose that while processing some project A we are required to move on to project B, whose completion is required in order to complete project A. Then we place the folder containing the data of A onto a stack, as pictured in Fig. (a), and begin to process B. However, suppose that while processing B we are led to project C, for the same reason. Then we place B on the stack above A, as pictured in Fig. (b), and begin to process C. Furthermore, suppose that while processing C we are likewise led to project D. Then we place C on the stack above B, as pictured in Fig. (c), and begin to process D.



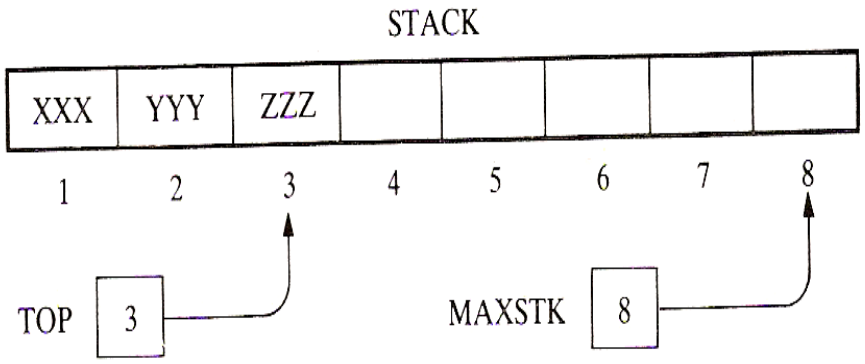
On the other hand, suppose we are able to complete the processing of project D. Then the only project we may continue to process is project C, which is on top of the stack. Hence we remove folder C from the stack, leaving the stack as pictured in Fig. (d), and continue to process C. Similarly, after completing the processing of C, we remove folder B from the stack, leaving the stack as pictured in Fig. (e), and continue to process B. Finally, after completing the processing of B, we remove the last folder, A, from the stack, leaving the empty stack pictured in Fig. (f), and continue the processing of our original project A.

Observe that, at each stage of the above processing, the stack automatically maintains the order that is required to complete the processing. An important example of such a processing in computer science is where A is a main program and B, C and D are subprograms called in the order given.

ARRAY REPRESENTATION OF STACKS

Stacks may be represented in the computer in various ways, usually by means of a one-way list or a linear array. Unless otherwise stated or implied, each of our stacks will be maintained by a linear array STACK; a pointer Variable TOP, which contains the location of the top element of the stack; and a variable MAXSTK which gives the maximum number of elements that can be held by the stack. The condition TOP = 0 or TOP = NULL will indicate that the stack is empty.

Figure on next page pictures such an array representation of a stack. (For notational convenience, the array is drawn horizontally rather than vertically.) Since TOP = 3, the stack has three-elements, XXX, YYY and ZZZ; and since MAXSTK = 8, there is room for 5 more items in the stack.



The operation of adding (pushing) an item onto a stack and the operation of removing (popping) an item from a stack may be implemented, respectively, by the following procedures, called PUSH and POP. In executing the procedure PUSH, one must first test whether there is

room in the stack for the new item; if not, then we have the condition known as overflow. Analogously, in executing the procedure POP, one must first test whether there is an element in the stack to be deleted; if not, then we have the condition known as underflow.

Procedure: PUSH (STACK, TOP, MAXSTK, ITEM)

This procedure pushes an ITEM onto a stack.

1. [Stack already filled?]
   If TOP = MAXSTK, then: Print: OVERFLOW, and Return.
2. Set TOP: =TOP+1. [Increases TOP by 1.]
3. Set ST ACK [TOP]: = ITEM. [Inserts ITEM in new TOP position.]
4. Return.

Procedure: POP (STACK, TOP, ITEM)

This procedure deletes the top element of STACK and assigns it to the variable ITEM.

1. [Stack has an item to be removed?]
   If TOP = 0, then: Print: UNDERFLOW, and Return.
2. Set ITEM: = STACK [TOP]. [Assigns TOP element to ITEM.]
3. Set TOP: = TOP - 1. [Decreases TOP by 1.]
4. Return.

Frequently, TOP and MAXSTK are global variables; hence the procedures may be called using only

PUSH (ST ACK, ITEM) and POP (ST ACI5, ITEM)

respectively. We note that the value of TOP is changed before the insertion in PUSH but the value of TOP is changed after the deletion in POP.

EXAMPLE

(a) Consider the stack in previous figure. We simulate the operation PUSH (STACK, WWW):

1. Since TOP = 3, control is transferred to Step 2.
2. TOP = 3 + 1 = 4.
3. STACK [TOP] = STACK [4] = WWW.
4. Return.

Note that WWW is now the top element in the stack.

(b) Consider again the same stack. This time we simulate the operation POP (STACK, ITEM):

1. Since TOP = 3, control is transferred to Step 2.
2. ITEM = ZZZ.
3. TOP = 3 - 1 = 2.
4. Return.

Observe that STACK [TOP] = STACK [2] =YYY is now the top element in the stack.

Minimizing Overflow

There is an essential difference between underflow and overflow in dealing with stacks. Underflow depends exclusively upon the given algorithm and the given input data, and hence there is no direct control by the programmer. Overflow, on the other hand, depends upon the arbitrary choice of the programmer for the amount of memory space reserved for each stack, and this choice does influence the number of times overflow may occur.

Generally speaking, the number of elements in a stack fluctuates as elements are added to or removed from a stack. Accordingly, the particular choice of the amount of memory for a given stack involves a time-space tradeoff. Specifically, initially reserving a great deal of space for each stack will decrease the number of times overflow may occur; however, this may be an expensive use of the space if most of the space is seldom used. On the other hand, reserving a

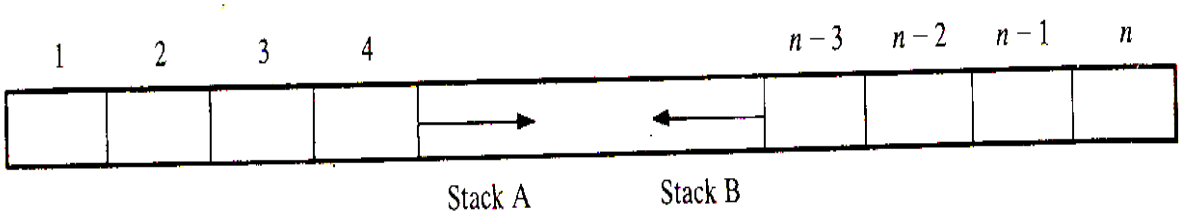
small amount of space for each stack may increase the number of times overflow occurs; and the time required for resolving an overflow, such as by adding space to the stack, may be more expensive than the space saved.

Various techniques have been developed which modify the array representation of stacks so that the amount of space reserved for more than one stack may be more efficiently used.

EXAMPLE

Suppose a given algorithm requires two stacks, A and B. One can define an array STACKA with n1 elements for stack A and an array ST ACKB with n_2 elements for stack B. Overflow will occur when either stack A contains more than n1 elements or stack B contains more than n_2 elements.

Suppose instead that we define a single array STACK with $n = n_1 + n_2$ elements for stacks A and B together. As pictured in Fig. , we define STACK [l] as the bottom of stack A and let A "grow" to the right, and we define STACK [1] as the bottom of stack B and let B "grow" to the left. In this case, overflow will occur only when A and B together have more than $n = n_1 + n_2$ elements. This technique will usually decrease the number of times overflow occurs even though we have not increased the total amount of space reserved for the two stacks. In using this data structure, the operations of PUSH and POP will need to be modified.



Lecture 12: Applications of Stacks

Summary of Lecture:

- Recursion
- Polish Notations

APPLICATION OF STACKS:

POLISH NOTATION

For most common arithmetic operations, the operator symbol is placed between its two operands.
For example,

A+B C-D E*F G/H

This is called infix notation. With this notation, we must distinguish between

(A+B)*C and A + (B * C)

by using either parentheses or some operator-precedence convention such as the usual precedence levels discussed above. Accordingly, the order of the operators and operands in an arithmetic expression does not uniquely determine the order in which the operations are to be performed.

Polish notation, named after the Polish mathematician Jan Lukasiewicz, refers to the notation in which the operator symbol is placed before its two operands. For example,

+AB -CD *EF /GH

We translate, step by step, the following infix expressions into Polish notation using brackets [ ] to indicate a partial translation:

(A + B) *C= [+AB]*C = *+ABC

A + (B*C) = A + [*BC] = +A*BC

(A + B) / (C - D) = [+ AB] / [- CD] = / + AB - CD

The fundamental property of Polish notation is that the order in which the operations are to be performed is completely determined by the positions of the operators and operands in the expression. Accordingly, one never needs parentheses when writing, expressions in Polish notation.

Reverse Polish notation refers to the analogous notation in which the operator symbol is placed after its two operands:

AB+ CD EF* GH/

Again, one never needs parentheses to determine the order of the operations in any arithmetic expression written in reverse Polish notation. This notation is frequently called postfix (or suffix) notation, whereas prefix notation is the term used for Polish notation, discussed in the preceding paragraph.

The computer usually evaluates an arithmetic expression written in infix notation in two steps. First, it converts the expression to postfix notation, and then it evaluates the postfix expression. In each step, the stack is the main tool that is used to accomplish the given task.

Evaluation of a Postfix Expression

Suppose P us an arithmetic expression written in postfix notation. The following

algorithm, which uses a STACK to hold operands, evaluates P.

Algorithm: This algorithm finds the VALUE of an arithmetic expression P written in postfix notation.

1. Add a right parenthesis ")" at the end of P. [This acts as a sentinel.]
2. Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel ")" is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator x is encountered, then:
    a. Remove the two top elements of STACK, where A is the top element and B is the next-top element.
    b. Evaluate B⊗A.
    c. Place the result of (b) back on STACK.

    [End of If structure.]

    [End of Step 2 loop.]
5. Set VALUE equal to the top element to STACK.
6. Exit.

We note that, when Step 5 is executed, there should be only one number on STACK.

Example:

Consider the following arithmetic expression P written in postfix notation:

P : 5, 6, 2, +, *, 12, 4, /, -,)

(Commas are used to separate the elements of P so that, 5,6,2 is not interpreted as the number 562.)

Symbol Scanned		STACK
(1)	5	5
(2)	6	5, 6
(3)	2	5, 6, 2
(4)	+	5, 8
(5)	*	40
(6)	12	40, 12
(7)	4	40, 12, 4
(8)	/	40, 3
(9)	–	37
(10)	)	

The equivalent infix expression Q follows :

Q : 5 * (6 + 2) - 12 / 4

Note that parentheses are necessary for the infix expression Q but not for the postfix expression P.

We evaluate P by simulating Algorithm 6.3. First we add a sentinel right parenthesis at end of P to obtain

P : 5, 6, 2, +, *, 12, 4, /, -,)

(1) (2) (3) (4) (5) (6) (7) (8) (9) (10) The elements of P have been labeled from left to right for easy reference. Above figure shows the contents of STACK as each element of P is scanned. The final number in STACK, 37, which is assigned to VALUE when the sentinel ")" is scanned, is the value of P.

Transforming Infix Expressions into Postfix Expressions

Let Q be an arithmetic expression written in infix notation. Besides operands and operators, Q may also contain left and right parentheses. We assume that the operators in Q consist only of exponentiations (↑), multiplications (*), divisions (/), additions (+) and subtractions (-), and that they have the usual three levels of precedence as given above. We also assume that operators on the same level, including exponentiations, are performed from left to right unless otherwise indicated by parentheses. (This is not standard, since expressions may contain unary operators and some languages perform the exponentiations from right to left. However, these assumptions simplify our algorithm.)

The following algorithm transforms the infix expression Q into its equivalent postfix expression P. The algorithm uses a stack to temporarily hold operators and left parentheses. The postfix expression P will be constructed from left to right using the operands from Q and the operators, which are removed from STACK. We begin by pushing a left parenthesis onto STACK and adding a right parenthesis at the end of Q. The algorithm is completed when STACK is empty.

Algorithm: POLISH(Q, P)

Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

1. Push "(" onto STACK, and add ")" to the end of Q.
2. Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:
3. If an operand is encountered, add it to P.
4. If a left parenthesis is encountered, push it onto STACK.
5. If an operator ⊗ is encountered, then:
   (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than ⊗.
   (b) Add ⊗ to STACK.

   [End of If structure.]
6. If a right parenthesis is encountered, then:
   (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
   (b) Remove the left parenthesis. [Do not add the left parenthesis to P.]

   [End of If structure.]

   [End of Step 2 loop.]
7. Exit.

The terminology sometimes used for Step 5 is that ⊗ will "sink" to its own level. .

EXAMPLE

Consider the following arithmetic infix expression Q:

Q: A+(B*C- (D/E↑F)*G)*H

We simulate the previous algorithm to transform Q into its equivalent postfix expression P.

First we push "(" onto STACK, and then we add ")" to the end of Q to obtain:

A + (B * C - (D / E ↑ F) * G) * H)
(1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13)(14) (15) (16)(17) (18)(19) (20)

The elements of Q have now been labeled from left to right for easy reference. Table below shows the status of STACK and of the string P as each element of Q is scanned. Observe that

(1) Each operand is simply added to P and does not change STACK.

(2) The subtraction operator (-) in row 7 sends * from STACK to P before it (-) is pushed onto STACK.

(3) The right parenthesis in row 14 sends j and then I from STACK to P, and then removes the left parenthesis from the top of STACK.

(4) The right parenthesis in row 20 sends * and then + from STACK to P, and then removes-the left parenthesis from the top of STACK.

After Step 20 is executed, the STACK is empty and

P: A B C * D E F ↑ / G * - H * +

which is the required postfix equivalent of Q.

Symbol Scanned	STACK	Expression P
(1) A	(	A
(2) +	(+	A
(3) (	(+ (	A
(4) B	(+ (	A B
(5) *	(+ (*	A B
(6) C	(+ (*	A B C
(7) −	(+ (−	A B C *
(8) (	(+ (− (	A B C *
(9) D	(+ (− (	A B C * D
(10) /	(+ (− (/	A B C * D
(11) E	(+ (− (/	A B C * D E
(12) ↑	(+ (− (/ ↑	A B C * D E
(13) F	(+ (− (/ ↑	A B C * D E F
(14))	(+ (−	A B C * D E F ↑ /
(15) *	(+ (− *	A B C * D E F ↑ /
(16) G	(+ (− *	A B C * D E F ↑ / G
(17))	(+	A B C * D E F ↑ / G * −
(18) *	(+ *	A B C * D E F ↑ / G * −
(19) H	(+ *	A B C * D E F ↑ / G * − H
(20))		A B C * D E F ↑ / G * − H * +

QUICKSORT is another application of stacks. (covered later on)

RECURSION

Recursion is an important concept in computer science. Many algorithms can be best described in terms of recursion.

Suppose P is a procedure containing either a Call statement to itself or a Call statement to a second procedure that may eventually result in a Call statement back to the original procedure P. Then P is called a *recursive procedure*. So that the program will not continue to run indefinitely, a recursive procedure must have the following two properties:

(1) There must be certain criteria, called base criteria, for which the procedure does not call itself.

(2) Each time the procedure does call itself (directly or indirectly), it must be closer to the *base criteria*.

A recursive procedure with these two properties is said to be *well-defined*.

Similarly, a function is said to be recursively defined if the function definition refers to itself. Again, in order for the definition not to be circular, it must have the following two properties:

(1) There must be certain arguments, called base values, for which the function does not refer to itself.

(2) Each time the function does refer to itself, the argument of the function must be closer to a base value.

A recursive function with these two properties is also said to be well-defined.

Review Questions:

1. What is a stack? What different operations can be performed on stacks?
2. How are stacks used? State the different applications.
3. Explain the concept of recursion.
4. What are Polish Notations?

Lecture 13: Review/ Class Test

Lecture 14: Simple Data Structure - QUEUE

Summary of Lecture:

- What is a Queue?
- What operations can be performed on it?

A queue is a linear list in which items may be added only at one end and items may be removed-only at the other end. The name "queue" likely comes from the everyday use of the term. Consider: queue of people waiting at a bus stop, as pictured in fig. below. Each new person who comes takes his or her place at the end of the line, and when the bus comes, the people at the front of the line board first Clearly, the first person in the line is the first person to leave. Thus queues are also called first-in first-out (FIFO) lists. Another example of a queue is a batch of jobs waiting to be processed, assuming no job has higher priority than the others..

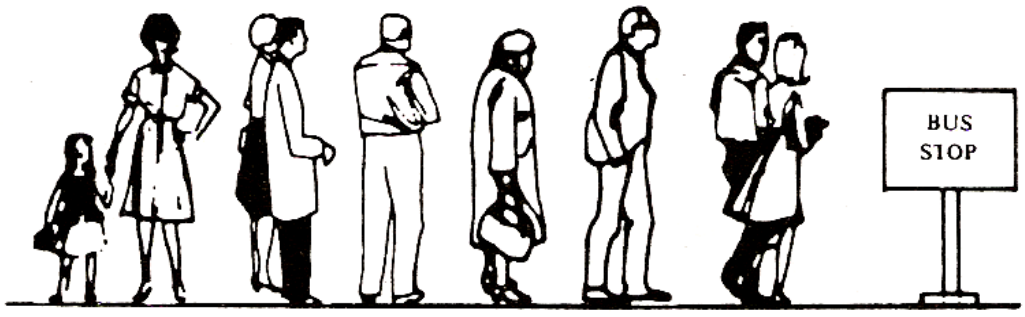


Figure: People waiting for a bus

A queue is a linear list of elements in which deletions can take place only at one end, called the front, and insertions can take place only at the other end, called the rear. The terms "front" and "rear" are used in describing a linear list only when it is implemented as, a queue.

Queues are also called first-in first-out (FIFO) lists, since the first element in a queue will be the first element out of the queue. In other words, the order in which elements enter a queue is the order in which they leave. This contrasts with stacks, which are last-in first-out (LIFO) lists.

Queues abound in everyday life. The automobiles waiting to pass through an intersection form a queue, in which the first car in line is the first car through; the people waiting in line at a bank form a queue, where the first person in line is the first person to be waited on; and so on. An important example of a queue in computer science occurs in a timesharing system,)n which programs with the same priority form a queue while waiting to be executed. (Another structure, called a priority queue, is discussed later on.)

Representation of Queues

Queues may be represented in the computer in various ways, usually by means of one-way lists or linear arrays. Unless otherwise stated or implied, each of our queues will be maintained by a linear array QUEUE and two pointer variables: FRONT, containing the location of the front element of the queue; and REAR, containing the location of the rear element of the queue. The condition FRONT = NULL will indicate that the queue is empty.

Figure below indicates the way elements will be deleted from the queue and the way new elements will be added to the queue. Observe that whenever an element is deleted from the queue, the value of FRONT is increased by 1; this can be implemented by the

assignment

FRONT: = FRONT + 1

Similarly, whenever an element is added to the queue, the value of REAR is increased by 1; this can be implemented by the assignment

REAR: = REAR + 1

This means that after N insertions, the rear element of the queue will occupy QUEUE [N] or, in other words, eventually the queue will occupy the last part of the array. This occurs even through the queue itself may not contain many elements.

Suppose we want to insert an element ITEM into a queue at the time the queue does occupy the last part of the array, i.e., when REAR = N. One way to do this is to simply move the entire queue to the beginning of the array, changing FRONT and REAR accordingly, and then inserting ITEM as above. This procedure may be very expensive. The procedure we adopt is to assume that the array

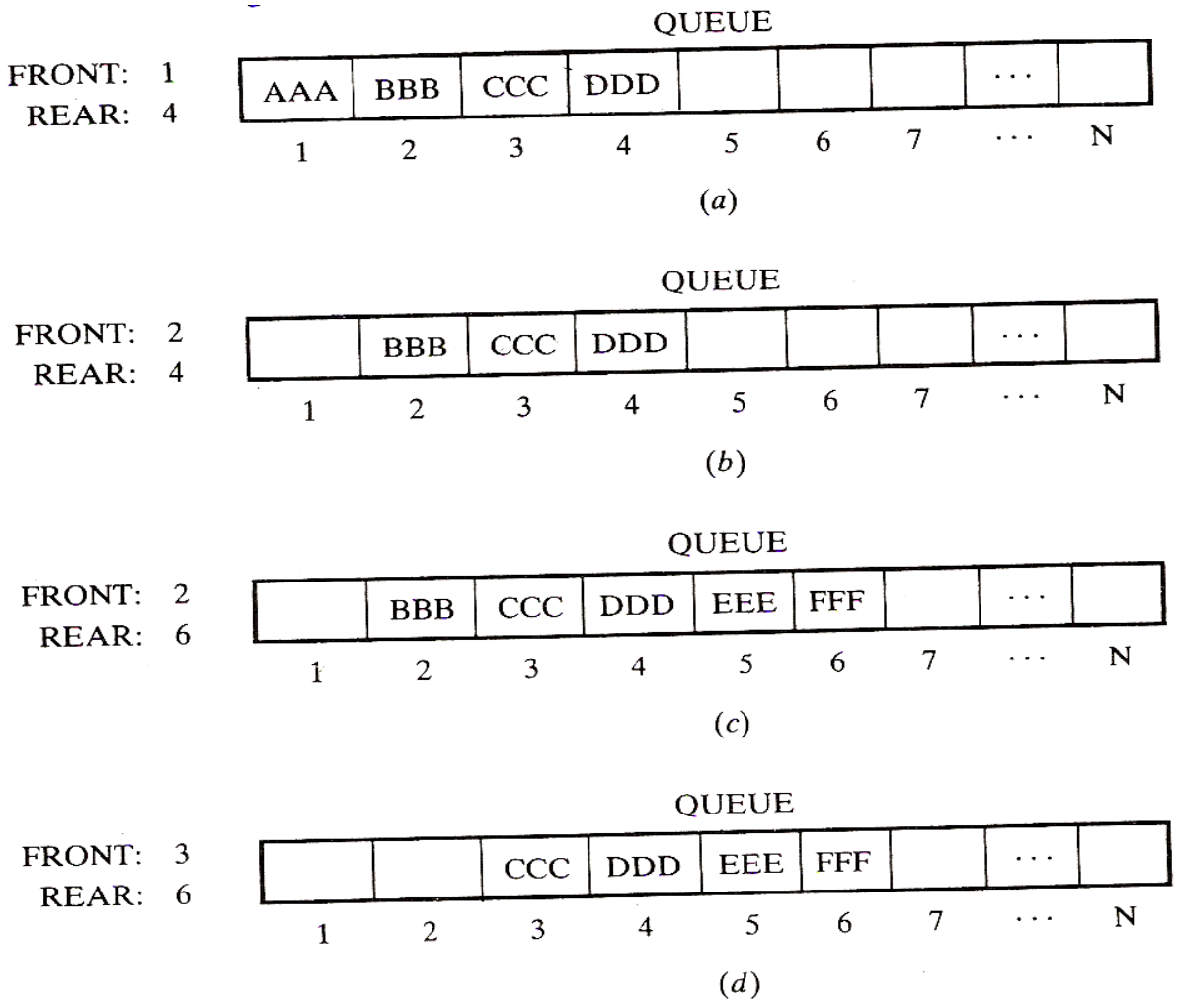


Figure: Array representation of a queue

QUEUE is circular, that is, that QUEUE [l] comes after QUEUE [N] in the array. With this assumption, we insert ITEM into the queue by assigning ITEM to QUEUE [l]. Specifically, instead of increasing REAR to N + 1, we reset REAR = 1 and then assign

QUEUE [REAR]: = ITEM

Similarly, if FRONT = N and an element of QUEUE is deleted, we reset FRONT = 1 instead of increasing FRONT to N + 1.

Suppose that our queue contains only one element, i.e., suppose that

$$FRONT = REAR \neq NULL$$

and suppose that the element is deleted. Then we assign

FRONT:= NULL and REAR: = NULL

to indicate that the queue is empty.

Procedure: QINSERT (QUEUE, N, FRONT, REAR, ITEM)

This procedure inserts an element ITEM into a queue.

1. [Queue already filled?]
   If FRONT = 1 and REAR = N, or if FRONT = REAR + 1, then:
      Write: OVERFLOW, and Return.
2. [Find new value of REAR.]
   If FRONT:= NULL, then: [Queue initially empty.]
      Set FRONT: = 1 and REAR: = 1.
   Else if REAR:=: N, then:
      Set REAR:= 1.
   Else:
      Set REAR:= REAR + 1.
   [End of If structure.]
3. Set QUEUE[REAR]:= ITEM. [This inserts new element.]
4. Return.

Procedure : QDELETE (QUEUE, N, FRONT, REAR, ITEM)

This procedure deletes an element from a queue and assigns it to The variable ITEM.

1. [Queue already empty?]
   If FRONT: = NULL, then: Write: UNDERFLOW, and Return.
2. Set ITEM: = QUEUE [FRONT].
3. [Find new value of FRONT.]
   If FRONT = REAR, then: [Queue has only one element to Start.]
      Set FRONT: = NULL and REAR: = NULL.
   Else if FRONT = N, then:
      Set FRONT:= 1.
   Else:
      Set FRONT: = FRONT + 1.
   [End of If structure.]
4. Return.

Lecture 15: Simple Data Structure - QUEUE

Summary of Lecture:

- Types of Queues
- Applications

Types of Queues:

DEQUES

A deque (pronounced either "deck" or "dequeue") is a linear list in which elements can be added or removed at either end but not in the middle. The term deque is a contraction of the name double-ended queue.

There are various ways of representing a deque in a computer. Unless it is -otherwise stated or implied, we will assume our deque is maintained by a circular array DEQUE with pointers LEFT and RIGHT, which point to the two ends of the deque. We assume that the elements extend from the left end to the right end in the array. The term "circular" comes from the fact that we assume that DEQUE [I] comes after DEQUE [N] in the array. Figure below pictures two deques, each with 4 elements maintained in an array with N = 8 memory locations. The condition LEFT = NULL will be used to indicate that a deque is empty.

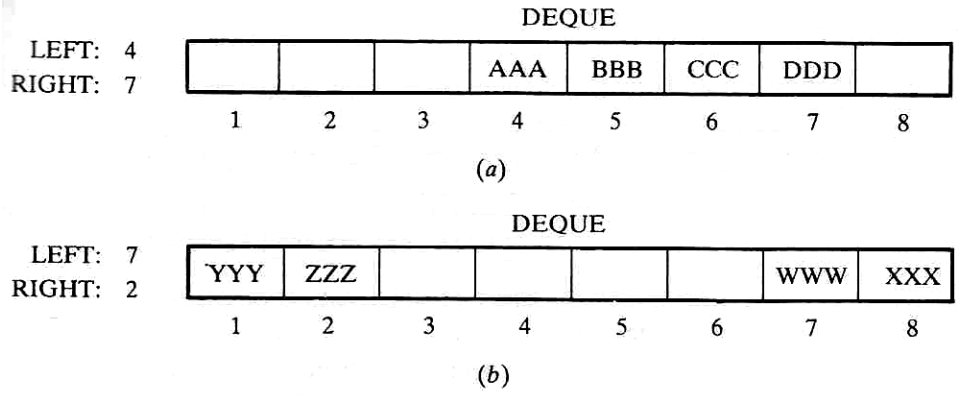
There are two kinds of a deque

Input-restricted deque- is a deque, which allows insertions at only one end of the list

but allows deletions at both ends of the list;

Output-restricted deque- is a deque, which allows deletions at only one end of the list but allows insertions at both ends of the list.

The procedures, which insert and delete elements in deques and the variations on those procedures, are given as supplementary problems. As with queues, a complication may arise (a) when there is overflow, that is, when an element is to be inserted into a deque which is already full, or (b) when there is underflow, that is, when an element is to be deleted from a deque, which is empty. The procedures must consider these possibilities.



PRIORITY QUEUES

A *priority queue* is a collection of elements such that each element has been assigned

a priority and such that the order in which elements are deleted and processed comes from the following rules:

(1) An element of higher priority is processed before any element of lower priority.

I

(2) Two elements with the same priority are processed according to the order in which they were added to the queue.

A prototype of a priority queue is a timesharing system: programs of high priority are processed first, and programs with the same priority form a standard queue.

There are various ways of maintaining a priority queue in memory. We discuss two of them here: one uses a one-way list, and the other uses multiple queues. The ease or difficulty in adding elements to or deleting them from a priority queue clearly depends on the representation that one chooses.

One-Way List Representation of a Priority Queue

One way to maintain a priority queue in memory is by means of a one-way list, _s' follows:

(a) Each node in the list will contain three items of information: an information field INFO, a priority number PRN and a link number LINK.

(b) A node X precedes a node Y in the list (1) when X has higher priority than Y or (2) when both have the same priority but X was added to the list before Y. This means that the order in the on the way list corresponds to the order of the priority queue.

Priority numbers will operate in the usual way: the lower the priority number, the higher the priority.

EXAMPLE

Figure (a) below shows a schematic diagram of a priority queue with 7 elements. The diagram does not tell us whether BBB was added to the list before or after DDD. On the other hand, the diagram does tell us that BBB was inserted before CCC, because BBB and CCC have the same priority number and BBB appears before CCC in the list. Figure (b) shows the way the priority queue may appear in memory using linear arrays INFO, PRN and LINK.

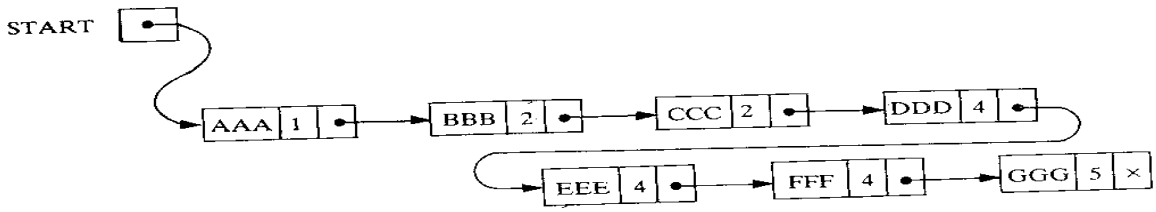


Figure (a)

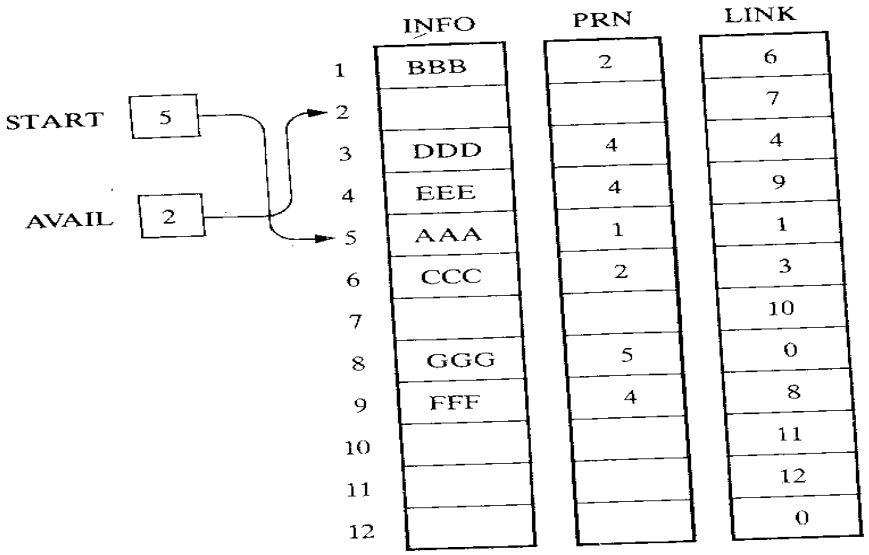


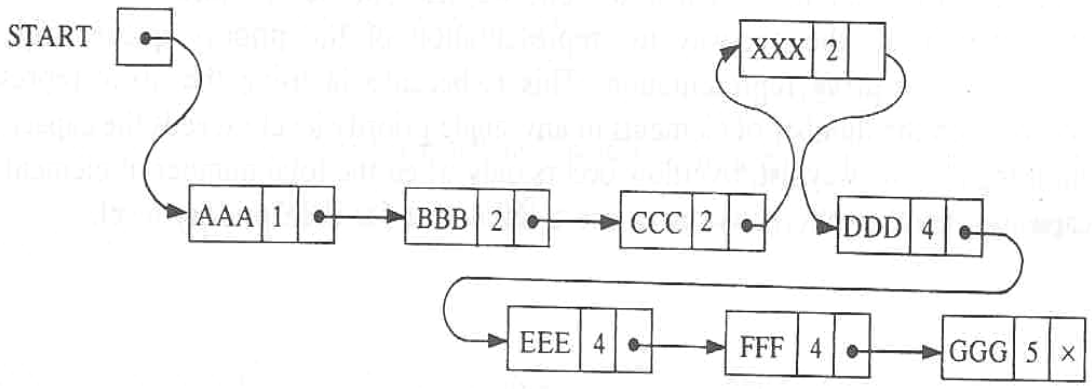
Figure (b)

The main property of the one-way list representation of a priority queue is that the

element in the queue that should be processed first always appears at the beginning of the one-way list. Accordingly it is a very simple matter to delete and process an element from our priority queue. The outline of the algorithm follows.

Array Representation of a Priority Queue

Another way to maintain a priority queue in memory is to use a separate queue for each level of priority (or for each priority number). Each such queue will appear in its own circular array and must have its own pair of pointers, FRONT and REAR. In fact, if each queue is allocated the same amount of space, a two-dimensional array QUEUE can be used instead of the linear arrays. Figure (d) indicates this representation for the priority queue in Fig. (c). Observe that FRONT [K] and REAR [K] contain, respectively, the front and rear elements of row K of QUEUE, the row that maintains the queue of elements with priority number K.



	FRONT	REAR
1	2	2
2	1	3
3	0	0
4	5	1
5	4	4

	1	2	3	4	5	6
1		AAA				
2	BBB	CCC	XXX			
3						
4	FFF				DDD	EEE
5				GGG		

The following are outlines of algorithms for deleting and inserting elements in a priority queue that is maintained in memory by a two-dimensional array QUEUE, as above. The details of the algorithms are left as exercises.

Algorithm : This algorithm deletes and processes the first element in a priority queue maintained by a two-dimensional array QUEUE.

1. [Find the first nonempty queue.]

   Find the smallest K such that FRONT [K] # NULL.

2. Delete and process the front element in row K of QUEUE.
3. Exit.

Algorithm : This algorithm adds an ITEM with priority number M to a priority queue maintained by a two-dimensional array QUEUE.

1. Insert ITEM as the rear element in row M of QUEU.
2. Exit.

Classroom exercises:

Using the order in which the operators are executed, translate each operator from infix to prefix notation.

(a) (A-B)*(D/E)=[-AB]*[/DE]= * - A B / D E

(b) (A+B ↑ D)/(E-F)+G=(A+[↑BD])/[-EF]+G

. =[+A↑BD]/[-EF]+G

= [/+A↑BD - EF] + G

=+ / + A ↑ B D- E F G

The prefix expression is not the reverse of the postfix expression. However, the order of the operands-A, B, D and E in part (a) and A, B, D, E, F and G in part (b)- is the same for all three expressions, infix, postfix and prefix.

Lecture 16: Problem Solving using Polish Notations, and review of previous topics

Lecture 17: Overview of complex data structures, Dynamic memory allocation

Summary of Lecture:

- Overview of Complex Data Structures
- Dynamic memory allocation

HASHING

The search time of each algorithm discussed so far depends on the number *n* of elements in the collection S of data. This section discusses a searching technique, called ***hashing*** or ***hash addressing****,* which is essentially independent of the number *n.*

First of all, we assume that there is a file *F* of *n* records with a set *K* of keys which uniquely determine the records in *F.* Secondly, we assume that *F* is maintained in memory by a table *T* of *m* memory locations and that *L* is the set of memory addresses of the locations in T. For notational convenience, we assume that the keys in *K* and the addresses in *L* are (decimal) integers. (Analogous methods will work with binary integers or with keys which are character strings, such as names, since there are standard ways of representing strings by integers.)

EXAMPLE:

Suppose a company with 68 employees assigns a 4-digit employee number to each employee which is used as the primary key in the company's employee file.

We can, in fact, use the employee number as the address of the record in memory. The search will require no comparisons at all. Unfortunately, this technique will require space for 10 000 memory locations, whereas space for fewer than 30 such locations would actually be used. Clearly, this tradeoff pf space for time is not worth the expense.

The general idea of using the key to determine the address of a record is an excellent idea, but it must be modified so that a great deal of space is not wasted. This modification takes the form of a function *H* from the set *K* of keys into the set *L* of memory addresses. Such a function,

$$H:K \rightarrow L$$

is called a *hash function* or *hashing function.* Unfortunately, such a function *H* may not yield

distinct values: it is possible that two different keys kj and *k2* will yield the same hash address. This situation is called *collision,* and some method must be used to resolve it. Accordingly, the topic of hashing is divided into two parts: (1) hash functions and (2) collision resolutions. We discuss these two parts separately.

Hash Functions

The two principal criteria used in selecting a hash function *H: K _ L* are as follows. First of all, the function *H* should be very easy and quick to compute. Second the function *H* should, as far as possible, uniformly distribute the hash addresses throughout the set *L* so that there are a minimum number of collisions. Naturally, there is no guarantee that the second condition can be completely fulfilled without actually knowing beforehand the keys and addresses. However, certain general techniques do help. One technique is to "chop" a key k into pieces and combine the pieces in some way to form the hash address *H(k).* (The term

"hashing" comes from this technique of "chopping" a key into pieces.)

We next illustrate some popular hash functions. We emphasize that each of these hash functions can be easily and quickly evaluated by the computer.

(a) *Division method.* Choose a number *m* larger than the number *n* of keys in *K.* (The number *m* is usually chosen to be a prime number or a number without small divisors, since this frequently minimizes the number of collisions.) The hash function *H* is defined by

$H(k) = k \pmod m$ or $H(k) = k \pmod m + 1$

He.re $k \pmod m$ denotes the remainder when *k* is divided by *m.* The second formula is used when we want the hash addresses to range from 1 to *m* rather than from 0 to *m* - 1.

(b) *Midsquare method.* The key *k* is squared. Then the hash function *H* is defined *by*

$$H(k) = /$$

where / is obtained by deleting digits from both ends of e We emphasize that the same positions *<Ii k2* must be used for all of the keys.

(C) *Folding method. T*he key *k* is partitioned into a number of parts, k_1 . . . , k_2, where each part, except possibly the last, has the same number of digits as the required address. Then the parts are added together, ignoring the last carry. That is,

$$H(k)= k + k +... + k$$

where the leading-digit carries, if any, are ignored. Sometimes, for extra "milling;" the even-numbered parts, k_2, k_4, . . . , are each reversed before the addition.

EXAMPLE:

Consider the company in Example 9.9, each of whose 68 employees is assigned a unique 4-digit employee number. Suppose L consists of 100 two-digit addresses: 00,01,02, . . . ,99. We apply the above hash functions to each of the following employee numbers:

3205, 7148, 2345

(a) Division method. Choose a prime number m dose to 99, such as $m = 97$. Then

$H(3205) = 4$, $H(7148) = 67$, $H(2345) = 17$

That is, dividing 3205 by 97 gives a remainder of 4, dividing 7148 by 97 gives a remainder of 67, and dividing 2345 by 97 gives a remainder of 17. In the case that the memory addresses begin with 01 rather than 00, we choose that the function $H(k) = k(mod\ m) + 1$ to obtain:

$H(3205) = 4 + 1 = 5$, $H(7148) = 67 + 1 = 68$, $H(2345) = 17 + 1 = 18$

(b)*Midsquare method.* The following calculations are performed:

k: 3205	7148	2345
k^2: 10272025	51093904	5499025
H(k): 72	93	99

Observe that the fourth and fifth digits, counting from the right, are chosen for the hash address.

(c)*Folding method.* Chopping the key *k* into two parts and adding yields the following hash addresses:

H(3205) = 32 + 05 = 37,*H(7148)* = 71 + 48 = 19,*H(2345)* = 23 + 45 = 68

Observe that the leading digit 1 in *H(7148)* is ignored. Alternatively, one may want to reverse the second part before adding, thus producing the following hash addresses:

H(3205) = 32 + 50 = 82,*H(7148)* = 71 + 84 + 55,*H(2345)* = 23 + 54 = 77

Hash Tables

Hash tables are a simple and effective method to implement dictionaries. Average time to search for an element is ***O***(1), while worst-case time is ***O***(*n)*.

Theory

A hash table is simply an array that is addressed via a hash function. For example, in Figure 1, **HashTable** is an array with 8 elements. Each element is a pointer to a linked list of numeric data. The hash function for this example simply divides the data key by 8, and uses the remainder as an index into the table. This yields a number from 0 to 7. Since the range of indices for **HashTable** is 0 to 7, we are guaranteed that the index is valid.

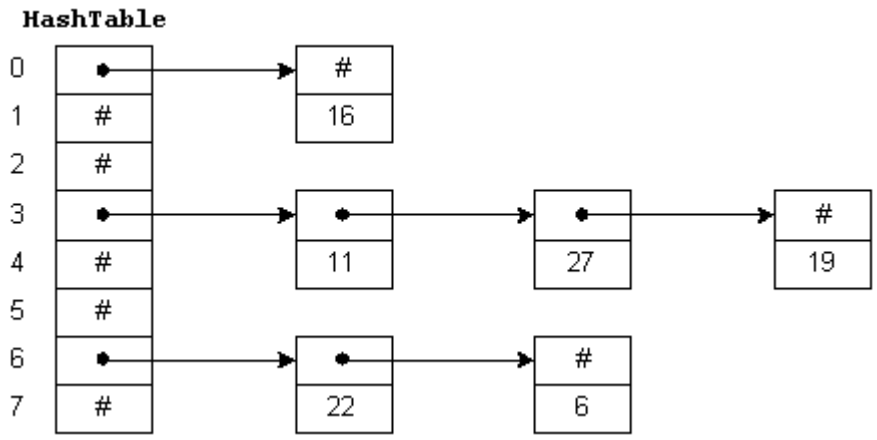


Figure 1: A Hash Table

To insert a new item in the table, we hash the key to determine which list the item goes on, and then insert the item at the beginning of the list. For example, to insert 11, we divide 11 by 8 giving a remainder of 3. Thus, 11 goes on the list starting at **HashTable[3]**. To find a number, we hash the number and chain down the correct list to see if it is in the table. To delete a number, we find the number and remove the node from the linked list.

Entries in the hash table are dynamically allocated and entered on a linked list associated with each hash table entry. This technique is known as *chaining*. An alternative method, where all entries are stored in the hash table itself, is known as direct or open addressing and may be found in the references.

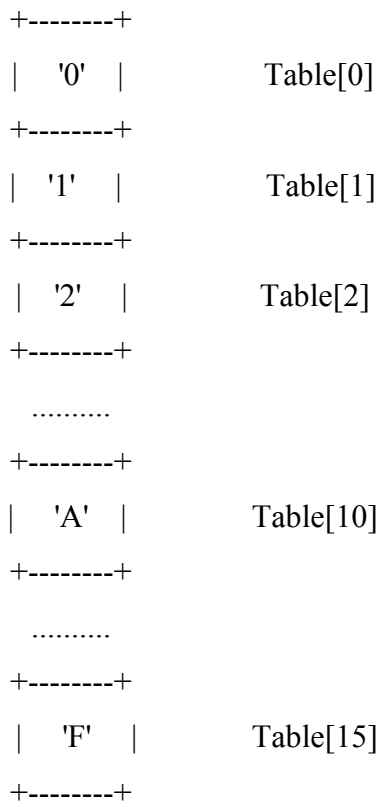
If the hash function is uniform, or equally distributes the data keys among the hash table indices, then hashing effectively subdivides the list to be searched. Worst-case behavior occurs when all keys hash to the same index.

Lookup Tables

Lookup tables are a sequence of data (numbers or characters) which are used for conversion purposes.

The start of the table represents the **base** value, and each location (table entry) contains the desired answer.

It is essentially an array. Consider the example of converting the numeric digits 0-15 to their hexadecimal character equivalents. This can be achieved as shown below,



As you can see from the table above, the index into the array is the value you trying to convert, with the answer stored at that element number.

Lets implement this in C

```
#include <stdio.h>

static char Table[] = "0123456789ABCDEF";

main() {
  int value, valid;
  char answer;

  valid = 0;
  while( valid == 0 ) {
    printf("Enter integer value to convert [0-15]\n");
    scanf(" %d", &value );
    if( (value < 0) || (value > 15) ) {
      printf("Input value outside legal range 0-15]\n");
    }
    else {
      valid = 1;
    }
  }
  answer = Table[value];
  printf("The answer is %c\n", answer );
}
```

Another use can be converting a range of degrees from Fahrenheit to Celcius.

The major advantages of lookup tables are

- fast
- easy to implement
- each conversion takes the same amount of time

Lookup tables are often used where the number of input values is limited (not suited to a large range of values) and where a small amount of inaccuracy is unimportant.

Lecture 18: Linked lists

Summary of Lecture:

- Overview of Complex Data Structures
- Linked list – What is it?

LINKED LISTS

A ***linked list***, or ***one-way list***, is a linear collection of data elements, called ***nodes***, where the linear order is given by means of ***pointers***.

That is, each node is divided into two parts:

the first part contains the *information* of the element,

and the second part, called the *link field* or *next pointer* field, contains the address of the next node in the list.

Figure below is a schematic diagram of a linked list with 6 nodes. Each node is pictured with two parts. The left part represents the information part of the node, which may contain an entire record of data items (e.g. NAME, ADDRESS, . . .). The right part represents the next pointer field of the node, and there is an arrow drawn from it to the next node in the list. This follows the usual practice of drawing an arrow from a field to a node when the address of the node appears in the given field. The pointer of the last node contains a special value, called the null pointer, which is any invalid address.

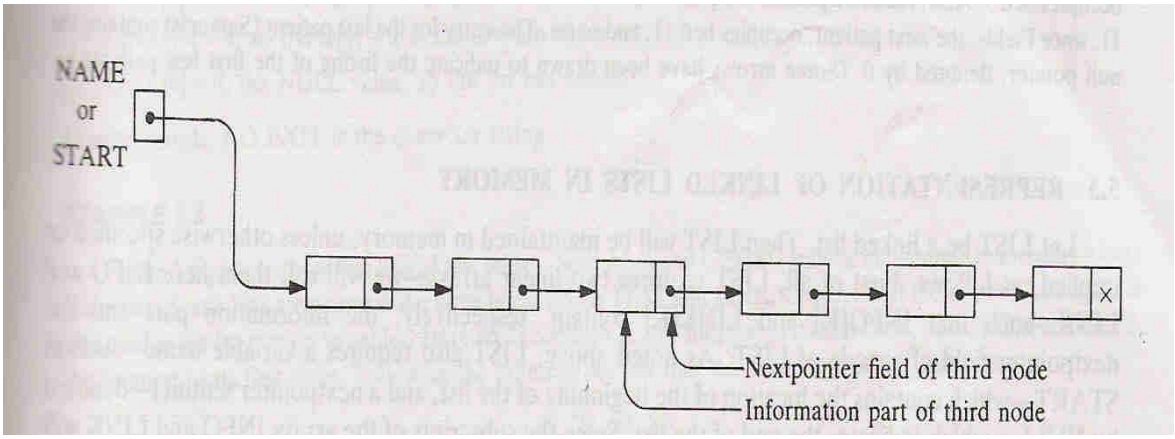
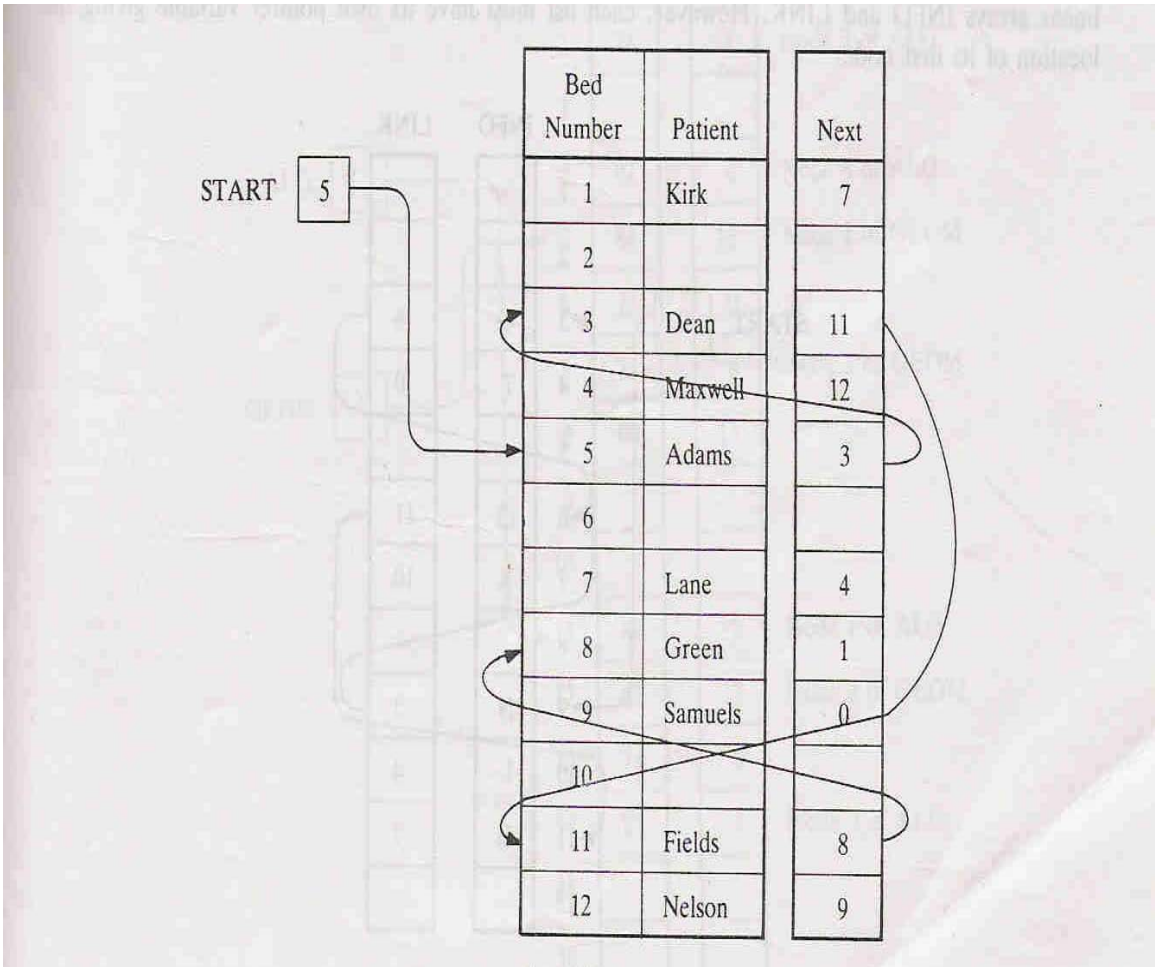


Fig: Linked list with 6 nodes

(In actual practice, 0 or a negative number is used for the null pointer.) The null pointer, denoted by x in the diagram, signals the end of the list. The linked list also contains a *list pointer variable*-called START or NAME-which contains the address of the first node in the list; hence there is an arrow drawn from START to the first node. Clearly, we need only this address in START to trace through the list. A special case is the list that has no nodes. Such a list is called the **null list** or **empty list** and is denoted by the null pointer in the variable START.

EXAMPLE

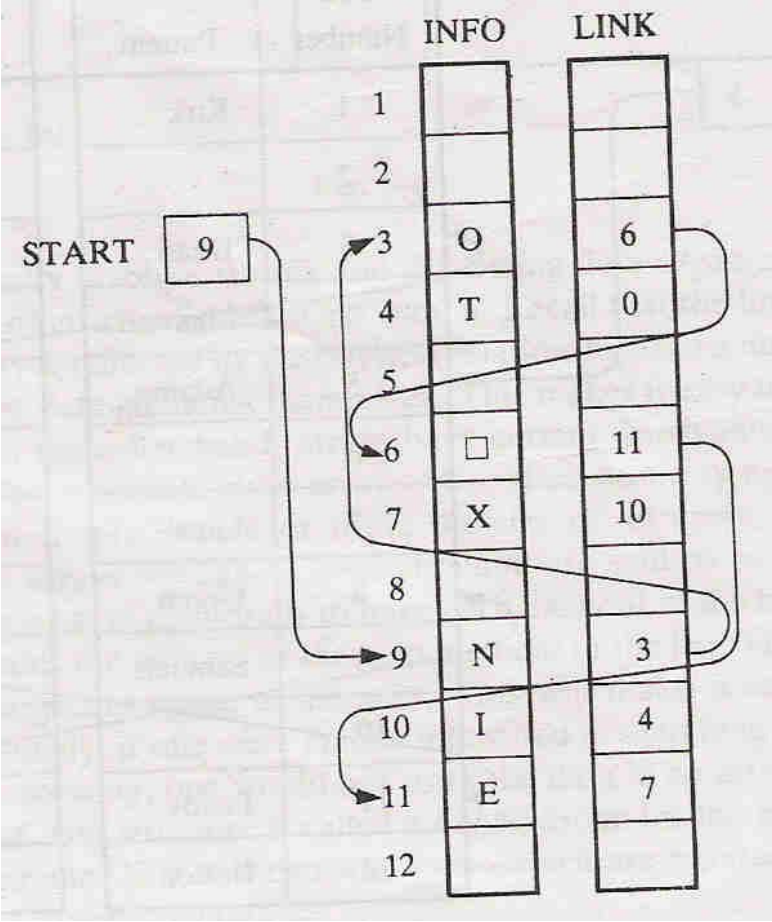
A hospital ward contains 12 beds, of which 9 are occupied as shown in Fig below. Suppose we want an alphabetical listing of the patients. This listing may be given by the pointer field, called Next in the figure. We use the variable START to point to the first patient. Hence START contains 5, since the first patient, Adams, occupies bed 5. Also, Adams's pointer is equal to 3, since Dean, the next patient, occupies bed 3; Dean's pointer is 11, since Fields, the next patient, occupies bed 11; and so on. The entry for the last patient (Samuels) contains the null pointer, denoted by O. (Some arrows have been drawn to indicate the listing of the first few patients.)



REPRESENTATION OF LINKED LISTS IN MEMORY

Let LIST be a linked list. Then LIST will be maintained in memory, unless otherwise specified or implied, as follows. First of all, LIST requires two linear arrays-we will call them here INFO and LINK-such that INFO[K] and LINK[K] contain, respectively, the information part and the next pointer field of a node of LIST. As noted above, LIST also requires a variable name-such as START-which contains the location of the beginning of the list, and a next pointer sentinel-denoted by NULL-which indicates the end of the list. Since the subscripts of the arrays INFO and LINK will usually be positive, we will choose NULL = 0, unless otherwise stated.

The following examples of linked lists indicate that the nodes of a list need not occupy adjacent elements in the arrays INFO and LINK, and that more than one list may be maintained in the same linear arrays INFO and LINK. However, each list must have its own pointer variable giving the location of its first node.



EXAMPLE:

Figure pictures a linked list in memory where each node of the list contains a single character. We can obtain the actual list of characters, or, in other words, the string, as follows:

START = 9, so INFO[9] = N is the first character.

LINK[9] = 3, so INFO[3] = 0 is the second character.

LINK[3] = 6, so INFO[6] = 0 (blank) is the third character.

LINK[6] = 11, so INFO[11] = E is the fourth character.

LINK[11] = 7, so INFO[7]= X is the fifth character.

LINK[7] = 10, so INFO 0] = I is the sixth character.

LlNK[10] = 4, so INFO] = T is the seventh character.

LlNK[4] = 0, the NULL value, so the list has ended.

In other words, NO EXIT is the character string.

We require two tests. First we have to check to see whether we have reached the end of the list; i.e., first we check *to* see whether

PTR= NULL

If *not,* then we check *to* see whether

INFO[PTR] = ITEM

The two tests cannot be performed at the same time, since INFO[PTR] is *not* defined when PTR = NULL. Accordingly, we use the first test *to* control the execution *of* a loop, and we let the second test take place inside the loop. The algorithm follows.

Algorithm: SEARCH (INFO , LINK, START, ITEM, LOC)

LIST is a linked list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR:= START.
2. Repeat Step 3 while PTR # NULL:
3. If ITEM = INFO[PTR], then:
   Set LOC: =, PTR, and Exit.
   Else:
   Set PTR: = LINK[PTR]. [PTR now points to the next node.]
   [End of If structure.]
   [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC: = NULL.
5. Exit.

The complexity of this algorithm is the same as that of the linear search algorithm i.e., the worst-case running time is proportional to the number n of elements in LIST, and the average-case running time is approximately proportional to n/2 (with the condition that ITEM appears once in LIST but with equal probability in any node of LIST).

LIST is Sorted

Suppose the data in LIST are sorted. Again we search for ITEM in LIST by traversing the list using a pointer variable PTR and comparing ITEM with the contents INFO[PTR] of each node, one by one, of LIST. Now, however, we can stop once ITEM exceeds INFO[PTR]. The algorithm follows as:

The complexity of this algorithm is still the same as that of other linear search algorithms; that is, the worst-case running time is proportional to the number n of elements in LIST, and the average-case running time is approximately proportional to n/2.

Algorithm: SRCHSL (INFO, LINK, START, ITEM, LOC)

LIST is a sorted list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC =NULL.

1. Set PTR: = START.
2. Repeat Step 3 while PTR ≠= NULL:
3. If ITEM < INFO[PTR], then:
   Set PTR:= LINK[PTR]. [PTR now points to next node]
   Else if ITEM = INFO[PTR], then:
   Set LOC: = PTR, and Exit. [Search is successful.]
   Else:
   Set LOC:=NULL, and Exit.[ITEM now exceeds INFO[PTR].]
   [End of If structure.]
   [End of Step 2 loop.]
4. Set LOC: = NULL.
5. Exit.

With a sorted linear array we can apply a binary search whose running time is proportional to $\log_2 n$. On the other hand, *a binary search algorithm cannot be applied to a sorted linked list, since there is no way of indexing the middle element in the list*. This property is one of the main drawbacks in using a linked list as a data structure.

Lecture 19: Memory allocation, More on Linked lists

Summary of Lecture:

- Memory Allocation
- Garbage Collection
- Types of linked lists

MEMORY ALLOCATION

The maintenance of linked lists in memory assumes the possibility of inserting new nodes into the lists and hence requires some mechanism that provides unused memory space for the new nodes. Analogously, some mechanism is required whereby the memory space of deleted nodes becomes available for future use.

Together with the linked lists in memory, a special list is maintained which consists of unused memory cells. This list, which has its own pointer, is called the list of available space or the free-storage list or the free pool.

Suppose our linked lists are implemented by parallel arrays as described in the preceding sections, and suppose insertions and deletions are to be performed on our linked lists. Then the unused memory cells in the arrays will also be linked together to form a linked list using AVAIL as its list pointer variable. (Hence this free-storage list will also be called the AVAIL list.) Such a data structure will frequently be denoted by writing

LIST (INFO, LINK, START, AVAIL)

Garbage Collection

Suppose some memory space becomes reusable because a node is deleted from a list or an entire list is deleted from a program. Clearly, we want the space to be available for future use. One way to bring this about is to immediately reinsert the space into the free-storage list. This is

what we will do when we implement linked lists by means of linear arrays. However, this method may be too time-consuming for the operating system of a computer, which may choose an alternative method, as follows.

The operating system of a computer may periodically collect all the deleted space onto the free-storage list. Any technique, which does this collection, is called *garbage collection.* Garbage collection usually takes place in two steps. First the computer runs through all lists, tagging those cells which are currently in use, and then the computer runs through the memory, collecting all untagged space onto the free-storage list. The garbage collection may take place when there is only some minimum amount of space or no space at all left in the free-storage list, or when the CPU is idle and has time to do the collection. Generally speaking, the garbage collection is invisible to the programmer.

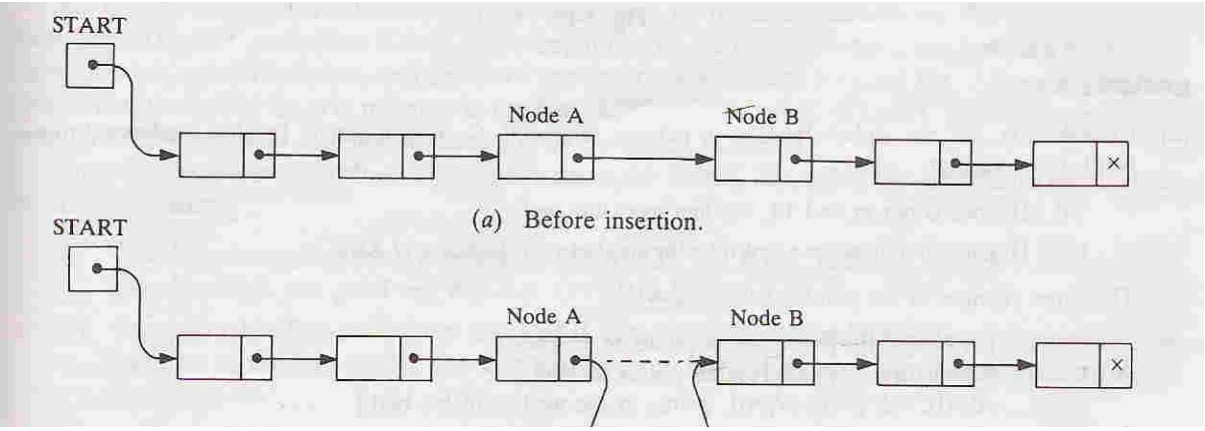
Overflow and Underflow

Sometimes new data are to be inserted into a data structure but there is no available space, i.e., the free-storage list is empty. This situation is usually called *overflow.* The programmer may handle overflow by printing the message OVERFLOW. In such a case, the programmer may then modify the program by adding space to the underlying arrays. Observe that overflow will occur with our linked lists when AVAIL = NULL and there is an insertion.

Analogously, the term *underflow* refers to the situation where one wants to delete data from a data structure that is empty. The programmer may handle underflow by printing the message UNDERFLOW. Observe that underflow will occur with our linked lists when START = NULL and there is a deletion.

INSERTION INTO A LINKED LIST

Let LIST be a linked list with successive nodes A and B, as pictured in Fig. *(a).* Suppose a node N is to be inserted into the list between nodes A and B. The schematic diagram of such an



(b) After Insertion

insertion appears in Fig. *(b)*. That is, node A now points to the new node N, and node N points to node B, to which A previously pointed.

Suppose our linked list is maintained in memory in the form

LIST (INFO, LINK, START, AVAIL)

These figures do not take into account that the memory space for the new node N will come from the AVAIL list. Specifically, for easier processing, the first node in the AVAIL list will be used for the new node N. Thus a more exact schematic diagram of such an insertion is that in Fig. below. Observe that three pointer fields are changed as follows:

(1) The next pointer field of node A now points to the new node N, to which AVAIL previously pointed.

(2) AVAIL now points to the second node in the free pool, to which node N previously pointed.

(1) The next pointer field of node N now points to. node B, to which node A previously pointed.

There are also two special cases. If the new node N is the first node in the list, then START will point to N; and if the new node N is the last node in the list, then N will contain the null pointer.

Insertion Algorithms

Algorithms, which insert nodes into linked lists, come up in various situations. We discuss three of them here. The first one inserts a node at the beginning of the list, the second one inserts a node after the node with a given location, and the third one inserts a node into a sorted list. All our algorithms assume that the linked list is in memory in the form LIST (INFO, LINK, START, AVAIL) and that the variable ITEM contains the new information to be added to the list.

Since our insertion algorithms will use a node in the AVAIL list, all of the algorithms will include the following steps:

(a) Checking to see if space is available in the AVAIL list. If not, that is, if AVAIL = NULL, then the algorithm will print the message OVERFLOW.

(b) Removing the first node from the AVAIL list. Using the variable NEW to keep track of the location of the new node, this step can be implemented by the pair of assignments (in this order)

NEW: = AVAIL, AVAIL: = LINK [AVAIL]

(c) Copying new information into the new node. In other words,

INFO [NEW]: = ITEM

Inserting at the Beginning of a List:

Suppose our linked list is not necessarily sorted and there is no reason to insert a new node in any special place in the list. Then the easiest place to insert the node is at the beginning of the list. An algorithm that does so follows.

Algorithm: INSFIRST (INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM as the first node in the list.

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list.]
   Set NEW:= AVAIL and AVAIL:= LINK[AVAIL].
3. Set INFO[NEW]:= ITEM. [Copies new data into new node.]
4. Set LINK[NEW]:= START. [New node now points to original first node.]
5. Set START:= NEW. [Changes START so it points to the new node.]
6. Exit.

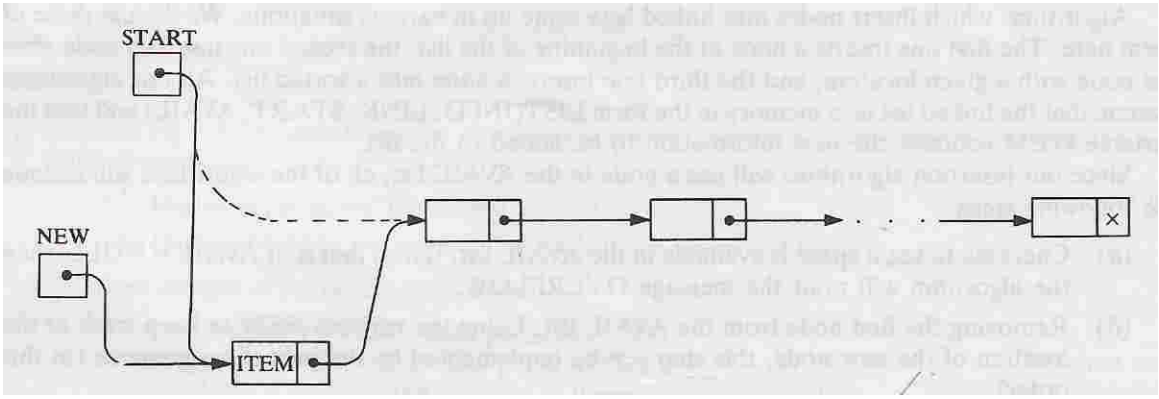


Fig: Insertion at the beginning of a list

Inserting after a Given Node

Suppose we are given the value of LOC where either LOC is the location of a node A in a linked LIST or LOC = NULL. The following is an algorithm which inserts ITEM into LIST so that ITEM follows node A or, when LOC = NULL, so that ITEM is the first node.

Let N denote the new node (whose location is NEW). If LOC = NULL, then N is inserted as the first node in LIST as in the previous algorithm. Otherwise, as pictured in Fig. below, we let node N point to node B (which originally followed node A) by the assignment

LINK [NEW]: = LINK [LOC]

and we let node A point to the new node N by the assignment

LINK [LOC] : = NEW

A formal statement of the algorithm follows.

Algorithm: INSLOC (INFO, LINK, START, AVAIL, LOC, ITEM)
This algorithm inserts ITEM so that ITEM follows the node with location LOC or inserts ITEM as the first node when LOC = NULL.

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list]
   Set NEW:= AVAIL and AVAIL:= LINK[AVAIL].
3. Set INFO [NEW]:= ITEM. [Copies new data into. new node.]
4. If LOC = NULL, then: [Insert as first node.]
   Set LINK [NEW] : = START and START: = NEW.
   Else: [Insert after node with location LOC.]
   Set LINK [NEW] : = LINK[LOC] and LINK[LOC]: = NEW.
   [End of If structure.]
5. Exit.

Inserting into a Sorted Linked List:

Suppose ITEM is to be inserted into a sorted linked LIST. Then ITEM must be inserted between nodes A and B so that

$$\text{INFO (A)} < \text{ITEM} \leq \text{INFO (B)}$$

The following is a procedure which finds the location LOC of node A, that is, which finds the location LOC of the last node in LIST whose value is less than ITEM.

Traverse the list, using a pointer variable PTR and comparing ITEM with INFO [PTR] at each node. While traversing, keep track of the location of the preceding node by using a pointer variable SAVE, as pictured in next fig. Thus SAVE and PTR are updated by the assignments

SAVE:= PTR and PTR:= LINK[PTR]

The traversing continues as long as INFO [PTR] > ITEM, or in other words, the traversing stops as soon as ITEM $\overline{<}$ INFO [PTR]. Then PTR points to node B, so SAVE will contain the location of the node A. I

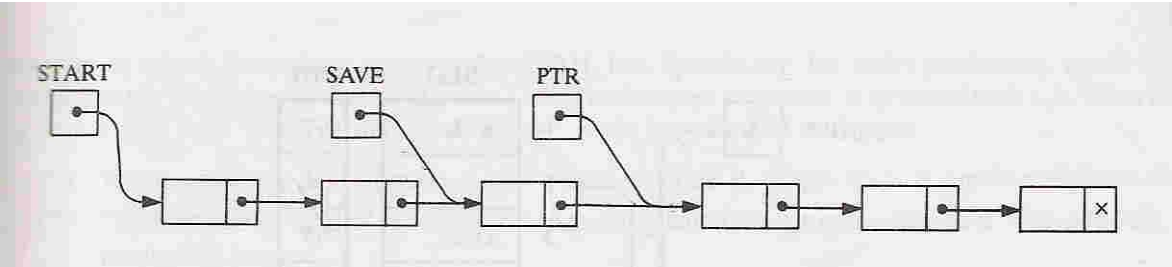
The formal statement of our procedure follows. The cases where the list is empty or where

ITEM < INFO [ST ART], so LOC = NULL, are treated separately, since they do not involve the variable SAVE.

Procedure: FINDA (INFO, LINK, START, ITEM, LOC)

This procedure finds the location LOC of the last node in a sorted list such that INFO [LOC] < ITEM, or sets LOC = NULL.

1. [List empty?] If START = NULL, then: Set LOC: = NULL, and Return.
2. [Special case?] If ITEM < INFO [START], then: Set LOC:= NULL, and Return .
3. Set SAVE:= START and PTR:= LINK[START]. [Initializes pointers.]
4. Repeat Steps 5 and 6 while PTR $\neq$ NULL.
5. If ITEM < INFO [PTR], then:
   Set LOC: = SAVE, and Return.
   [End of If structure.]
6. Set SAVE: = PTR and PTR: = LINK [PTR]. [Updates pointers.]
   [End of Step 4 loop.]
7. Set LOC: = SAVE.
8. Return.



Algorithm: INSSRT (INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM into a sorted linked list.

1. [Use previous procedure to find the location of the node preceding ITEM.]

   Call FINDA (INFO, LINK, START, ITEM, LOC).

2. [Use Algorithm INSLOC to insert ITEM after the node with location LOC.]

   Call INSLOC (INFO, LINK, START, AVAIL, LOC, ITEM).

3. Exit.

DELETION FROM A LINKED LIST:

Let LIST be a linked list with a node N between nodes A and B, as pictured in Fig.(a). Suppose node N is to be deleted from the linked list. The schematic diagram of such a deletion appears in Fig. (b). The deletion occurs as soon as the nextpointer field of node A is changed so that it points to node B. (Accordingly, when performing deletions, one must keep track of the address of the node which immediately precedes the node that is to be deleted.)

Suppose our linked list is maintained in memory in the form

LIST(INFO, LINK, START, AVAIL)

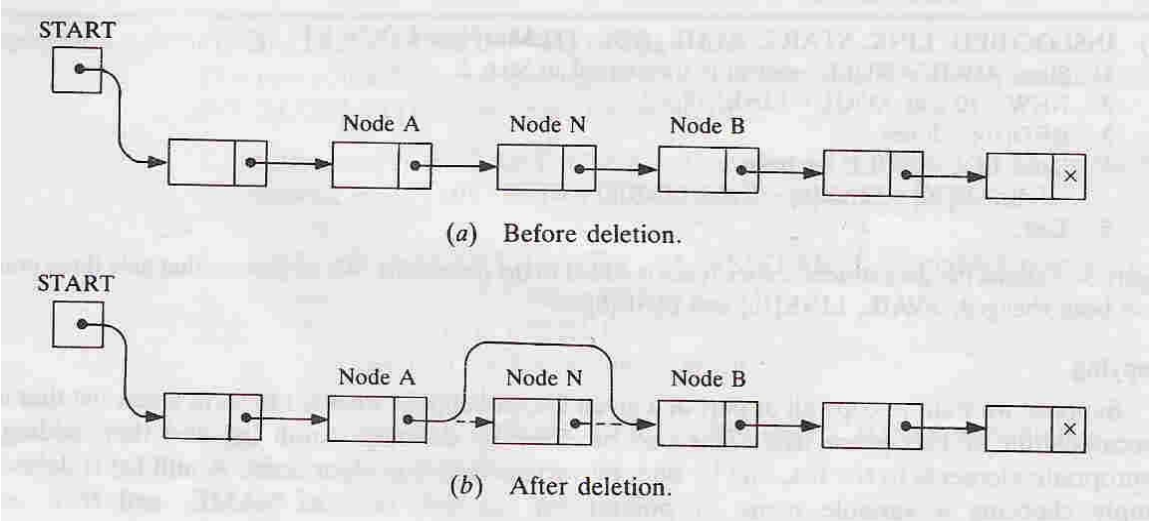


Figure above does not take into account the fact that, when a node N is deleted from our list, we

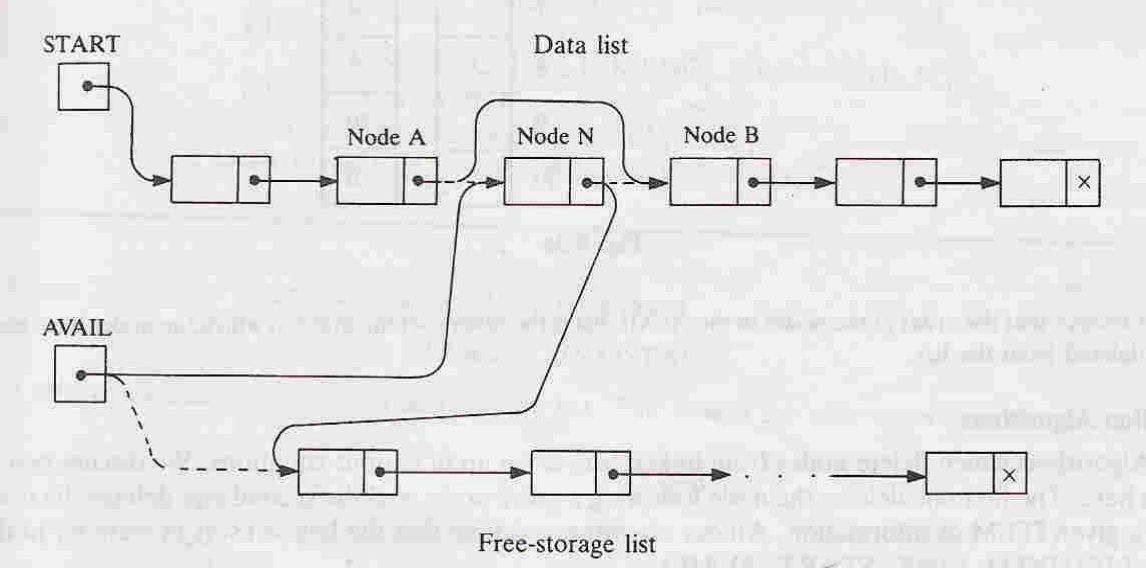
will immediately return its memory space to the AVAIL list. Specifically, for easier processing, it will be returned to the beginning of the AVAIL list. Thus a more exact schematic diagram of such a deletion is the one in Fig. below. Observe that three pointer fields are changed as follows:

(1) The nextpointer field of node A now points to node B, where node N previously pointed.

(2) The nextpointer field of N now points to the original first node in the free pool, where AVAIL previously pointed.

(3) AVAIL now points to the deleted node N.

There are also two special cases. If the deleted node N is the first node in 'the list, then START will point to node B; and if the deleted node N is the last node in the list, then node A will contain the NULL pointer.



Deletion Algorithms

Algorithms, which delete nodes from linked lists, come up in various situations. We discuss two of them here. The first one deletes the node following a given node, and the second one deletes the node with a given ITEM of information. All our algorithms assume that the linked list is in memory in the form LIST(INFO, LINK, START, AVAIL).

All of our deletion algorithms will return the memory space of the deleted node N to the beginning of the AVAIL list. Accordingly, all of our algorithms will include the following pair of assignments, where LOC is the location of the deleted node N:

LINK[LOC]:= AVAIL and then AVAIL:=LOC

These two operations are pictured in Fig. below.

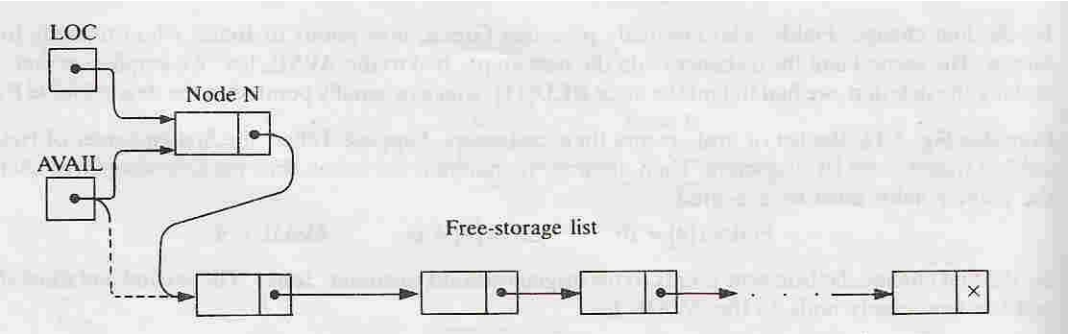


Fig: LINK[LOC]:= AVAIL and AVAIL:= LOC.

Some of our algorithms may want to delete either the first node or the last node from the list. An algorithm that does so must check to see if there is a node in the list. If not, i.e., if START = NULL, then the algorithm will print the message UNDERFLOW.

Deleting the Node Following a Given Node

Let LIST be a linked list in memory. Suppose we are given the location LOC of a node N in LIST. Furthermore, suppose we are given the location LOCP of the node preceding N or, when N is the first node, we are given LOCP = NULL. The following algorithm deletes N from the

list.

Algorithm: DEL (INFO, LINK, START, AVAIL, LOC, LOCP)

This algorithm deletes the node N with location LOC. LOCP is the location of the node which precedes N or, when N is the first node, LOCP = NULL.

1. If LOCP = NULL, then:

    Set START: = LINK[ST ART]. [Deletes first node.]

  Else:

    Set LINK[LOCP] : = LINK[LOC]. [Deletes node N.]

  [End of If structure.]

2. [Return deleted node to the AVAIL list.]

  Set LINK[LOC]:= AVAIL and AVAIL:= LOC.

3. Exit.

Figure below is the schematic diagram of the assignment

START:= LINK[START]

which effectively deletes the first node from the list. This covers the case when N is the first node.

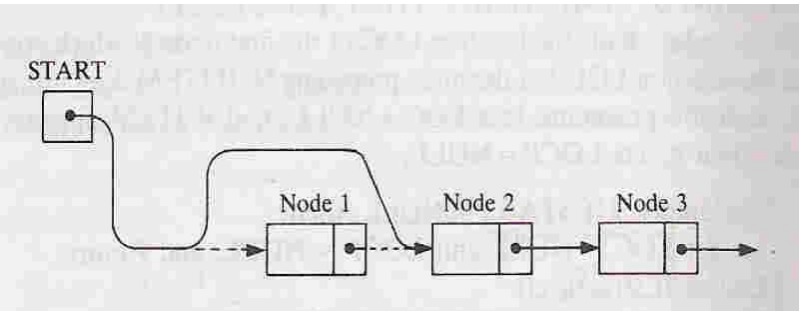


Fig: START:= LINK[START].

Figure below is the schematic diagram of the assignment

LINK[LOCP] : = LINK[LOC]

which effectively deletes the node N when N is not the first node.

The simplicity of the algorithm comes from the fact that we are already given the location LOCP of the node, which precedes node N. In many applications, we must first find LOCP.

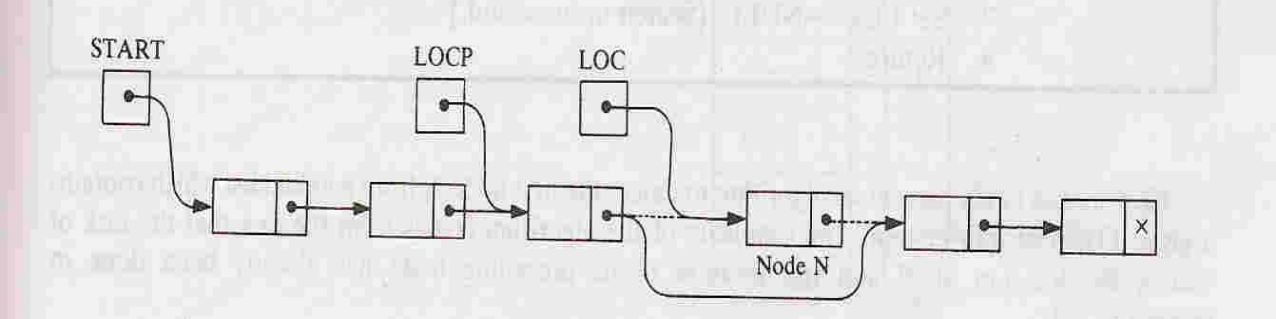


Fig: LINK [LOCP] : = LINK[LOC]

Lecture 20: More on linked lists

Summary of Lecture:

- Operations of linked lists
- Types of linked lists

Deleting the Node with a Given ITEM of Information:

Let LIST be a linked list in memory. Suppose we are given an ITEM of information and we want to delete from the LIST the first node N which contains ITEM. (If ITEM is a key value, then only one node can contain ITEM.) Recall that before we can delete N from the list, we need to know the location of the node preceding N. Accordingly, first we give a procedure which finds the location LOC of the node N containing ITEM and the location LOCP of the node preceding node N. If N is the first node, we set LOCP = NULL, and if ITEM does not appear in LIST, we set LOC = NULL.

Traverse the list, using a pointer variable PTR and comparing ITEM with INFO[PTR] at each node. While traversing, keep track of the location of the preceding node by using a pointer variable SAVE. Thus SAVE and PTR are updated by the assignments

SAVE:=PTR and PTR : = LINK[PTR]

The traversing continues as long as INFO[PTR] $\neq$= ITEM, or in other words, the traversing stops as soon as ITEM = INFO[PTR]. Then PTR contains the location LOC of node N and SAVE contains the location LOCP of the node preceding N.

The formal statement of our procedure follows. The cases where the list is empty or where INFO [START] = ITEM (i.e., where node N is the first node) are treated separately, since they

do not involve the variable SAVE.

Procedure: FINDB (INFO, LINK, START, ITEM, LOC, LOCP)

This procedure finds the location LOC of the first node N which contains ITEM and the location LOCP of the node preceding N. If ITEM does not appear in the list, then the procedure sets LOC = NULL; and if ITEM appears in the first node, then it sets LOCP = NULL.

1. [List empty?] If START = NULL, then:
   Set LOC: = NULL and LOCP : = NULL, and Return.
   [End of If structure.]
2. [ITEM in first node?] If INFO[ST ART] = ITEM, then:
   Set LOC:= START and LOCP = NULL, and Return.
   [End of If structure.]
3. Set SAVE:= START and PTR:= LINK[START]. [Initializes pointers.]
4. Repeat Steps 5 and 6 while PTR =/= NULL.
5. If INFO[PTR] = ITEM, then:
   Set LOC:= PTR and LOCP:= SAVE, and Return.
   [End of If structure.]
6. Set SAVE:= PTR and PTR:= LINK[PTR]. [Updates pointers.]
   [End of Step 4 loop.]
7. Set LOC: = NULL. [Search unsuccessful.]
8. Return.

Now we can easily present an algorithm to delete the first node N from a linked list, which contains a given ITEM of information. The simplicity of the algorithm comes from the fact that the task of finding the location of N and the location of its preceding node has already been done in this procedure.

Algorithm: DELETE(INFO, LINK, START, AVAIL, ITEM)

This algorithm deletes from a linked list the first node N that contains the given ITEM of information.

1. [Use above procedure to find the location of N and its preceding node.]
   Call FINDB(INFO, LINK, START, ITEM, LOC, LOCP)
2. If LOC = NULL, then: Write: ITEM not in list, and Exit.
3. [Delete node.]
   If LOCP = NULL, then:
   Set START:= LINK[START]. [Deletes first node.]
   Else:
   Set LINK[LOCP] : = LINK[LOC).
   [End of If structure.]
4. [Return deleted node to the AVAIL list.]
   Set LINK[LOC) : = AVAIL and AVAIL: = LOC.
5. Exit.

HEADER LINKED LISTS:

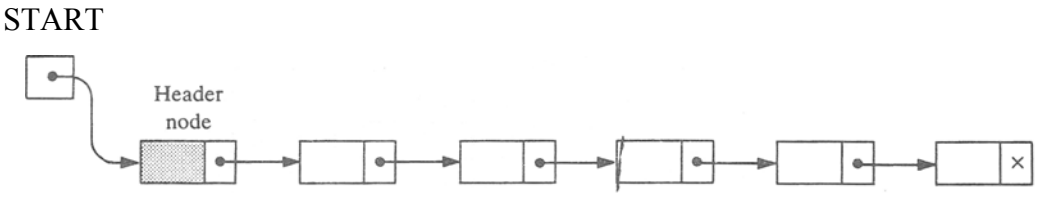
A *header* linked list is a linked list which always contains a special node, called the *header node,* at the beginning of the list. The following are two kinds of widely used header lists:

(1) A *grounded header list* is a header list where the last node contains the null pointer. (The term "grounded" comes from the fact that many texts use the electrical ground symbol to indicate the null pointer.)

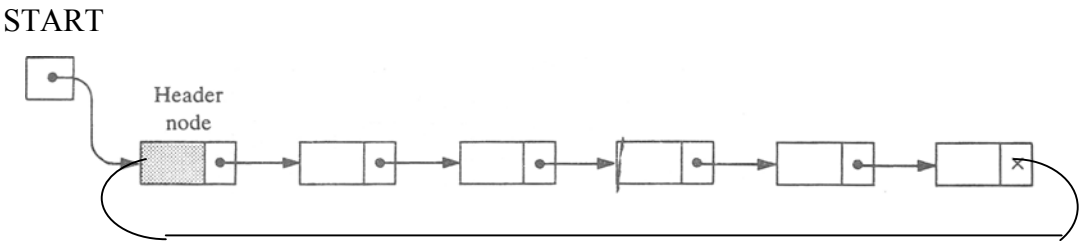
(2) A *circular header list* is a header list where the last node points back to the header node.

Figure below contains schematic diagrams of these header lists. Unless otherwise stated or implied, our header lists will always be circular. Accordingly, in such a case, the header node also acts as a sentinel indicating the end of the list.

Observe that the list pointer START always points to the header node. Accordingly, LINK[START] = NULL indicates that a grounded header list-is empty, and LINK[START] = START indicates that a circular header list is empty.



(a) Grounded header list.



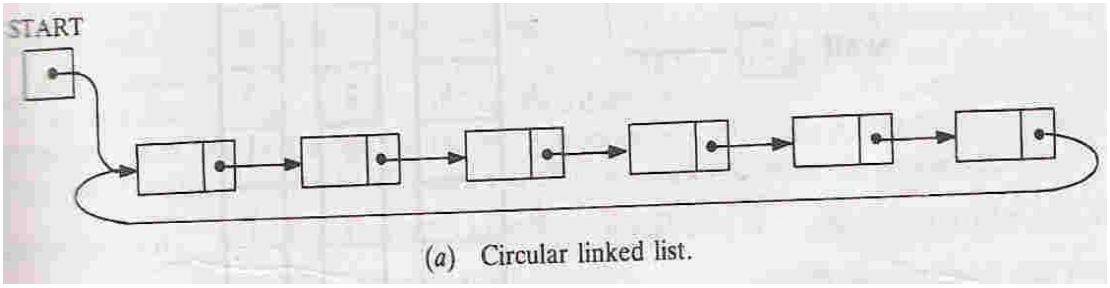
(b) Circular header list.

There are two other variations of linked lists:

(1) A linked list whose last node points back to the first node instead of containing the null pointer, called a ***circular list.***

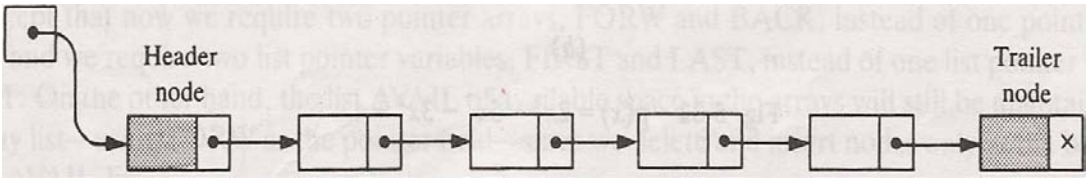
(2) A linked list, which contains both a special header node at the beginning of the list and a special trailer node at the end of the list.

Figure below contains schematic diagrams of these lists.



(*a*) Circular linked list.

START



(b) Linked list with header and trailer nodes.

Polynomials:

Header linked lists are frequently used for maintaining polynomials in memory. The header node plays an important part in this representation, since it is needed to represent the zero polynomial. This representation of polynomials will be presented in the context of a specific example.

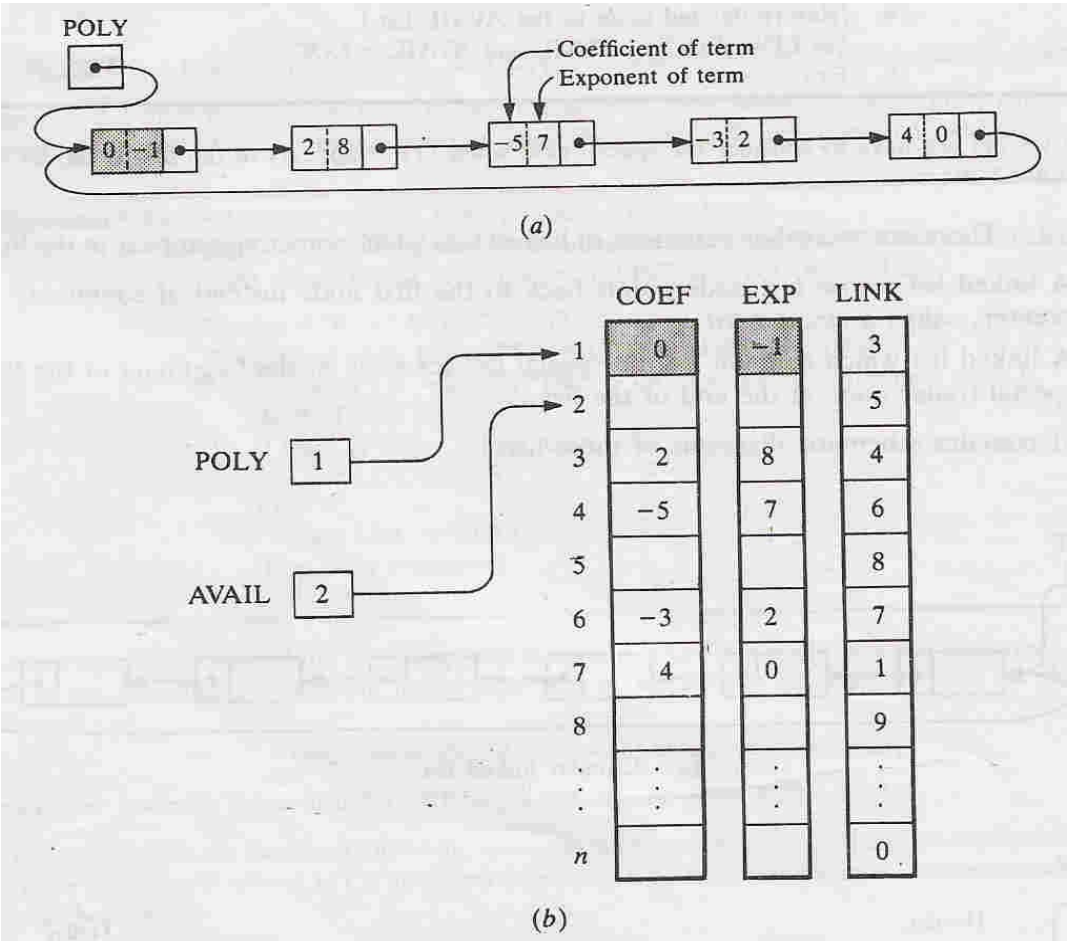
EXAMPLE:

Let *p(x)* denote the following, polynomial in one variable (containing four nonzero terms):

$$p(x) = 2x^8 - 5x^7 - 3x^2 + 4$$

Then *p(x)* may be represented by the header list pictured in Fig.*(a),* where each node corresponds to a nonzero term of *p(x).* Specifically, the information part of the node is divided into two fields representing, respectively, the coefficient and the exponent of the corresponding term, and the nodes are linked according to decreasing degree.

Observe that the list pointer variable POLY points to the header node, whose exponent field is assigned a negative number, in this case -1. Here the array representation of the list-will require three linear arrays, which we will call COEF, EXP and LINK. One such representation appears in Fig. *(b).*



TWO-WAY LISTS

Each list discussed above is called a one-way list, since there is only one way that the list can be traversed. That is, beginning with the list pointer variable START, which points to the first node or the header node, and using the nextpointer field LINK to point to the next node 'in the list, we can traverse the list in only one direction. Furthermore, given the location LOC of a node N in such a list, one has immediate access to the next node in the list (by evaluating LINK[LOC]), but one does not have access to the preceding node without traversing part of the list. This means, in particular, that one must traverse that part of the list preceding N in order to delete N from the list.

A two-way list, can be traversed in two directions in the usual forward direction from the beginning of the list to the end, or in the backward direction from the end of the list to the beginning. Furthermore, given the location LOC of a node N in list, one now has immediate access to both the next node and the preceding node in the list. This means, in particular, that one is able to delete N from the list without traversing any part of the list.

A *two-way list* is a linear collection of data elements, called *nodes,* where each node N is divided into three parts:

1) An information field INFO which contains the data of N
2) A pointer field FORW which contains the location of the next node in the list
3) A pointer field BACK which contains the location of the preceding node in the list

The list also requires two list pointer variables: FIRST, which points to the first node in the list, and LAST, which points to the last node in the list. Figure below contains a schematic diagram of such a list. Observe that the null pointer appears in the FORW field of the last node in the list and also in the BACK field of the first node in the list.

FIRST

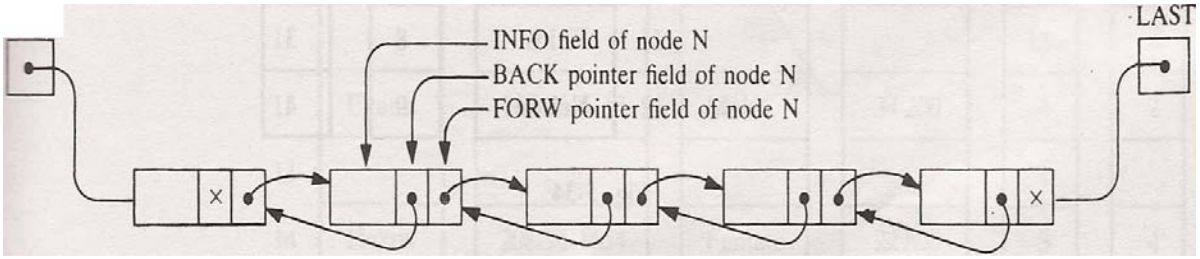


Fig: Two-way list

Observe that, using the variable FIRST and the pointer field FORW, we can traverse a two-way list in the forward direction as before. On the other hand, using the variable LAST and the pointer field BACK, we can also traverse the list in the backward direction.

Suppose LOCA and LOCB are the locations, respectively, of nodes A and B in a two-way

list. Then the way that the pointers FORW and BACK are defined gives us the following:

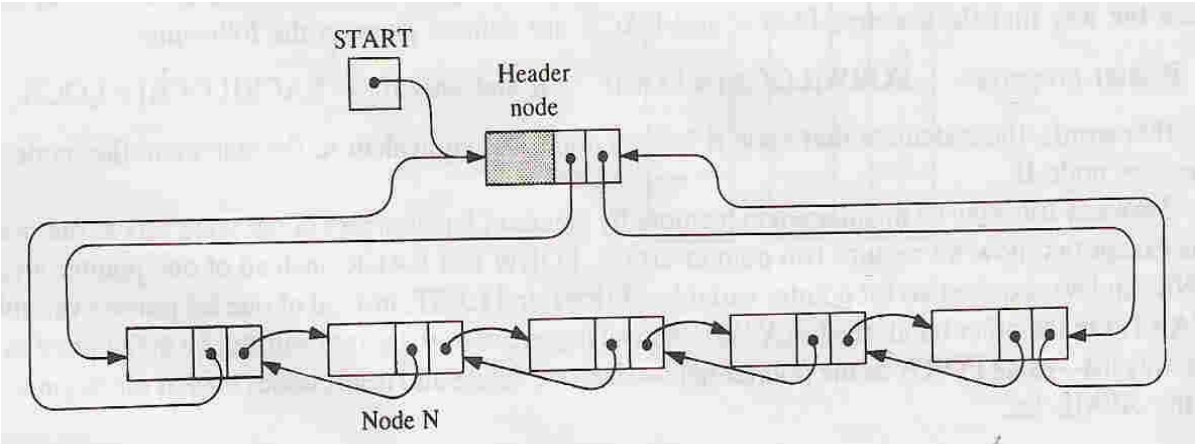
Pointer property: FORW[LOCA] = LOCB if and only if BACK[LOCB] = I.OCA

In other words, the statement that node B follows node A is equivalent to the statement that node A precedes node B.

Two-way lists may be maintained in memory by means of linear arrays in the same way as one-way lists except that now we require two pointer arrays, FORW and BACK, instead of one pointer array LINK, and we require two list pointer variables, FIRST and LAST, instead of one list pointer variable START. On the other hand, the list AVAIL of available space in the arrays will still be maintained as a one-way list-using FORW as the pointer field-since we delete and insert nodes only at the beginning of the AVAIL list.

Two-Way Header Lists:

The advantages of a two-way list and a circular header list may be combined into a two-way circular header list as pictured in Fig. below. The list is circular because the two end nodes point back to the header node. Observe that such a two-way list requires only one list pointer variable START, which points to the header node. This is because the two pointers in the header node point to the two ends of the list.



Review Questions:

1. Suppose LIST is a linked list in memory consisting of numerical values. Write a procedure for each of the following:
    (a) Finding the maximum MAX of the values in LIST.
    (b) Finding the average MEAN of the values in LIST.
    (c) Finding the product PROD of the elements in LIST.
2. What is the concept of Garbage Collection?
3. What is the Avail Pool?
4. In what different ways can insertions be done in a linked list?
5. How do you delete elements from a linked list?

Lecture 21: Implementation and discussion of previous topics

Lecture 22 : TREES

Summary of Lecture:

- What is a tree?
- Types of trees
- Applications of trees

Trees

So far, we have been studying mainly linear types of data structures: strings, arrays, lists, stacks and queues. This lesson defines a nonlinear data structure called a tree. This structure is mainly used to represent data containing a hierarchical relationship between elements, e.g., records, family trees and tables of contents.

GENERAL TREES

A *general tree* (sometimes called a *tree*) is defined to be a nonempty finite set T of elements, called nodes, such that:

(1) T contains a distinguished element R, called the *root* of T.
(2) The remaining elements of T form an ordered collection of zero or more disjoint trees $T1, T_2, \ldots, T_m$.

The trees $T_1, T_2, \ldots, T_m$ are called *subtrees* of the root R, and the roots of T_1 $T_2, \ldots,$ T

$_m$ are called *successors* of R.

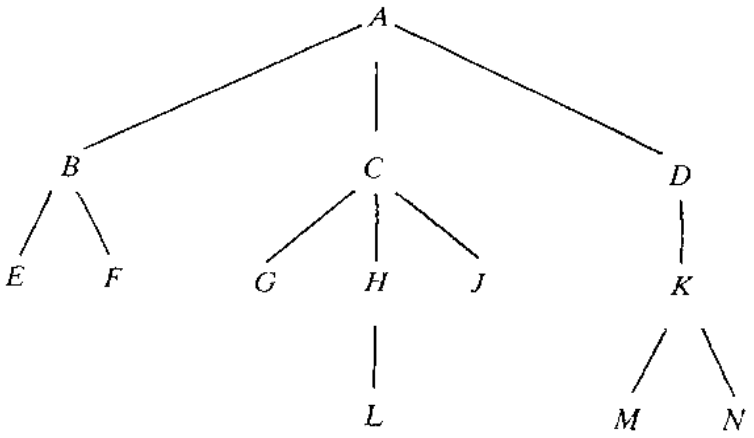
Terminology from family relationships, graph theory and horticulture is used for general trees in the same way as for binary trees. In particular, if N is a node with successors S_1 , S_2, . . ., S_m then N is called the *parent* of the S_i's, the S_i's are called *children* of N, and the S_i's, are called *siblings* of each other.

The term "tree" comes up, with slightly different meanings, in many different areas of mathematics and computer science. Here we assume that our general tree T is *rooted*, that is, that T has a distinguished node R called the root of T; and that T is *ordered*, that is, that the children of each node N of T have a specific order. These two properties are not always required for the definition of a tree.

EXAMPLE:

Figure below pictures a general tree *T* with 13 nodes,

A, B, C, D, E, F, G, H, J, K, L, M, N



Unless otherwise stated, the root of a tree *T* is the node at the top of the diagram, and the children of a node are ordered from left to right. Accordingly, *A* is the root of *T,* and *A* has three children; the first child *B,* the second child C and. the third child *D.* Observe that:

(a) The node C has three children.

(b) Each of the nodes *Band K* has two children.

(c) Each of the nodes *D* and *H* has only one child.

(d) The nodes *E, F,* G, *L, J, M* and *N* have no children.

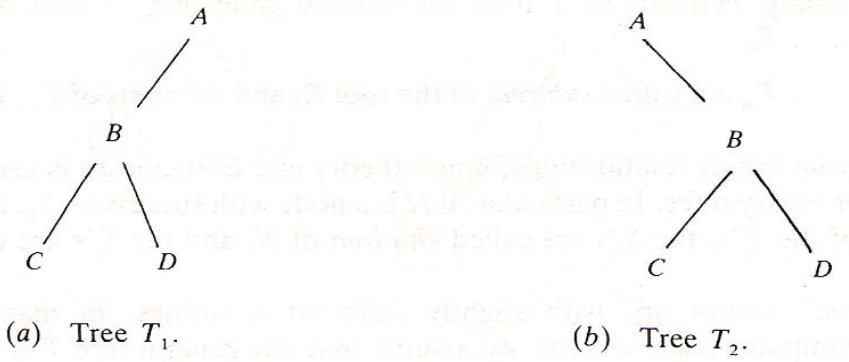
The last group of nodes, those with no children, are called *terminal nodes.*

A binary tree *T* is not a special case of a general tree *T:* binary trees and general trees are different objects. The two basic differences follow:

(1) A binary tree *T'* may be empty, but a general tree *T is* nonempty.

(2) Suppose a node *N* has only one child. Then the child is distinguished as a left child or right child in a binary tree *T',* but no such distinction exists in a general tree *T.*

The trees T1 and T2 in Fig below illustrate the second difference. Specifically, as binary trees, T_1 and T_2 are distinct trees, since *B* is the left child of *A* in the tree T_1 but *B* is the right child of *A* in the tree T_2. On the other hand, there is no difference between the trees T_1 and T_2 as general trees.



(*a*) Tree T_1. (*b*) Tree T_2.

A *forest F* is defined to be an ordered collection of zero or more distinct trees. Clearly, if we delete the root *R* from a general tree *T,* then we obtain the forest *F* consisting of the subtrees of *R* (which ma) be empty). Conversely, if *F* is a forest, then we may adjoin a node *R* to *F* to form a general tree *T* where *R* is the root of *T* and the subtrees of *R* consist of the original trees in *F*.

Computer Representation of General Trees

Suppose *T* is a general tree. T is usually maintained in memory by means of a linked representation, which uses three parallel arrays INFO, CHILD (or DOWN) and SIBL (or HORZ), and a pointer variable ROOT as follows. First of all, each node *N* of *T* will correspond to a location *K* such that:

(1) INFO[K] contains the data at node *N*.

(2) CHILD[K] contains the location of the first child of *N.* The condition

CHILD[K] = NULL indicates that *N* has no children.

(3) SIBL[K] contains the location of the next sibling of *N.* The condition SIBL[K] = NULL indicates that *N* is the last child of its parent.

Furthermore, ROOT will contain the location of the root *R* of *T.* Although this representation may seem artificial, it has the important advantage that each node *N* of *T,* regardless of the number children of *N,* will contain exactly three fields.

Terminology

Terminology describing family relationships is frequently used to describe relationships between the nodes of a tree T.

N is a **node** in T with **successors** S_1, S_2 and S_3.

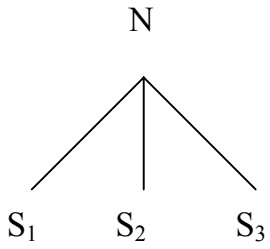


Figure: Tree 'T'

N is called the **parent** (or **father**) of S_1, S_2 and S_3. Analogously, S_1 is called the left **child** (or **son**) of N, and S_2 is called the right child (or son) of N. Furthermore, S_1 and S_2 are said to be **siblings** (or **brothers**).

A tree with the parent node having 0, 1 or 2 successors is a **binary tree**. Every node N in a binary tree T, except the root, has a unique parent, called the **predecessor** of N.

The terms **descendant** and **ancestor** have their usual meaning. That is, a node L is called a descendant of a node N (and N is called an ancestor of L) if there is a succession of children from N to L. In particular, L is called a left or right descendant of N according to whether L belongs to the left or right subtree of N.

Terminology from graph theory and horticulture is also used with a binary tree T. Specifically, the line drawn from a node N of T to a successor is called an **edge**, and a sequence of consecutive edges is called a **path**. A **terminal node** is called a **leaf**, and a path ending in a leaf is called a **branch**.

Each node in a binary tree T is assigned a **level number**, as follows. The root R of the tree T is assigned the level number 0, and every other node is assigned a level number, which is 1 more than the level number of its parent. Furthermore, those nodes with the same level number are said to belong to the same generation.

The **depth** (or **height**) of a tree T is the maximum number of nodes in a branch of T. This turns out to be 1 more than the largest level number of T. The tree T in Fig. (a) below has depth 5.

Binary trees T and T' are said to be **similar** if they have the same structure or, in other words, if they have the same shape. The trees are said to be **copies** if they are similar and if they have the same contents at corresponding nodes.

BINARY TREES

A binary tree T is defined as a finite set of elements, called nodes, such that:
(a) T is empty (called the null tree or empty tree), or
(b) T contains a distinguished node R, called the root of T, and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2.
If T does contain a root R, then the two trees T_1 and T_2 are called, respectively, the left and right subtrees of R. If T_1 is nonempty, then its root is called the left successor of R; similarly, if T2 is nonempty, then its root is called the right successor of R.

A binary tree T is frequently presented by means of a diagram. Specifically, the diagram in Fig. (a) represents a binary tree T as follows. (i) T consists of 11 nodes, represented by the letters A through L, excluding I. (ii) The root of T is the node A at the top of the diagram. (iii) A left-downward slanted line from a node N indicates a left successor of N, and a right-downward slanted line from N indicates a right successor of N. Observe that:

(a) B is a left successor and C is a right successor of the node A.

(b) The left subtree of the root A consists of the nodes B, D, E and F, and the right subtree of A consists of the nodes C, G, H, J, K and L.

Any node N in a binary tree T has 0, 1 or 2 successors. The nodes A, B, C and H have two successors, the nodes E and J have only one successor, and the nodes D, F, G, Land K have no successors. The nodes with no successors are called terminal nodes.

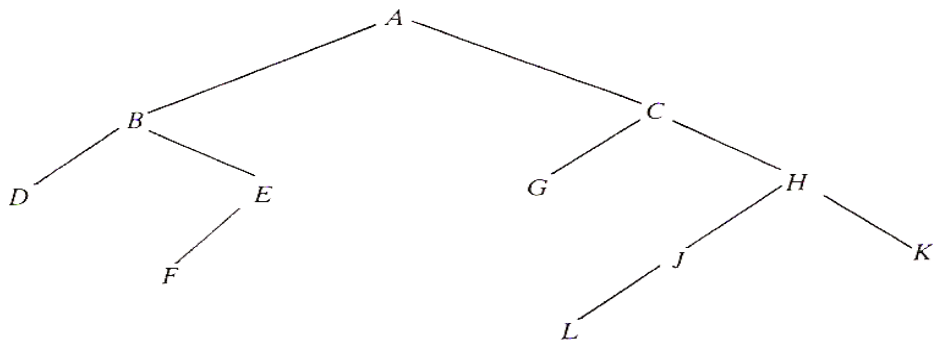


Figure (a)

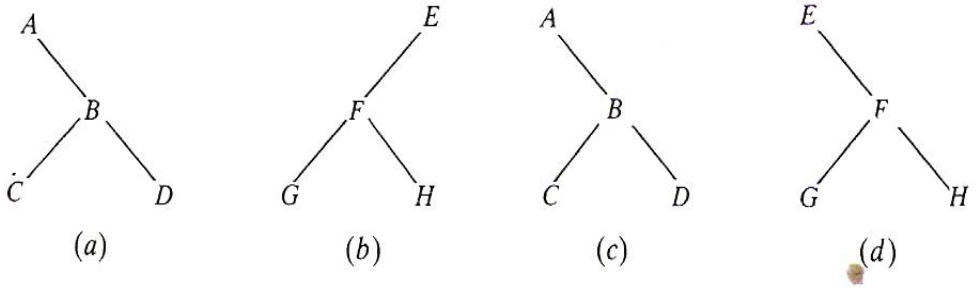
The above definition of the binary tree T is recursive since T is defined in terms of the binary subtrees T_1 and T_2. This means, in particular, that every node N of T contains a left and a right subtree. Moreover, if N is a terminal node, then both its left and right sub trees are empty.

Binary trees T and T' are said to be similar if they have the same structure or, in other words, if they have the same shape. The trees are said to be copies if they are similar and if they have the same contents at corresponding nodes.

EXAMPLE

Consider the four binary trees in Fig. below. The three trees (a), (c) and (d) are *similar*. In particular, the trees (a) and (c) are *copies* since they also have the same data at corresponding nodes. The tree (b) is neither similar nor a copy of the tree (d) because, in

a binary tree, we distinguish between a left successor and a right successor even when there is only one successor.



EXAMPLE: Algebraic Expressions

Consider any algebraic expression E involving only binary operations, such as

E=(a-b)/((c*d)+e)

E can be represented by means of the binary tree T pictured in figure below. That is, each variable or constant in E appears as an "internal" node in T whose left and right subtrees correspond to the operands of the operation. For example:

(a) In the expression E, the operands of + are c * d and e.

(b) In the tree T, the sub trees of the node + correspond to the sub expressions c * d and e.

Clearly every algebraic expression will correspond to a unique tree, and vice versa.

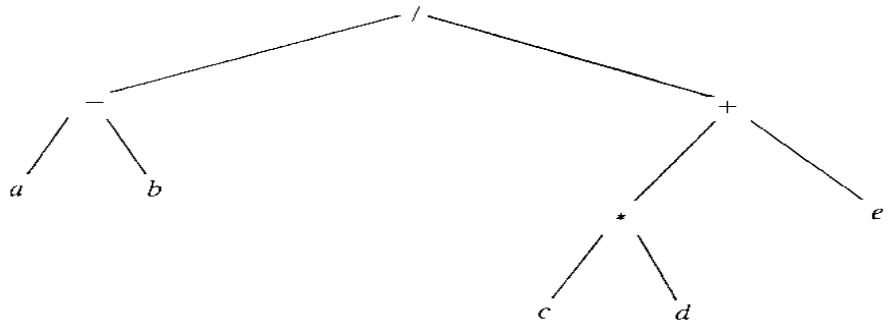


Figure: E = (a – b) / ((c * d) + e)

Complete Binary Trees

Consider any binary tree T. Each node of T can have at most two children. Accordingly, one can show that level r of T can have at most 2' nodes. The tree T is said to be complete if all its levels, except possibly the last, have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible. Thus there is a unique complete tree T_n with exactly n nodes (we are, of course, ignoring the contents of the nodes). The complete tree T_{26} with 26 nodes appears in Fig. below.

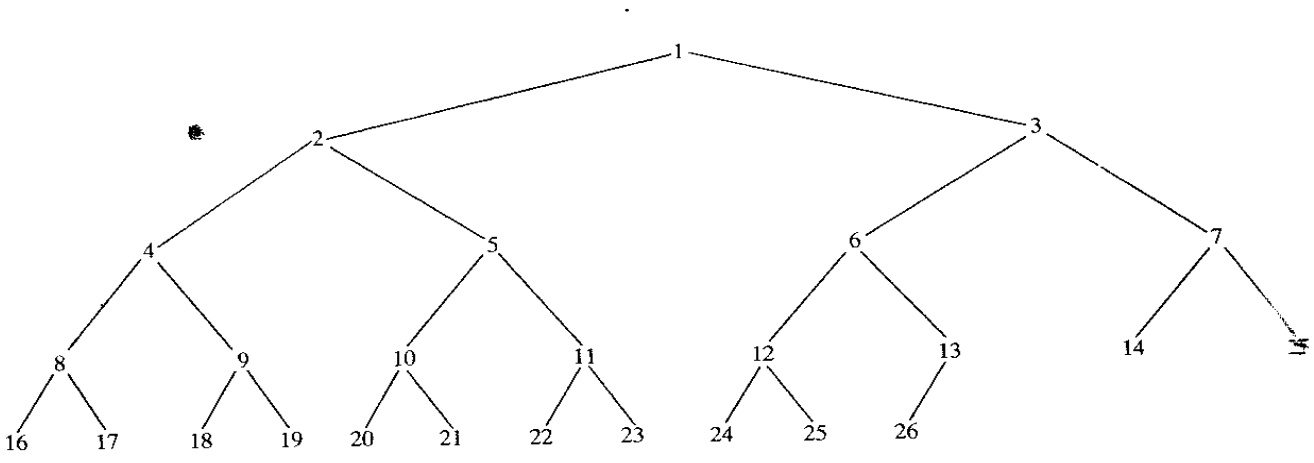


Figure: Complete tree T_{26}

The nodes of the complete binary tree T_{26} in Fig. above have been purposely labeled by the integers 1, 2, . . . , 26, from left to right, generation by generation. With this labeling, one can easily determine the children and parent of any node K in any complete tree T_n Specifically, the left and right children of the node K are, respectively, 2 * K and 2 * K + 1, and the parent of K is the node [K/2]. For example, the children of node 9 are the nodes 18 and 19, and its parent is the node [9/2] = 4. The depth d_n of the complete tree T_n with n nodes is given by

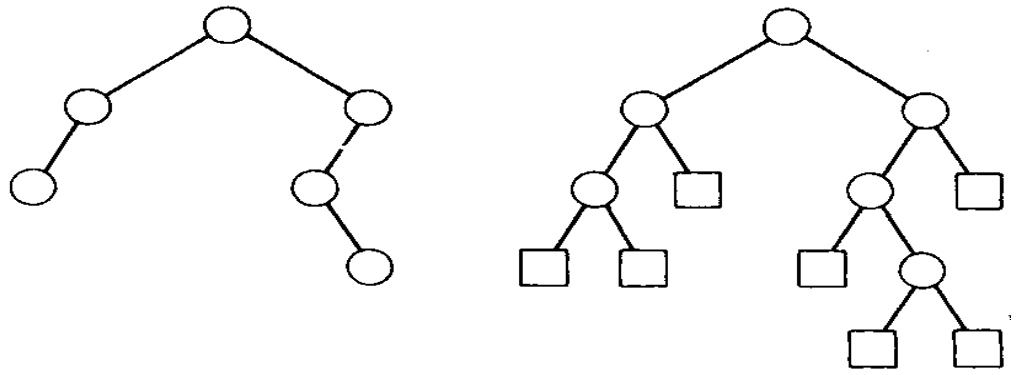
$$D_n = [\text{Log}_2 n + 1]$$

This is a relatively small number. For example, if the complete tree T_n has n = 1000000 nodes, then its depth D_n = 21.

Extended Binary Trees: 2-Trees

A binary tree T is said to be a **2-tree** or an **extended binary tree** if each node N has either 0 or 2 children such a case, the nodes with 2 children are called internal nodes, and the nodes with 0 children are called **external nodes**. Sometimes the nodes are distinguished in diagrams by using circles for internal nodes and squares for external nodes.....

The term "extended binary tree" comes from the following operation. Consider any binary tree T, such as the tree in Fig. (a). Then T may be "converted" into a 2-tree by replacing each empty subtree by a new node, as pictured in Fig.(b). Observe that the new tree is, indeed, a 2-tree. Furthermore, the nodes in the original tree T are now the internal nodes in the extended tree, and the new nodes are the external in the extended tree.



(a) Binary tree T. (b) Extended 2-tree.

An important example of a 2-tree is the tree T corresponding to any algebraic expression E that uses only binary operations. As illustrated earlier, the variables in E will appear as the external nodes, and the operations in E will appear as, internal **nodes**.

REPRESENTING BINARY TREES IN MEMORY

Let T be a binary tree. This section discusses two ways of representing T in memory. The first and always is called the link representation of T and is analogous to the way linked lists are represented in memory. The second way, which uses a single array, called the sequential representation of T. The main requirement of any representation of T is that one should have direct access to the root R of T and, given any node N of T, one should have direct access to the children of N.

Linked Representation of Binary Trees

Consider a binary tree T. Unless otherwise stated or implied, T will be maintained in memory by means of a linked representation which uses three parallel arrays, INFO, LEFT and RIGHT, and a pointer variable ROOT as follows. First of all, each node N of T will correspond to a location K such that:

(1) INFO [K] contains the data at the node N.

(2) LEFT [K] contains the locations of the left child of node N.

(3) RIGHT [K] contains the Location of the right child of node N.

Furthermore, ROOT will contain the location of the root R of T. If any subtree is empty, then the corresponding pointer will contain the null value; if the tree T itself is empty, then ROOT will contain the null value.

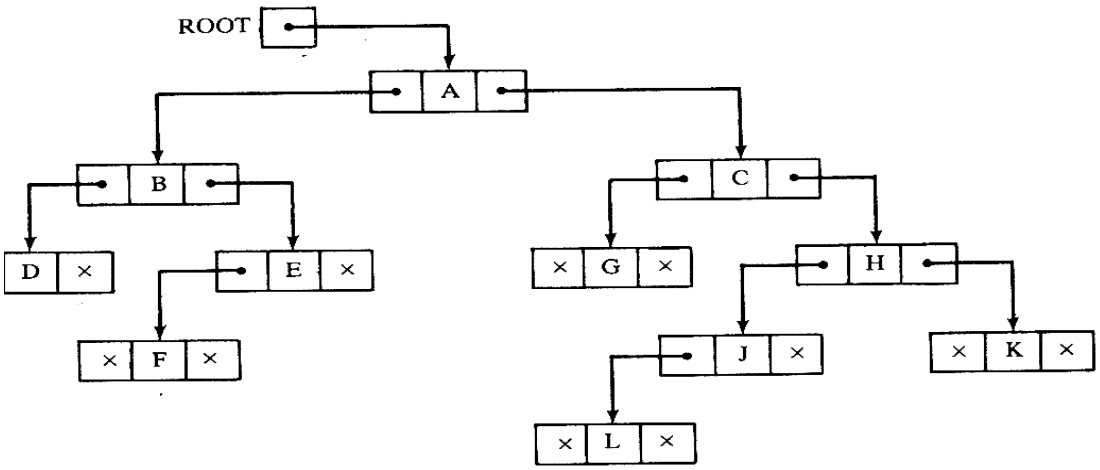
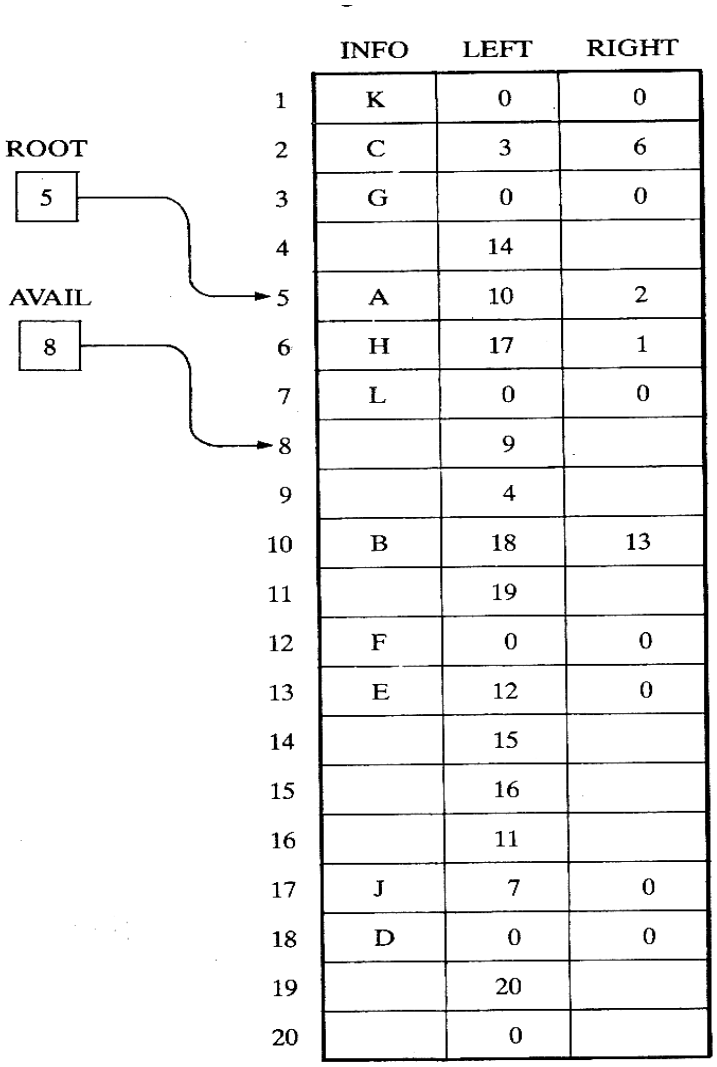


Figure (a)



	INFO	LEFT	RIGHT
1	K	0	0
2	C	3	6
3	G	0	0
4		14	
5	A	10	2
6	H	17	1
7	L	0	0
8		9	
9		4	
10	B	18	13
11		19	
12	F	0	0
13	E	12	0
14		15	
15		16	
16		11	
17	J	7	0
18	D	0	0
19		20	
20		0	

Figure (b)

Sequential Representation of Binary Trees

Suppose T is a binary tree that is complete or nearly complete. Then there is an efficient way of maintaining T in memory called the *sequential representation* of T. This representation uses only a single linear array TREE as follows:

(a) The root R of T is stored in TREE [1].

(b) If a node N occupies TREE [K], then its left child is stored in TREE [2 * K]

and its right child is stored in TREE [2 * K + 1].

Again, NULL is used to indicate an empty sub tree. In particular, TREE [1] = NULL indicates that the tree is empty.

The sequential representation of the binary tree T in Fig. (a) appears in Fig. (b). Observe that we require 14 locations in the array TREE even though T has only 9 nodes. In fact, if we included null entries for the successors of the terminal nodes, then we would actually require TREE [29] for the right successor of TREE [14]. Generally speaking, the sequential representation of a tree with depth d will require an array with approximately 2^{d+l} elements. Accordingly, this sequential representation is usually inefficient unless, as stated above, the binary tree T is complete or nearly complete. For example, the tree *T* in Fig. 7-1 has 11 nodes and depth 5, which means it would require an array with approximately $2^6 = 64$ elements.

Lecture 23: Memory representation of trees, traversal algorithms

Summary of Lecture:

- How is a tree represented in memory?
- Types of trees
- Tree traversal algorithms
- Applications of trees

TRAVERSING BINARY TREES

There are three standard ways of traversing a binary tree T with root R. These three algorithms, called preorder, inorder and postorder, are as follows:

Preorder:
(1) Process the root R.
(2) Traverse the left sub tree of R in preorder.
(3) Traverse the right subtree of R in preorder.

Inorder:
(1) Traverse the left subtree of R in inorder.
(2) Process the root R.
(3) Traverse the right subtree of R in inorder.

Postorder:
(1) Traverse the left subtree of R in postorder.
(2) Traverse the right subtree of R in postorder.
(3) Process the root R.

Observe that each algorithm contains the same three steps, and that the left subtree of R is

always traversed before the right subtree. The difference between the algorithms is the lime at which the root R is processed. Specifically, in the "pre" algorithm, the root R is processed before the subtrees are traversed; in the "in" algorithm, the root R is processed between the traversals of the subtrees; and in the "post" algorithm, the root R is processed after the sub trees are traversed.

The three algorithms are sometimes called, respectively, the node-left-right (NLR) traversal, the left-node-right (LNR) traversal and the left-right-node (LRN) traversal.

TRAVERSAL ALGORITHMS USING STACKS

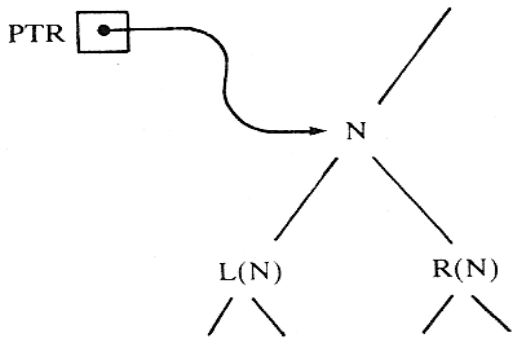
Suppose a binary tree T is maintained in memory by some linked representation

TREE (INFO, LEFT, RIGHT, ROOT)

This section discusses the implementation of the three standard traversals of T, which were defined recursively in the last section, by means of no recursive procedures using stacks. These are as:

Preorder Traversal

The preorder traversal algorithm uses a variable PTR (pointer), which will contain the location of the node N currently being scanned. This is pictured in fig below where L(N) denotes the left child of node N and R(N) denotes the right child. The algorithm also uses an array STACK, which will hold the addresses of nodes for future processing.



Algorithm: Initially push NULL onto STACK and then set PTR: = ROOT. Then repeat the following steps until PTR = NULL or, equivalently, while PTR ≠ NULL.

(a) Proceed down the left-most path rooted at PTR, processing each node N on the path and pushing each right child R (N), if any, onto STACK. The traversing ends after a node N with no left child L (N) is processed. (Thus PTR is updated using the assignment PTR: = LEFT[PTR], and the traversing stops when LEFT[PTR] = NULL.)

(b) [Backtracking.] Pop and assign to PTR the top element on STACK.
If PTR≠NULL, then return to Step (a); otherwise Exit.

(We note that the initial element NULL on STACK is used as a sentinel.)

Inorder Traversal

The inorder traversal algorithm also uses a variable pointer PTR, which will contain the location of the node N currently being scanned, and an array STACK, which will hold the addresses of nodes for future processing. In fact, with this algorithm, a node is processed only when it is popped from STACK.

Algorithm:

Initial by push NULL onto STACK (for a sentinel) and then set PTR: = ROOT. Then repeat the following steps until NULL is popped from STACK.

(a) Proceed down the left-most path rooted at PTR, pushing each node N onto STACK and stopping when a node N with no left child is pushed onto

STACK.

(b) [Backtracking.] Pop and process the nodes on STACK. If NULL is popped, then Exit. If a node N with a right child R(N) is processed, set PTR = R(N) (by assigning PTR: = RIGHT[PTRD and return to Step (a).

Review Questions:

Define:

Tree

Binary tree

Node

Child

Siblings

Similar trees

Degree of a tree

Path

Successors

Copy tree

Terminal nodes

Depth of a Tree

Lecture 24: More on trees

HEADER NODES; THREADS

Consider a binary tree T. Variations of the linked representation of T are frequently used because certain operations on T are easier to implement by using the modifications. Some of these variations, which are analogous to header and circular linked lists, are discussed in this section.

Header Nodes

Suppose a binary tree T is maintained in memory by means of a linked representation. Sometimes an extra, special node is called a *header node* is added to the beginning of T. When this extra node is used, the tree pointer variable, which we will call HEAD (instead of ROOT), will point to the header node, and the left pointer of the header node will point to the root of T. Figure on next page shows a schematic picture of the binary tree in Fig.(a) below that uses a linked representation with a header nodes.

Suppose a binary tree T is empty. Then T will still contain a header node, but the left pointer of the header node will contain the null value. Thus the condition

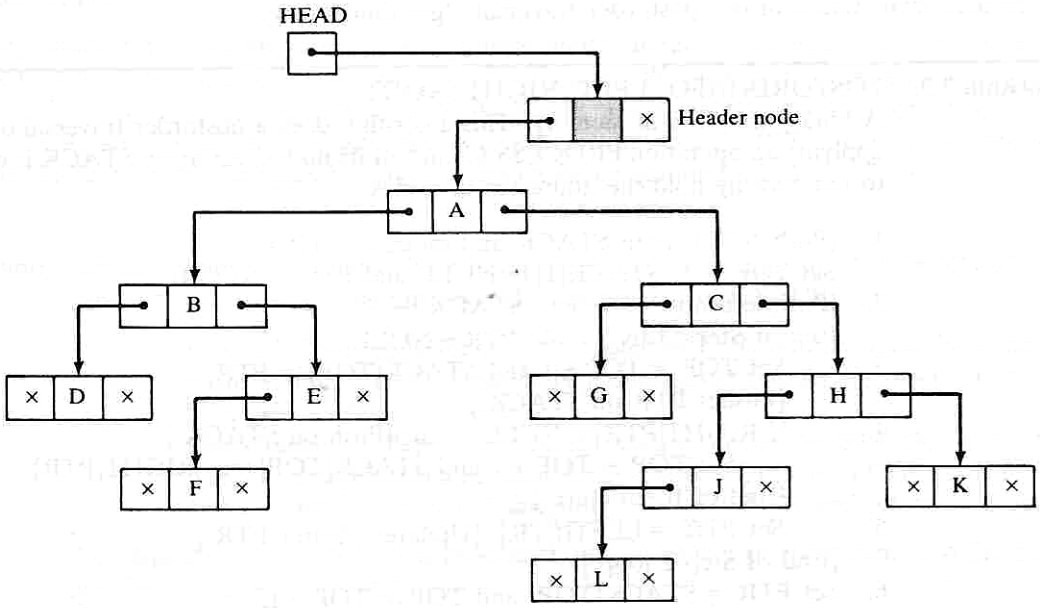


Figure (a)

LEFT[HEAD] = NULL

will indicate an empty tree.

BINARY SEARCH TREES

Now we discuss one of the most important data structures in computer science, a binary search tree. This structure enables one to search for and find an element with an average running time f (n) = O ($\log_2$ n). It also enables one to easily insert and delete elements. This structure contrasts with the following structures:

(a) *Sorted linear array:* Here one can search for and find an element with a running

time f(n) = O(log2 n), but it is expensive to insert and delete elements.

(b) *Linked list*: Here one can easily insert and delete elements, but it is expensive to search for and find an element, since one must use a linear search with running time f(n) = O(n).

Although each node in a binary search tree may contain an entire record of data, the definition of the binary tree depends on given fields whose values are distinct and may be ordered.

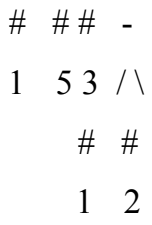
Suppose T is a binary tree. Then T is called a binary search tree (or binary sorted tree) if each node N of T has the following property: The value at N is greater than every value in the left subtree of N and is less than every value in the right subtree of N. (It is not difficult to see that this property guarantees that the inorder traversal of T will yield a sorted listing of the elements of T.)

Expression Trees

There are a number of ways of writing expressions: normal *infix* notation, *prefix* (function) notation, *postfix* (reverse Polish) notation, and as trees. Trees are hard to represent in our normal text-based writing systems, however, they are easy to use in a computer.

For this example assume we have the +, -, *, and / operators and that we just have integers. The tree for (7-5) * (3+(1-2)) would look like:





Each node in the tree can be represented as a struct/class, with the following fields:

- The operator, which we can encode as a char variable. We'll represent integers with the special operator '#' and we'll have a field that contains the integer value.
- The integer value. This field is always there, but is only used with the '#' operator.
- A pointer to the the left subtree if it's not the '#' operator (a terminal node).
- A pointer to the right subtree as above.

Processing trees is made especially easy because it can be done in a nice, compact, natural, recursive manner.

See the two examples of how to declare this:

- Expression Tree Struct
- Expression Tree Class

Expression Trees can also be dealt in this manner:

Algebraic expressions such as

$$a/b + (c - d)e$$

have an inherent tree-like structure. For example, Figure below is a representation of the expression in Equation. This kind of tree is called an *expression tree*.

The terminal nodes (leaves) of an expression tree are the variables or constants in the expression (*a*, *b*, *c*, *d*, and *e*). The non-terminal nodes of an expression tree are the operators (+, -, ×, and ÷). Notice that the parentheses which appear in Equation do not appear in the tree. Nevertheless, the tree representation has captured the intent of the parentheses since the subtraction is lower in the tree than the multiplication.

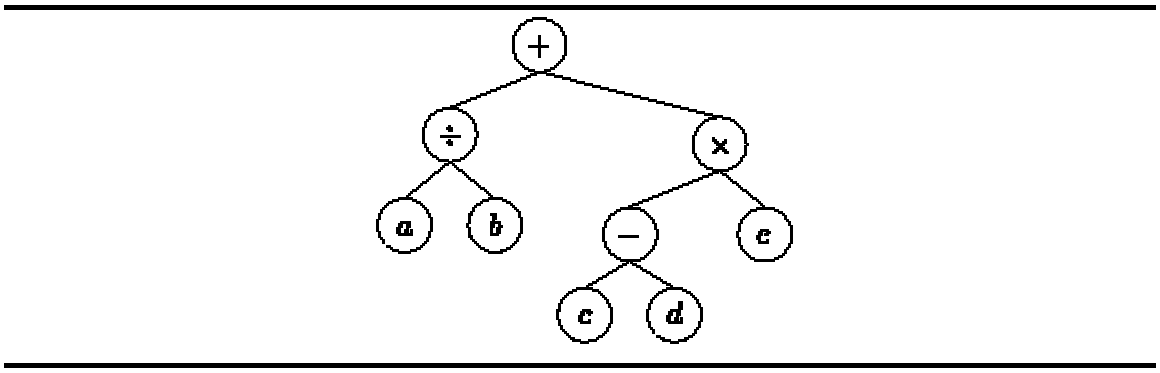


Figure: Tree representing the expression *a*/*b*+(*c*-*d*)*e*

The common algebraic operators are either unary or binary. E.g., addition, subtraction, multiplication and division are all binary operations and negation is a unary operation. Therefore, the non-terminal nodes of the corresponding expression trees have either one or two non-empty subtrees. I.e., expression trees are usually binary trees.

What can we do with an expression tree? Perhaps the simplest thing to do is to print the expression represented by the tree. Notice that an inorder traversal of the tree in Figure above visits the nodes in the order

$$a, \div, b, +, c, -, d, c.$$

Except for the missing parentheses, this is precisely the order in which the symbols appear in Equation above.

This suggests that an *inorder* traversal should be used to print the expression. Consider an inorder traversal which, when it encounters a terminal node simply prints it out; and when it encounters a non-terminal node, does the following:

1. Print a left parenthesis; and then
2. traverse the left subtree; and then
3. print the root; and then
4. traverse the right subtree; and then
5. print a right parenthesis.

Applying this procedure to the tree given in Figure we get

$$((a \div b) + ((c - d) \times c)),$$

Which, despite the redundant parentheses, represents exactly the same expression as Equation above.

Lecture 25: Problem solving using tree algorithms

Lecture 26: Types of trees and Graphs

Summary of Lecture:

- Quad trees

Quad Trees

Definition

A *quad tree* is a tree whose nodes either are terminal nodes or have 4 children. The children are ordered 1, 2, 3, 4.

Quad Tree Strategy

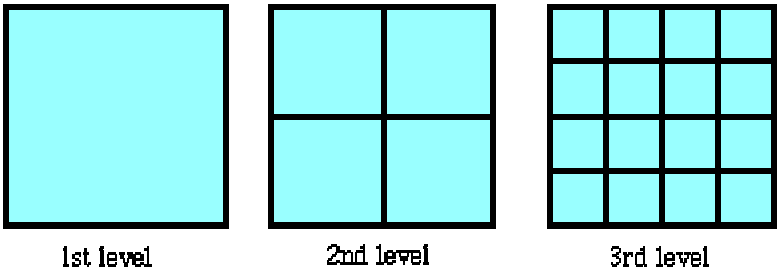
Here we will outline the strategy behind using quad trees as a data structure for pictures. The key is to "Divide and Conquer".

Let's say we divide the picture area into 4 sections. Those 4 sections are then further divided into 4 subsections. We continue this process, repeatedly dividing a square region by 4. We must impose a limit to the levels of division otherwise we could go on dividing the picture forever. Generally, this limit is imposed due to storage considerations or to limit processing time or due to the resolution of the output device. A *pixel* is the smallest subsection of the quad tree.

To summarise, a square or *quadrant* in the picture is either:

a) Entirely one colour
b) composed of 4 smaller sub-squares

In terms of a quad tree, the children of a node represent the 4 quadrants. The root of the tree is the entire picture.



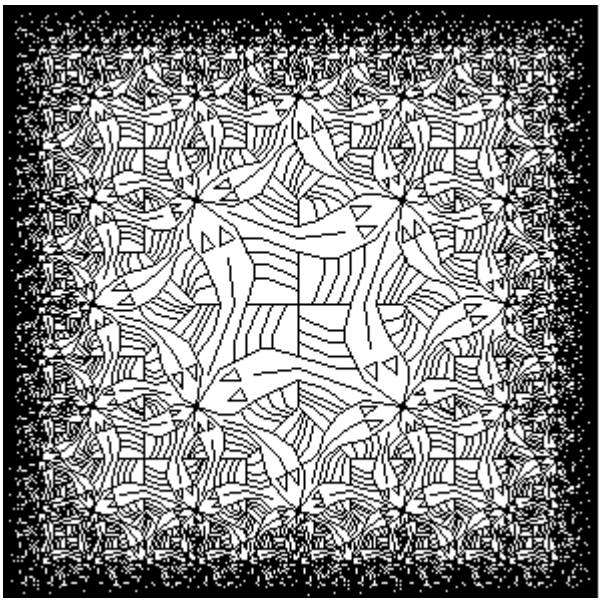
First three levels of a quad tree

To represent a picture using a quad tree, each leaf must represent a uniform area of the picture. If the picture is black and white, we only need one bit to represent the colour in each leaf; for example, 0 could mean black and 1 could mean white.

Note that no node may allow all its descendants to have the same colour. A minimum level of division must be maintained.

Advantages and Disadvantages of Quad Tree Picture Representation

Quad trees used extensively in computer graphics. Mainly, quad trees can be manipulated and accessed much quicker than other models. For that reason, quad trees are very popular in fractal graphics. Recursive pictures can be implemented easily using quad trees: the root of the quad tree has four children, where one of the children is the actual image and the other three point to the root.



A recursive image

Other advantages of quad trees include:

- Erasing a picture takes only one step. All that is required is to set the root node to neutral.
- Zooming to a particular quadrant in the tree is a one step operation.
- To reduce the complexity of the image, it suffices to remove the final level of nodes.
- Accessing particular regions of the image is a very fast operation. This is useful for updating certain regions of an image, perhaps for an environment with multiple windows.

The only drawback of quad trees is that they take up a lot of space. If a quad tree is implemented using links, most of the memory will be taken up by the links. Nevertheless, there are ways of compacting quad trees, which is important for transferring data efficiently.

Graphs

INTRODUCTION

This topic investigates another nonlinear data structure: the *graph.* As we have done with other data structures, we discuss the representation of graphs in memory and present various operations and algorithms on them.

GRAPH THEORY TERMINOLOGY

This section summarizes some of the main terminology associated with the theory of graphs.

Graphs and Multigraphs

A graph G consists of two things:

(1) A set *V of* elements ca1led *nodes* (or *points* or *vertices)*

(2) A set *E* of *edges* such that each edge *e* in *E* is identified with a unique (unordered) pair *[u, v]* of nodes in *V,* denoted by *e = [u, v]*

Sometimes we indicate the parts of a graph by writing G = *(V, E).*

Suppose *e = [u, v].* Then the nodes *u* and *v* are called the *endpoints* of *e,* and *u* and *v* are said to be *adjacent nodes* or *neighbors.* The *degree* of a node *u,* written *deg (u),* is the number of edges containing *u.* If *deg (u)* = O-that is, if *u* does not belong to any edge-then *u* is called an *isolated* node.

A *path P* of *length n* from a node *u* to a node *v* is defined as a sequence of *n* + 1 nodes.

$$P=(v_0, v_1, v_2, \ldots, v_n)$$

such that $u = vo;$ v_{i-1} is adjacent to v_i for $i = 1,2, \ldots, n;$ and $v_n = v.$ The path *P* is said to

be *closed* if $v_o = v_n$. The path *P* is said to be *simple* if all the nodes are distinct, with the exception that v_o may equal v_n; that is, *P* is simple if the nodes $v_o, v_1 \ldots, v_{n-1}$ are distinct and the nodes $v_1, v_2 \ldots, v_n$ are distinct. A *cycle* is a closed simple path with length 3 or more. A cycle of length *k* is called a *k-cycle.*

A graph G is said to be *connected* if there is a path between any two of its nodes. If there is a path *P* from a node *u* to a node *v,* then, by eliminating unnecessary edges, one can obtain a simple path Q from *u* to *v;* accordingly, we can state the following proposition.

Proposition : A graph G is connected if and only if there is a simple path between any two nodes in G.

A graph G is said to be *complete* if every node *u* in G is adjacent to every other node *v* in G.

Clearly such a graph is connected. A complete graph with *n* nodes will have $n(n-1)/2$ edges.

A connected graph *T* without any cycles is called a *tree graph* or *free tree* or, simply, a *tree.* This means, in particular, that there is a unique simple path *P* between any two nodes *u* and *v* in *T.* Furthermore, if *T* is a finite tree with *m* nodes, then *T* will have $m-1$ edges.

A graph G is said to be *labeled* if its edges are assigned data. In particular, G is said to be *weighted* if each edge e in G is assigned a nonnegative numerical value *wee)* called the *weight* or *length* of e. In such a case, each path *P* in G is assigned a *weight* or *length, which* is the sum of the weights of the edges along the path *P.* If we are given no other information about weights, we may assume any graph G to be weighted by assigning the weight $w(e) = 1$ to each edge *e* in G.

The definition of a graph may be generalized by permitting the following:

(1) Multiple edges. Distinct edges *e* and *e′* are called *multiple edges* if they connect the same endpoints, that is, if *e = [u, v]* and *e′ = [u, v]*.

(2) *Loops.* An edge *e* is called a *loop* if it has identical endpoints, that is, if *e = [u, u]*.

Such a generalization *M* is called a *multigraph.* In other words, the definition of a graph usually does not allow either multiple edges or loops.

A multi graph *M* is said to be *finite* if it has a finite number of nodes and a finite number of edges *Observe that a graph G with a finite number of nodes must automatically h*ave *a* finite number *of edges* and so must be finite; but this is not necessarily true for a multigraph *M,* since *M* may have multiple edges.

EXAMPLE

(a) Figure *(a)* is a picture of a connected graph with 5 *nodes-A, B,* C, *D* and E-and 7 edges:

[A, B], [B, C], *[C, D], [D, E], [A, E], [C, E], [A,* C]

There are two simple paths of length 2 from *B* to *E: (B, A, E)* and *(B,* C, *E).* There is only one simple path of length 2 from *B* to *D: (B,* C, *D).* We note that *(B, A, D)* is not a path, since *[A, D]* is not an edge. There are two 4-cycles in the graph:

[A, B, C, *E, A]* and *[A,* C, *D, E, A]*.

Note that *deg(A)* = 3, since *A* belongs to 3 edges. Similarly, deg(C) = 4 and *deg(D)* = 2.

(b) Figure *8-1(b)* is not a graph but a multigraph. The reason is that it has multiple *edges-e4 = [B,* C] and *e5 = [B,* C]-and it has a loop, *e6 = [D, D].* The definition of a graph

usually does not allow either multiple edges or loops.

(c) Figure (c) is a tree graph with $m = 6$ nodes and, consequently, $m - 1 = 5$ edges. The reader can verify that there is a unique simple path between any two nodes of the tree graph.

(d) Figure *(d)* is the same graph as in Fig. *(a),* except that now the graph is weighted. Observe that $P_1 = (B, C, D)$ and $P_2 = (B, A, E, D)$ are both paths from node *B* to node *D.* Although P_2 contains more edges than P_i the weight $w(P_2) = 9$ is less than the weight $w(P_1) = 10$.

Directed Graphs

A *directed graph* G, also called a *digraph* or *graph,* is the same as a multigraph except that each edge *e* in G is assigned a direction, or in other words, each edge *e* is identified- with an ordered pair *(u, v)* of nodes in G rather than an unordered pair *[u, v].*

Suppose G is a directed graph with a directed edge $e = (u, v)$. Then *e* is also called an *arc.* Moreover, the following terminology is used:

(1) *e begins* at *u* and *ends* at *v.*

(2) u is the *origin* or *initial point* of *e,* and *v* is the *destination* or *terminal point* of *e.*

(3) u is a *predecessor* of *v,* and *v* is a *successor* or *neighbor* of *u.*

(4) u is *adjacent to v,* and *v* is *adjacent to u.*

The *outdegree* of a node *u* in G, written *outdeg(u),* is the number of edges beginning at u. Similarly, *the indegree* of *u,* written indeg(u), is the number of edges ending at *u.* A node *u* is called a *source* if it has a positive outdegree but zero indegree. Similarly, *u* is called a *sink* if it has a zero outdegree but a positive indegree.

The notions of *path, simple path* and *cycle* carryover from undirected graphs to

directed graphs except that now the direction of each edge in a path (cycle) must agree with the direction of the path (cycle). A node *v* is said to be *reachable* from a node *u* if there is a (directed) path from *u* to *v.*

A directed graph G is said to be *connected,* or *strongly connected,* if for each pair *u, v* of nodes in G there is a path from *u* to *v* and there is also a path from *v* to *u.* On the other hand, G is said to be *unilaterally connected* if for any pair *u, v* of nodes in G there is a path from *u* to *v* or a path from *v* to *u.*

Lecture 27: Study of Complexity factor, Big 'Oh' criteria

Summary of Lecture:

- Adjacency Matrix
- Big Oh criteria

SEQUENTIAL REPRESENTATION OF GRAPHS; ADJACENCY MATRIX; PATH MATRIX

There are two standard ways of maintaining a graph G in the memory of a computer. One way, called the *sequential representation* of G, is by means of its adjacency matrix *A.* The other way, called the *linked representation* of G, is by means of linked lists of neighbors.

Regardless of the way one maintains a graph G in the memory of the computer, the graph G is normally input into the computer by using its formal definition: a collection of nodes and a collection of edges.

Adjacency Matrix

Suppose G is a simple directed graph with *m* nodes, and suppose the nodes of G have been ordered and are called $v_1, v_2, \ldots, v_m$. Then the *adjacency matrix* $A = (a_{ij})$ of the graph G is the *m* x *m* matrix defined as follows:

$a_{ij} =$ 1 *if* v_i is adjacent to v_j that is, if there is an edge (v_i, v_j)

0 otherwise

Such a matrix *A,* which contains entries of only 0 and 1, is called a *bit matrix* or a *Boolean matrix.*

The adjacency matrix *A* of the graph G does depend on the ordering of the nodes of G; that is, a different ordering of the nodes may result in a different adjacency matrix. However, the matrices resulting from two different orderings are closely related in that one can be obtained from the other by simply interchanging rows and columns. Unless otherwise stated, we will assume that the nodes of our graph G have a fixed ordering.

Suppose G is an undirected graph. Then the adjacency matrix *A* of G will be a *symmetric matrix,* i.e., one in which $a_{ij} = a_{ji}$ for every *i* and *j.* This follows from the fact that each undirected edge *[u, v]* corresponds to the two directed edges *(u, v)* and *(v, u).*

The above matrix representation of a graph may be extended to multigraphs. Specifically, if G is a multigraph, then the *adjacency matrix* of G is the *m* x *m* matrix $A = (a_{ij})$ defined by setting *au* equal to the number of edges from v_i to v_j. .

EXAMPLE :

Consider the graph G in fig below. Suppose the nodes are stored in memory in a linear array *DATA* as fo*llows;*

DATA : X,Y,Z,W

Then we assume that the ordering of the nodes in G is as follows v_1=X, v_2= Y, v_3 = Z and v_4 =W. The adjacency matrix *A* of G is as follows:

Note that the number of 1's in A is equal to the number of edges in G.

Consider the powers A, A^2, A^3…….of the adjacency matrix A of a graph G. Let

$a_k(i, j)$ = the ij entry in the matrix A^K

Observe that $a_1 (i, j) = a_{ij}$ gives the number of paths of length 1 from node *vi* to node V_j. One can show that $a_2 (i, j)$ gives the number of paths of length 2 from v_i to V_j.

Selection of most suitable Data structures

Sorting	**Complexity**
Bubble	$O(n^2)$
Insertion	$O(n^2)$
Selection	$O(n^2)$
Quick Sort	$O(n)$
Shell Sort	$O(n(\log n)^2)$
Heap sort	$O(n^2)$

Big-Oh notation

running time = F(N)

We are interested in how much the running time grows as N (the input size) grows. When the input size N is *sufficiently large* we are interested in the **rate of growth of F**.

We are not interested in the fact that the running time of an algorithm is better than another for a particular input size.

We look at the rate of growth making the following assumptions (approximate analysis):

- For sufficiently large N, the value of a function (e.g. 10N3 + N2 + 40N + 80) is largely determined by its dominant term (N3).
- The exact value of the leading constant of the dominant term is not meaningful across different machines / compilers.
- Small values of N are not important. The meaning of sufficiently large varies from function to function.

There are several ways of estimating the running time of a program. We will look at growth rate as represented by Big-Oh notation which establishes a relative order among functions by comparing dominant terms.

For example the running time of a quadratic algorithm is specified as O(N2) (pronounced “order N squared”).

Big-Oh is an upper-bound.

The running time function for linear, O(N log N), quadratic, and cubic are represented by the following graphs:

Note that quadratic and cubic algorithms are not competitive with the others for reasonably large inputs.

Functions in order of increasing growth rate:

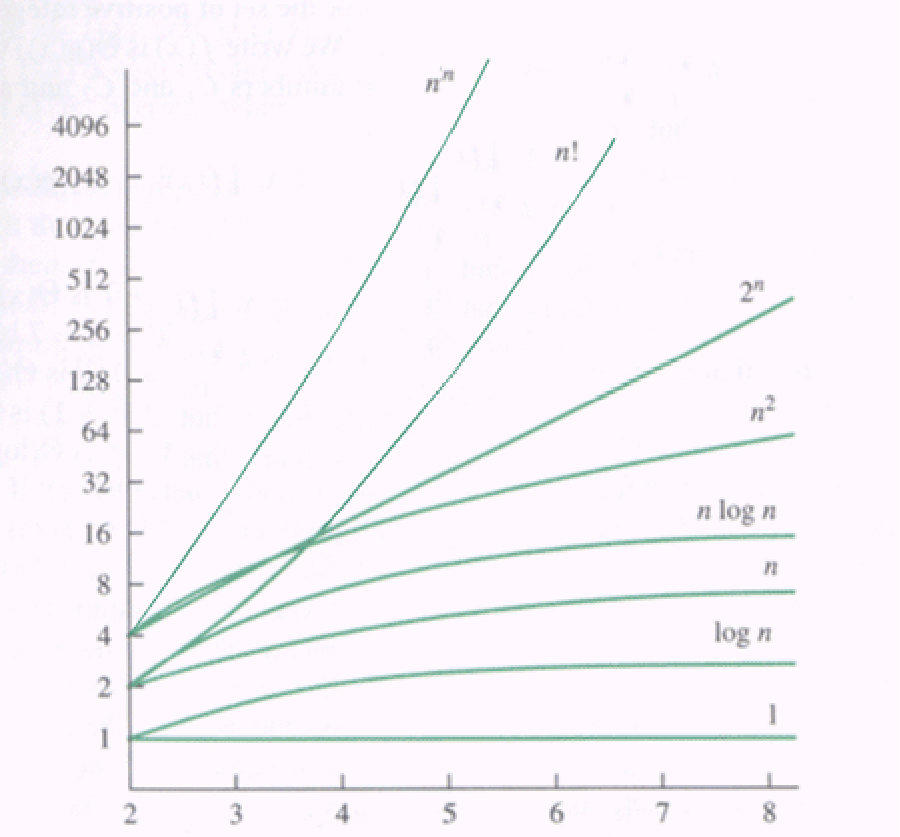
Function	Name
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	N log N
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Summary of Big - Oh

Sometimes, it is very necessary to compare the order of some common used functions including the following:

1 log$n$ n $n$log$n$ n^2 2^n $n!$ n^n

Now, we can use what we've learned above about the concept of big-Oh and the calculation methods to calculate the order of these functions. The result shows that each function in the above list is big-oh of the functions following them. The figure below displays the graphs of these functions, using a scale for the values of the functions that doubles for each successive marking on the graph.



The most common complexities are :

O(1) - not proportional to any variable number, i.e. a fixed/constant amount of time

O(N) - proportional to the size of N (this includes a loop to N and loops to constant multiples of N such as 0.5N, 2N, 2000N - no matter what that is, if you double N you expect (on average) the program to take twice as long)

O(N^2) - proportional to N squared (you double N, you expect it to take four times longer - usually two nested loops both dependent on N).

O(log N) - this is tricker to show - usually the result of binary splitting , binary search - if you half the data at each stage then the number of stages until you have a single item is given (roughly) by log2 N. So binary search takes log2 N time to find an item. All logs grow a constant

amount apart (log2 N = log10 N / log2 N and log2 N is constant) so on big oh notation we normally just say log N not log2 N. Log N grows very slowly - try doing log of 10, 100, 100000, 10000000 on your calculator - use "log" which is log10 on most calculators.

O(N log N) this is merge sort and some other algorithms, usually caused by doing log N splits but also doing N amount of work at each "layer" of splitting.

Lecture 28:

Overview of previous topics and implementation of following practical problems:

(Introductory concepts of pointers also, if not done earlier)

1. Write a menu driven program to insert only even elements inside a Stack.
2. Write a menu driven program to delete only odd elements from a Queue.
3. Write a program to print the sum of all the elements inside a Stack.
4. Write a program to match parenthesis in an expression entered by the user.
5. Write a program to convert an expression from infix to postfix notation.
6. Write a program to sort the elements inside a Stack.
7. Write a program to accept an array from the user and delete an element.
8. Write a program to generate a linked list of prime numbers.
9. Design a method for keeping two stacks within a single array so that neither stack overflows until all of memory is used and an entire stack is never shifted to a different location within the array. Write C functions push1, push2, pop1, and pop2 to manipulate the two stacks. (Hint: Two stacks grow towards each other.)
10. Write a program that will reverse a linked list while traversing it only once. At the conclusion each node should point to the node that was previously its predecessors, the head should point to the node that was formerly at the end and the node that was formerly first should have a NULL link.

Lecture 29: Seminar- Complexity factor/ Applications of Data Structures/ Advanced Data Structures